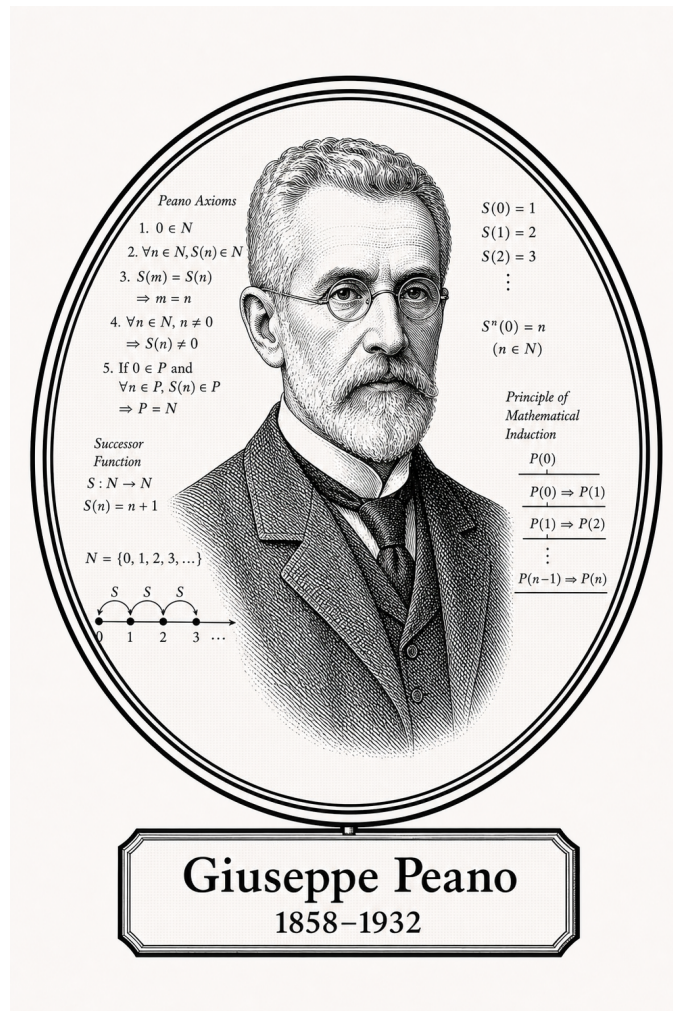


FROM CANTOR TO ITO

# Origins of Numbers

Discrete Number Systems



Giuseppe Peano

1858-1932

*Self-Study Notes*

William Sollers

July 1, 2026

*For every reader learning to make mathematics their own.*

# Contents

|          |                                               |            |
|----------|-----------------------------------------------|------------|
| <b>1</b> | <b>Formalizing the Number Systems</b>         | <b>1</b>   |
| <b>2</b> | <b>Identity, Equality, and Equivalence</b>    | <b>3</b>   |
| 2.1      | Identity, Equality, and Equivalence . . . . . | 3          |
| 2.2      | Equality . . . . .                            | 3          |
| 2.3      | Properties of Equality . . . . .              | 4          |
| 2.4      | Substitution . . . . .                        | 6          |
| 2.5      | Equivalence . . . . .                         | 9          |
| <b>3</b> | <b>Arithmetic Operations and Relations</b>    | <b>46</b>  |
| 3.1      | Arithmetic Operations and Relations . . . . . | 46         |
| 3.2      | Closure . . . . .                             | 46         |
| 3.3      | Identity Elements . . . . .                   | 49         |
| 3.4      | Inverse Operations . . . . .                  | 50         |
| 3.5      | Algebraic Laws . . . . .                      | 53         |
| 3.6      | Derived Laws . . . . .                        | 56         |
| 3.7      | Order Relations . . . . .                     | 58         |
| 3.8      | Order Compatibility . . . . .                 | 60         |
| 3.9      | Composed Relations . . . . .                  | 64         |
| 3.10     | Structure-Preserving Maps . . . . .           | 65         |
| 3.11     | Algebraic Structures . . . . .                | 66         |
| 3.12     | Field and Ordered-Field Laws . . . . .        | 66         |
| <b>4</b> | <b>Constructing a Number System</b>           | <b>122</b> |
| 4.1      | Ground Rules . . . . .                        | 122        |
| 4.2      | Finite Modular Number Systems . . . . .       | 123        |
| 4.3      | The $\{0, 1\}$ Number System . . . . .        | 127        |
| <b>5</b> | <b>Peano Systems</b>                          | <b>137</b> |
| 5.1      | Presburger Arithmetic . . . . .               | 137        |
| 5.2      | Defining Peano Systems . . . . .              | 138        |
| 5.3      | Examples of Peano Systems . . . . .           | 160        |
| 5.4      | Recursion on Peano Systems . . . . .          | 164        |

|          |                                                  |            |
|----------|--------------------------------------------------|------------|
| <b>6</b> | <b>Natural Numbers (<math>\mathbb{N}</math>)</b> | <b>210</b> |
| 6.1      | Axioms for $\mathbb{N}$                          | 210        |
| 6.1.1    | Axioms for $\mathbb{N}$                          | 210        |
| 6.2      | Canonical Natural Number System                  | 211        |
| 6.3      | Addition on $\mathbb{N}$                         | 212        |
| 6.4      | Multiplication on $\mathbb{N}$                   | 219        |
| 6.5      | Order on $\mathbb{N}$                            | 228        |
| 6.6      | Powers and Growth                                | 236        |
| 6.7      | Well Ordering Principle                          | 241        |
| 6.8      | Induction                                        | 244        |
| 6.9      | Algebraic Structure of $\mathbb{N}$              | 244        |
| <b>7</b> | <b>Constructing the Whole Numbers</b>            | <b>294</b> |
| 7.1      | Why Whole Numbers Are Introduced                 | 294        |
| 7.2      | Definition of the Whole Numbers                  | 295        |
| 7.3      | Extending Successor                              | 296        |
| 7.4      | Extending Addition to $\mathbb{W}$               | 298        |
| 7.4.1    | Extending Addition to $\mathbb{W}$               | 298        |
| 7.5      | Extending Multiplication to $\mathbb{W}$         | 302        |
| 7.5.1    | Extending Multiplication to $\mathbb{W}$         | 302        |
| 7.6      | Order on $\mathbb{W}$                            | 306        |
| 7.6.1    | Order on $\mathbb{W}$                            | 306        |
| 7.7      | Algebraic Structure of $\mathbb{W}$              | 310        |
| 7.7.1    | Algebraic Structure of $\mathbb{W}$              | 310        |
| 7.8      | Transition to $\mathbb{Z}$                       | 313        |
| 7.8.1    | Transition to $\mathbb{Z}$                       | 313        |
| <b>8</b> | <b>Integers (<math>\mathbb{Z}</math>)</b>        | <b>353</b> |
| 8.1      | Construction of $\mathbb{Z}$                     | 353        |
| 8.2      | Operations on $\mathbb{Z}$ -Classes              | 357        |
| 8.3      | Algebraic Structure of $\mathbb{Z}$              | 365        |
| 8.4      | Order on $\mathbb{Z}$                            | 367        |
| 8.5      | Embedding $\mathbb{W}$ into $\mathbb{Z}$         | 379        |
| 8.6      | Divisibility in $\mathbb{Z}$                     | 381        |
| 8.7      | Transition to $\mathbb{Q}$                       | 385        |

# Chapter 1

## Formalizing the Number Systems



### Why Formalization Is Needed

Ordinary arithmetic is one of the most familiar parts of mathematics. We learn to count, add, subtract, multiply, and divide long before we ask what sort of objects numbers are or why the usual rules are justified. For foundational work, however, intuition is not enough. The basic number systems must be specified by precise axioms and constructions, so that their operations and order relations are not merely assumed but accounted for.

### The Natural-Number Structure

The first place where this becomes visible is the natural numbers. A foundational treatment cannot simply point to the counting numbers and begin calculating. It has to identify a first element, a successor operation, and the induction principle that makes recursive definitions and proofs possible. Peano's role was to isolate this structure: a first element, a successor map, conditions saying that successor is injective and never returns to the first element, and an induction principle strong enough to characterize the system. This is the role of Peano's 1889 presentation of arithmetic [[Peano, 1889](#)].

### Structural Characterization

Dedekind's work emphasized the same structural point of view. What matters is not the particular names attached to the elements, but the pattern determined by a distinguished first element, successor, recursion, and induction. This perspective underlies the treatment of Peano systems earlier in the volume, and Dedekind's name also sits behind the cut-based construction of the real numbers developed later in the usual foundational tradition [[Dedekind, 1901](#)].

## Later Number Systems

Once the natural numbers have been fixed, the later systems require their own explicit constructions. The integers are built so that subtraction becomes an internal operation. The rational numbers are built so that division by nonzero quantities is represented inside the system. The real numbers are then introduced to address the completeness failures of the rationals. In each case, the construction supplies the operations, the order, and the structural properties needed for analysis, rather than treating them as background facts.

## Proofs

## Capstone

# Chapter 2

## Identity, Equality, and Equivalence



### 2.1 Identity, Equality, and Equivalence

### 2.2 Equality

#### Definition (Identity)

**Definition 2.2.1** (Identity). Two expressions  $x$  and  $y$  are *identical* when they denote the same object.

*Remark* (Standard quantified statement).

$$x = y \iff x \text{ and } y \text{ denote the same object.}$$

*Remark* (Interpretation). Identity is sameness, not similarity. It is the relation used when no distinction remains between two denotations.

*Remark* (Dependencies).

- [Relation](#)

#### Axiom (Leibniz's Law)

**Axiom 2.2.2** (Leibniz's Law). Identical objects have exactly the same properties.

$$\forall x \forall y (x = y \implies \forall P (P(x) \iff P(y))).$$

*Remark* (Standard quantified statement).

$$\text{Indiscernible}(x, y) \iff \forall P (P(x) \iff P(y)).$$



*Remark* (Interpretation). Leibniz’s law says that equality permits no observable mathematical difference. Any property true of one equal object is true of the other.

#### Definition (Equality on a Domain)

**Definition 2.2.3** (Equality on a Domain). Let  $A$  be a domain. *Equality on  $A$*  is the relation, written  $=$ , that holds exactly when two terms or constructions denote the same element of  $A$ .

*Remark* (Standard quantified statement).

$$\text{Equality}_A(=) \iff \forall x, y \in A, (x = y \iff x \text{ and } y \text{ denote the same element of } A).$$

*Remark* (Interpretation). Equality is the identity relation restricted to the domain currently under discussion. Later equivalence relations may identify objects for a purpose, but equality records literal sameness.

*Remark* (Dependencies).

- [Relation](#)

#### Definition (Definitional and Propositional Equality)

**Definition 2.2.4** (Definitional and Propositional Equality). *Definitional equality* records an introduced meaning, written with  $:=$ . *Propositional equality* is a statement of sameness inside the theory, written with  $=$ .

*Remark* (Standard quantified statement).

$$a := b \text{ introduces notation, while } a = b \text{ asserts equality.}$$

*Remark* (Interpretation). The distinction prevents definitions from being confused with theorems. Definitions create abbreviations; propositional equalities must be available as axioms, assumptions, or proved statements.

## 2.3 Properties of Equality

#### Axiom (E1, Reflexivity of Equality)

**Axiom 2.3.1** (Reflexivity of Equality).

$$\forall x, x = x.$$

*Remark* (Standard quantified statement).

$$\text{Reflexive}(=) \iff \forall x, x = x.$$

*Remark* (Interpretation). Every object is itself. This is the primitive equality axiom used before symmetry, transitivity, or arithmetic structure is derived.

### Theorem (E2, Symmetry of Equality)

**Theorem 2.3.2** (Symmetry of Equality).

$$\forall x \forall y, (x = y \implies y = x).$$

*Remark* (Standard quantified statement).

$$\text{Symmetric}(=) \iff \forall x \forall y, (x = y \implies y = x).$$

*Remark* (Interpretation). If  $x$  is the same object as  $y$ , then  $y$  is the same object as  $x$ . Symmetry follows from Leibniz's law by substituting into the property  $P(t) \equiv (t = x)$  and using reflexivity.

*Remark* (Dependencies).

- [Leibniz's Law](#)
- [Reflexivity of Equality](#)

### Theorem (E3, Transitivity of Equality)

**Theorem 2.3.3** (Transitivity of Equality).

$$\forall x \forall y \forall z, (x = y \wedge y = z \implies x = z).$$

*Remark* (Standard quantified statement).

$$\text{Transitive}(=) \iff \forall x \forall y \forall z, (x = y \wedge y = z \implies x = z).$$

*Remark* (Interpretation). Sameness chains: if the first object is the second and the second is the third, then the first is the third.

*Remark* (Dependencies).

- [Leibniz's Law](#)
- [Reflexivity of Equality](#)
- [Symmetry of Equality](#)

### Corollary (Equality Is an Equivalence Relation)

**Corollary 2.3.4** (Equality Is an Equivalence Relation). *The equality relation is reflexive, symmetric, and transitive.*

*Remark* (Standard quantified statement).

$$\text{EquivalenceRelation}(=) \iff \text{Reflexive}(=) \wedge \text{Symmetric}(=) \wedge \text{Transitive}(=).$$

*Remark* (Interpretation). Equality is the strictest equivalence relation: it groups only objects that are literally the same.

*Remark* (Dependencies).

- [Reflexivity of Equality](#)
- [Symmetry of Equality](#)
- [Transitivity of Equality](#)

## 2.4 Substitution

### Axiom (Substitution of Equals)

**Axiom 2.4.1** (Substitution of Equals). For every formula  $\varphi(z)$  whose substitution is admissible,

$$x = y \implies (\varphi(x) \iff \varphi(y)).$$

*Remark* (Standard quantified statement).

$$\forall x \forall y (x = y \implies (\varphi(x) \iff \varphi(y))).$$

*Remark* (Interpretation). Substitution is the operational meaning of equality. Equal objects may replace one another in legitimate statements without changing truth.

### Proposition (Substitution Preserves Predicates)

**Proposition 2.4.2** (Substitution Preserves Predicates).

$$x = y \implies (P(x) \iff P(y)).$$

*Remark* (Standard quantified statement).

$$\forall x \forall y, x = y \implies (P(x) \iff P(y)).$$

*Remark* (Interpretation). Predicates cannot distinguish equal arguments.

*Remark* (Dependencies).

- [Substitution of Equals](#)

### Proposition (Substitution Preserves Relations on the Left)

**Proposition 2.4.3** (Substitution Preserves Relations on the Left).

$$x = y \implies (R(x, z) \iff R(y, z)).$$

*Remark* (Standard quantified statement).

$$\forall x \forall y \forall z, x = y \implies (R(x, z) \iff R(y, z)).$$

*Remark* (Interpretation). Relations respect equality in their first coordinate.

*Remark* (Dependencies).

- [Substitution of Equals](#)

**Proposition (Substitution Preserves Relations on the Right)**

**Proposition 2.4.4** (Substitution Preserves Relations on the Right).

$$x = y \implies (R(z, x) \iff R(z, y)).$$

*Remark* (Standard quantified statement).

$$\forall x \forall y \forall z, x = y \implies (R(z, x) \iff R(z, y)).$$

*Remark* (Interpretation). Relations respect equality in their second coordinate.

*Remark* (Dependencies).

- [Substitution of Equals](#)

**Proposition (Substitution Preserves Relations)**

**Proposition 2.4.5** (Substitution Preserves Relations).

$$x = x' \wedge y = y' \implies (R(x, y) \iff R(x', y')).$$

*Remark* (Standard quantified statement).

$$\forall x, x', y, y', (x = x' \wedge y = y') \implies (R(x, y) \iff R(x', y')).$$

*Remark* (Interpretation). Relations are congruent with respect to equality in all displayed arguments.

*Remark* (Dependencies).

- [Substitution Preserves Relations on the Left](#)
- [Substitution Preserves Relations on the Right](#)

**Proposition (Substitution Preserves Functions)**

**Proposition 2.4.6** (Substitution Preserves Functions).

$$x = y \implies f(x) = f(y).$$

*Remark* (Standard quantified statement).

$$\forall x \forall y, x = y \implies f(x) = f(y).$$

*Remark* (Interpretation). Functions send equal inputs to equal outputs.

*Remark* (Dependencies).

- [Substitution of Equals](#)

#### Proposition (Left Congruence for Operations)

**Proposition 2.4.7** (Left Congruence for Operations).

$$x = x' \implies x O y = x' O y.$$

*Remark* (Standard quantified statement).

$$\forall x, x', y, x = x' \implies x O y = x' O y.$$

*Remark* (Interpretation). A binary operation respects equality in its left argument.

*Remark* (Dependencies).

- [Substitution Preserves Functions](#)

#### Proposition (Right Congruence for Operations)

**Proposition 2.4.8** (Right Congruence for Operations).

$$y = y' \implies x O y = x O y'.$$

*Remark* (Standard quantified statement).

$$\forall x, y, y', y = y' \implies x O y = x O y'.$$

*Remark* (Interpretation). A binary operation respects equality in its right argument.

*Remark* (Dependencies).

- [Substitution Preserves Functions](#)

#### Proposition (Full Congruence for Operations)

**Proposition 2.4.9** (Full Congruence for Operations).

$$x = x' \wedge y = y' \implies x O y = x' O y'.$$

*Remark* (Standard quantified statement).

$$\forall x, x', y, y', (x = x' \wedge y = y') \implies x O y = x' O y'.$$

*Remark* (Interpretation). Binary operations are congruent with respect to equality in both arguments.

*Remark* (Dependencies).

- [Left Congruence for Operations](#)
- [Right Congruence for Operations](#)

#### Proposition (Congruence with Respect to Equality Is Automatic)

**Proposition 2.4.10** (Congruence with Respect to Equality Is Automatic). *Every function, predicate, relation, and operation respects equality in its displayed arguments.*

*Remark* (Standard quantified statement).

$$\begin{aligned} x = y &\implies f(x) = f(y), \\ x = y &\implies (P(x) \iff P(y)), \\ x = x' \wedge y = y' &\implies (R(x, y) \iff R(x', y')), \\ x = x' \wedge y = y' &\implies x O y = x' O y'. \end{aligned}$$

*Remark* (Interpretation). Respecting equality is automatic. Respecting a nontrivial equivalence relation is not automatic; that is why quotient constructions must prove separate respect lemmas before operations, predicates, or relations descend to classes.

*Remark* (Dependencies).

- [Substitution Preserves Predicates](#)
- [Substitution Preserves Relations](#)
- [Substitution Preserves Functions](#)
- [Full Congruence for Operations](#)

## 2.5 Equivalence

#### Definition (Equivalence Relation)

**Definition 2.5.1** (Equivalence Relation). Let  $A$  be a set. A relation  $\sim$  on  $A$  is an *equivalence relation* if it is reflexive, symmetric, and transitive.

*Remark* (Standard quantified statement).

$$\text{EquivalenceRelation}_A(\sim) \iff \left( \forall x \in A, x \sim x \right) \wedge \left( \forall x, y \in A, x \sim y \implies y \sim x \right) \wedge \left( \forall x, y, z \in A, (x \sim y \wedge y \sim z) \implies x \sim z \right)$$

*Remark* (Interpretation). An equivalence relation formalizes sameness for a chosen purpose.

*Remark* (Dependencies).

- [Relation](#)
- [Reflexive Relation](#)
- [Symmetric Relation](#)
- [Transitive Relation](#)

**Corollary (Equality Is the Finest Equivalence)**

**Corollary 2.5.2** (Equality Is the Finest Equivalence). *Let  $\sim$  be an equivalence relation on  $A$ . For all  $a, b \in A$ ,*

$$a = b \implies a \sim b.$$

*Remark* (Standard quantified statement).

$$\forall a, b \in A, a = b \implies a \sim b.$$

*Remark* (Interpretation). Equality is the finest equivalence relation because it identifies only objects that are already the same. Any coarser equivalence relation must at least identify equal representatives.

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Reflexivity of Equality](#)

**Definition (Equivalence Class)**

**Definition 2.5.3** (Equivalence Class). Let  $\sim$  be an equivalence relation on  $A$ . The *equivalence class* of  $a \in A$  is

$$[a] := \{x \in A \mid x \sim a\}.$$

*Remark* (Standard quantified statement).

$$x \in [a] \iff x \sim a.$$

*Remark* (Interpretation). The class  $[a]$  collects all elements that are equivalent to the representative  $a$ .

*Remark* (Dependencies).

- [Equivalence Relation](#)

### Definition (Quotient Set)

**Definition** (Quotient Set). The *quotient set* of  $A$  by  $\sim$  is the set of equivalence classes

$$A/\sim := \{[a] \mid a \in A\}.$$

### Theorem (Equivalence Classes Partition the Domain)

**Theorem 2.5.4** (Equivalence Classes Partition the Domain). *The equivalence classes of an equivalence relation on  $A$  are nonempty, cover  $A$ , and are pairwise disjoint.*

*Remark* (Standard quantified statement).

$$\forall a \in A, a \in [a], \quad A = \bigcup_{a \in A} [a], \quad [a] \cap [b] \neq \emptyset \implies [a] = [b].$$

*Remark* (Interpretation). Equivalence classes divide the domain into non-overlapping blocks.

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Equivalence Class](#)

### Proposition (Representatives Related iff Classes Equal)

**Proposition 2.5.5** (Representatives Related iff Classes Equal).

$$a \sim b \iff [a] = [b].$$

*Remark* (Standard quantified statement).

$$\forall a, b \in A, a \sim b \iff [a] = [b].$$

*Remark* (Interpretation). Changing representatives inside an equivalence class does not change the class.

*Remark* (Dependencies).

- [Equivalence Classes Partition the Domain](#)

## Canonical Projection

### Definition (Canonical Projection)

**Definition 2.5.6** (Canonical Projection). Let  $\sim$  be an equivalence relation on  $A$ . The *canonical projection* is the map

$$\pi : A \rightarrow A/\sim, \quad \pi(a) := [a].$$

*Remark* (Dependencies).



- [Quotient Set](#)
- [Equivalence Class](#)

#### Proposition (Projection Is Surjective)

**Proposition 2.5.7** (Projection Is Surjective). *The canonical projection  $\pi : A \rightarrow A/\sim$  is surjective.*

*Remark* (Standard quantified statement).

$$\forall C \in A/\sim \exists a \in A, \pi(a) = C.$$

*Remark* (Dependencies).

- [Canonical Projection](#)
- [Quotient Set](#)

#### Proposition (Projection Identifies Exactly the Equivalent Elements)

**Proposition 2.5.8** (Projection Identifies Exactly the Equivalent Elements). *For all  $a, b \in A$ ,*

$$\pi(a) = \pi(b) \iff a \sim b.$$

*Remark* (Dependencies).

- [Canonical Projection](#)
- [Representatives Related iff Classes Equal](#)

#### Theorem (Existence of a Quotient)

**Theorem 2.5.9** (Existence of a Quotient). *For any equivalence relation  $\sim$  on  $A$ , the quotient set  $A/\sim$  exists as the set of equivalence classes, and the canonical projection*

$$\pi : A \rightarrow A/\sim$$

*is well-defined.*

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Quotient Set](#)
- [Canonical Projection](#)

## Induced Structure on a Quotient

### Definition (Operation Respects an Equivalence)

**Definition 2.5.10** (Operation Respects an Equivalence). Let  $O : A \times A \rightarrow A$  be a binary operation and let  $\sim$  be an equivalence relation on  $A$ . The operation  $O$  *respects*  $\sim$  if

$$\forall a, a', b, b' \in A, (a \sim a' \wedge b \sim b') \implies (a O b) \sim (a' O b').$$

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Binary Arithmetic Operation](#)
- [Cartesian Product](#)

### Definition (Left and Right Respect)

**Definition 2.5.11** (Left and Right Respect). The operation  $O$  *respects*  $\sim$  on the left if

$$a \sim a' \implies (a O b) \sim (a' O b),$$

and respects  $\sim$  on the right if

$$b \sim b' \implies (a O b) \sim (a O b').$$

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Equivalence Relation](#)

### Lemma (Respect Splits)

**Lemma 2.5.12** (Respect Splits). *A binary operation  $O$  respects  $\sim$  if and only if it respects  $\sim$  on the left and on the right.*

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Left and Right Respect](#)

### Theorem (Induced Operation on a Quotient)

**Theorem 2.5.13** (Induced Operation on a Quotient). *If  $O : A \times A \rightarrow A$  respects  $\sim$ , then*

$$[a] \overline{O} [b] := [a O b]$$

*defines a well-defined binary operation on  $A/\sim$ .*

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Representatives Related iff Classes Equal](#)
- [Existence of a Quotient](#)

#### Theorem (Induced Unary Operation)

**Theorem 2.5.14** (Induced Unary Operation). *Let  $f : A \rightarrow A$ . If*

$$a \sim a' \implies f(a) \sim f(a'),$$

*then*

$$\bar{f}([a]) := [f(a)]$$

*defines a well-defined unary operation on  $A/\sim$ .*

*Remark* (Dependencies).

- [Representatives Related iff Classes Equal](#)
- [Existence of a Quotient](#)

#### Definition (Predicate Respects an Equivalence)

**Definition 2.5.15** (Predicate Respects an Equivalence). Let  $P$  be a predicate on  $A$ . The predicate  $P$  *respects*  $\sim$  if

$$\forall a, a' \in A, a \sim a' \implies (P(a) \iff P(a')).$$

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Predicate](#)

#### Theorem (Induced Predicate on a Quotient)

**Theorem 2.5.16** (Induced Predicate on a Quotient). *If  $P$  respects  $\sim$ , then*

$$\bar{P}([a]) :\iff P(a)$$

*defines a well-defined predicate on  $A/\sim$ .*

*Remark* (Dependencies).

- [Predicate Respects an Equivalence](#)
- [Representatives Related iff Classes Equal](#)

### Definition (Relation Respects an Equivalence)

**Definition 2.5.17** (Relation Respects an Equivalence). Let  $R$  be a binary relation on  $A$ . The relation  $R$  *respects*  $\sim$  if

$$\forall a, a', b, b' \in A, (a \sim a' \wedge b \sim b') \implies (R(a, b) \iff R(a', b')).$$

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Relation](#)

### Theorem (Induced Relation on a Quotient)

**Theorem 2.5.18** (Induced Relation on a Quotient). If  $R$  respects  $\sim$ , then

$$\overline{R}([a], [b]) :\iff R(a, b)$$

defines a well-defined binary relation on  $A/\sim$ .

*Remark* (Dependencies).

- [Relation Respects an Equivalence](#)
- [Representatives Related iff Classes Equal](#)

### Theorem (Induced Map out of a Quotient)

**Theorem 2.5.19** (Induced Map out of a Quotient). Let  $g : A \rightarrow B$ . If

$$a \sim a' \implies g(a) = g(a'),$$

then there is a unique well-defined map

$$\overline{g} : A/\sim \rightarrow B$$

such that

$$\overline{g}([a]) = g(a) \quad \text{and} \quad \overline{g} \circ \pi = g,$$

where  $\pi : A \rightarrow A/\sim$  is the canonical projection.

*Remark* (Dependencies).

- [Canonical Projection](#)
- [Projection Identifies Exactly the Equivalent Elements](#)
- [Projection Is Surjective](#)

### Theorem (Induced Partial Operation on a Quotient)

**Theorem 2.5.20** (Induced Partial Operation on a Quotient). *Let  $D \subseteq A$  be saturated under  $\sim$ , meaning*

$$a \in D \wedge a \sim a' \implies a' \in D.$$

*If  $f : D \rightarrow A$  respects  $\sim$  on  $D$ , then*

$$\bar{f}([a]) := [f(a)]$$

*is a well-defined partial operation on  $A/\sim$  with domain*

$$D/\sim := \{[a] \in A/\sim : a \in D\}.$$

*Remark* (Dependencies).

- [Induced Unary Operation](#)
- [Induced Predicate on a Quotient](#)

### Proposition (Representative Independence Is Necessary)

**Proposition 2.5.21** (Representative Independence Is Necessary). *If a rule on classes*

$$[a] \bar{O} [b] := [a O b]$$

*is well-defined on  $A/\sim$ , then  $O$  respects  $\sim$ .*

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Projection Identifies Exactly the Equivalent Elements](#)
- [Induced Operation on a Quotient](#)

### Lemma (Distinct Equivalence Classes Are Disjoint)

**Lemma 2.5.22** (Distinct Equivalence Classes Are Disjoint).

$$[a] \neq [b] \implies [a] \cap [b] = \emptyset.$$

*Remark* (Standard quantified statement).

$$\forall a, b \in A, [a] \neq [b] \implies [a] \cap [b] = \emptyset.$$

*Remark* (Interpretation). Two equivalence classes either coincide or do not overlap.

*Remark* (Dependencies).

- [Equivalence Classes Partition the Domain](#)

# Proofs

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Symmetry of Equality).

$$\forall x \forall y, (x = y \implies y = x).$$

*Professional Standard Proof.*

TODO: professional standard proof for equality-symmetry. □

*Detailed Learning Proof.*

TODO: detailed learning proof for equality-symmetry. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Leibniz's Law](#)
- [Reflexivity of Equality](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Transitivity of Equality).

$$\forall x \forall y \forall z, (x = y \wedge y = z \implies x = z).$$

*Professional Standard Proof.*

TODO: professional standard proof for equality-transitivity. □

*Detailed Learning Proof.*

TODO: detailed learning proof for equality-transitivity. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Leibniz's Law](#)
- [Reflexivity of Equality](#)
- [Symmetry of Equality](#)



*Remark* (Return). [Return to Theorem](#)

**Corollary** (Equality Is an Equivalence Relation). *The equality relation is reflexive, symmetric, and transitive.*

*Professional Standard Proof.*

TODO: professional standard proof for equality-is-equivalence-relation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for equality-is-equivalence-relation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Reflexivity of Equality](#)
- [Symmetry of Equality](#)
- [Transitivity of Equality](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Substitution Preserves Predicates).

$$x = y \implies (P(x) \iff P(y)).$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-predicates. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-predicates. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution of Equals](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Substitution Preserves Relations on the Left).

$$x = y \implies (R(x, z) \iff R(y, z)).$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-relations-left. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-relations-left. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution of Equals](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Substitution Preserves Relations on the Right).

$$x = y \implies (R(z, x) \iff R(z, y)).$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-relations-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-relations-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution of Equals](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Substitution Preserves Relations).

$$x = x' \wedge y = y' \implies (R(x, y) \iff R(x', y')).$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-relations-full. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-relations-full. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution Preserves Relations on the Left](#)
- [Substitution Preserves Relations on the Right](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Substitution Preserves Functions).

$$x = y \implies f(x) = f(y).$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-functions. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-functions. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution of Equals](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Left Congruence for Operations).

$$x = x' \implies x \mathbin{O} y = x' \mathbin{O} y.$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-operations-left. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-operations-left. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution Preserves Functions](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Right Congruence for Operations).

$$y = y' \implies x \mathbin{O} y = x \mathbin{O} y'.$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-operations-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-operations-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution Preserves Functions](#)



*Remark* (Return). [Return to Theorem](#)

**Proposition** (Full Congruence for Operations).

$$x = x' \wedge y = y' \implies x \mathbin{O} y = x' \mathbin{O} y'.$$

*Professional Standard Proof.*

TODO: professional standard proof for substitution-preserves-operations-full. □

*Detailed Learning Proof.*

TODO: detailed learning proof for substitution-preserves-operations-full. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Congruence for Operations](#)
- [Right Congruence for Operations](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Congruence with Respect to Equality Is Automatic). *Every function, predicate, relation, and operation respects equality in its displayed arguments.*

*Professional Standard Proof.*

TODO: professional standard proof for congruence-with-respect-to-equality-is-automatic.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for congruence-with-respect-to-equality-is-automatic.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Substitution Preserves Predicates](#)
- [Substitution Preserves Relations](#)
- [Substitution Preserves Functions](#)
- [Full Congruence for Operations](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Equality Is the Finest Equivalence). *Let  $\sim$  be an equivalence relation on  $A$ . For all  $a, b \in A$ ,*

$$a = b \implies a \sim b.$$

*Professional Standard Proof.*

TODO: professional standard proof for equality-is-finest-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for equality-is-finest-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Reflexivity of Equality](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Equivalence Classes Partition the Domain). *The equivalence classes of an equivalence relation on  $A$  are nonempty, cover  $A$ , and are pairwise disjoint.*

*Professional Standard Proof.*

TODO: professional standard proof for equivalence-classes-partition-domain. □

*Detailed Learning Proof.*

TODO: detailed learning proof for equivalence-classes-partition-domain. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Equivalence Class](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Representatives Related iff Classes Equal).

$$a \sim b \iff [a] = [b].$$

*Professional Standard Proof.*

TODO: professional standard proof for representatives-related-iff-classes-equal. □

*Detailed Learning Proof.*

TODO: detailed learning proof for representatives-related-iff-classes-equal. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Equivalence Classes Partition the Domain](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Projection Is Surjective). *The canonical projection  $\pi : A \rightarrow A/\sim$  is surjective.*

*Professional Standard Proof.*

TODO: professional standard proof for projection-is-surjective. □

*Detailed Learning Proof.*

TODO: detailed learning proof for projection-is-surjective. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Canonical Projection](#)
- [Quotient Set](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Projection Identifies Exactly the Equivalent Elements). *For all  $a, b \in A$ ,*

$$\pi(a) = \pi(b) \iff a \sim b.$$

*Professional Standard Proof.*

TODO: professional standard proof for projection-identifies-equivalent-elements. □

*Detailed Learning Proof.*

TODO: detailed learning proof for projection-identifies-equivalent-elements. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Canonical Projection](#)
- [Representatives Related iff Classes Equal](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Existence of a Quotient). *For any equivalence relation  $\sim$  on  $A$ , the quotient set  $A/\sim$  exists as the set of equivalence classes, and the canonical projection*

$$\pi : A \rightarrow A/\sim$$

*is well-defined.*

*Professional Standard Proof.*

TODO: professional standard proof for existence-of-a-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for existence-of-a-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Equivalence Relation](#)
- [Quotient Set](#)
- [Canonical Projection](#)



*Remark* (Return). [Return to Theorem](#)

**Lemma** (Respect Splits). *A binary operation  $O$  respects  $\sim$  if and only if it respects  $\sim$  on the left and on the right.*

*Professional Standard Proof.*

TODO: professional standard proof for respect-splits. □

*Detailed Learning Proof.*

TODO: detailed learning proof for respect-splits. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Left and Right Respect](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Operation on a Quotient). *If  $O : A \times A \rightarrow A$  respects  $\sim$ , then*

$$[a] \overline{O} [b] := [a O b]$$

*defines a well-defined binary operation on  $A/\sim$ .*

*Professional Standard Proof.*

TODO: professional standard proof for induced-operation-on-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-operation-on-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Representatives Related iff Classes Equal](#)
- [Existence of a Quotient](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Unary Operation). *Let  $f : A \rightarrow A$ . If*

$$a \sim a' \implies f(a) \sim f(a'),$$

*then*

$$\overline{f}([a]) := [f(a)]$$

*defines a well-defined unary operation on  $A/\sim$ .*

*Professional Standard Proof.*

TODO: professional standard proof for induced-unary-operation-on-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-unary-operation-on-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Representatives Related iff Classes Equal](#)
- [Existence of a Quotient](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Predicate on a Quotient). *If  $P$  respects  $\sim$ , then*

$$\overline{P}([a]) :\Longleftrightarrow P(a)$$

*defines a well-defined predicate on  $A/\sim$ .*

*Professional Standard Proof.*

TODO: professional standard proof for induced-predicate-on-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-predicate-on-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Predicate Respects an Equivalence](#)
- [Representatives Related iff Classes Equal](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Relation on a Quotient). *If  $R$  respects  $\sim$ , then*

$$\overline{R}([a], [b]) :\Longleftrightarrow R(a, b)$$

*defines a well-defined binary relation on  $A/\sim$ .*

*Professional Standard Proof.*

TODO: professional standard proof for induced-relation-on-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-relation-on-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Relation Respects an Equivalence](#)
- [Representatives Related iff Classes Equal](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Map out of a Quotient). *Let  $g : A \rightarrow B$ . If*

$$a \sim a' \implies g(a) = g(a'),$$

*then there is a unique well-defined map*

$$\bar{g} : A/\sim \rightarrow B$$

*such that*

$$\bar{g}([a]) = g(a) \quad \text{and} \quad \bar{g} \circ \pi = g,$$

*where  $\pi : A \rightarrow A/\sim$  is the canonical projection.*

*Professional Standard Proof.*

TODO: professional standard proof for induced-map-out-of-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-map-out-of-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Canonical Projection](#)
- [Projection Identifies Exactly the Equivalent Elements](#)
- [Projection Is Surjective](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Partial Operation on a Quotient). *Let  $D \subseteq A$  be saturated under  $\sim$ , meaning*

$$a \in D \wedge a \sim a' \implies a' \in D.$$

*If  $f : D \rightarrow A$  respects  $\sim$  on  $D$ , then*

$$\bar{f}([a]) := [f(a)]$$

*is a well-defined partial operation on  $A/\sim$  with domain*

$$D/\sim := \{[a] \in A/\sim : a \in D\}.$$

*Professional Standard Proof.*

TODO: professional standard proof for induced-partial-operation-on-quotient. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-partial-operation-on-quotient. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Induced Unary Operation](#)
- [Induced Predicate on a Quotient](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Representative Independence Is Necessary). *If a rule on classes*

$$[a] \overline{O} [b] := [a \ O \ b]$$

*is well-defined on  $A/\sim$ , then  $O$  respects  $\sim$ .*

*Professional Standard Proof.*

TODO: professional standard proof for representative-independence-is-necessary. □

*Detailed Learning Proof.*

TODO: detailed learning proof for representative-independence-is-necessary. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Operation Respects an Equivalence](#)
- [Projection Identifies Exactly the Equivalent Elements](#)
- [Induced Operation on a Quotient](#)



*Remark* (Return). [Return to Theorem](#)

**Lemma** (Distinct Equivalence Classes Are Disjoint).

$$[a] \neq [b] \implies [a] \cap [b] = \emptyset.$$

*Professional Standard Proof.*

TODO: professional standard proof for distinct-equivalence-classes-are-disjoint. □

*Detailed Learning Proof.*

TODO: detailed learning proof for distinct-equivalence-classes-are-disjoint. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Equivalence Classes Partition the Domain](#)

# Capstone

# Chapter 3

## Arithmetic Operations and Relations



### 3.1 Arithmetic Operations and Relations

### 3.2 Closure

#### Definition (Unary Arithmetic Operation)

**Definition 3.2.1** (Unary Arithmetic Operation). Let  $K$  be a domain. A *unary arithmetic operation* on  $K$  is a function

$$O : K \rightarrow K.$$

*Remark* (Standard quantified statement).

$$\text{UnaryOperation}_K(O) \iff \forall x \in K, O(x) \in K.$$

*Remark* (Interpretation). Unary closure is the abstract form behind successor, negation, and other single-input transformations.

*Remark* (Dependencies).

- [Function](#)

#### Definition (Binary Arithmetic Operation)

**Definition 3.2.2** (Binary Arithmetic Operation). Let  $K$  be a domain. A *binary arithmetic operation* on  $K$  is a function

$$O : K \times K \rightarrow K.$$

*Remark* (Standard quantified statement).

$$\text{BinaryOperation}_K(O) \iff \forall x, y \in K, x O y \in K.$$

*Remark* (Interpretation). Binary closure is the abstract form behind addition and multiplication.

*Remark* (Dependencies).

- [Function](#)
- [Cartesian Product](#)

#### Definition (Unary Closure)

**Definition 3.2.3** (Unary Closure). A unary operation  $O$  is *closed on  $K$*  if applying it to an element of  $K$  produces an element of  $K$ .

*Remark* (Standard quantified statement).

$$\forall x \in K, O(x) \in K.$$

*Remark* (Interpretation). The operation does not leave the domain.

*Remark* (Dependencies).

- [Unary Arithmetic Operation](#)

#### Definition (Binary Closure)

**Definition 3.2.4** (Binary Closure). A binary operation  $O$  is *closed on  $K$*  if combining two elements of  $K$  produces an element of  $K$ .

*Remark* (Standard quantified statement).

$$\forall x, y \in K, x O y \in K.$$

*Remark* (Interpretation). The operation is internal to the domain.

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Operation Well-Definedness)

**Definition 3.2.5** (Operation Well-Definedness). Let  $\sim$  be an equivalence relation on  $A$ . A binary operation  $O$  respects  $\sim$  if

$$a \sim a' \wedge b \sim b' \implies a O b \sim a' O b'.$$

*Remark* (Standard quantified statement).

$$\forall a, a', b, b' \in A, (a \sim a' \wedge b \sim b') \implies a O b \sim a' O b'.$$

*Remark* (Interpretation). This is the condition that lets an operation descend to equivalence classes.

*Remark* (Dependencies).

- [Equivalence Relation](#)

#### Lemma (Left Respect)

**Lemma 3.2.6** (Left Respect). *If a binary operation respects  $\sim$ , then*

$$a \sim a' \implies a O b \sim a' O b.$$

*Remark* (Standard quantified statement).

$$\forall a, a', b \in A, a \sim a' \implies a O b \sim a' O b.$$

*Remark* (Interpretation). Full respect includes respect in the left coordinate.

*Remark* (Dependencies).

- [Operation Well-Definedness](#)

#### Lemma (Right Respect)

**Lemma 3.2.7** (Right Respect). *If a binary operation respects  $\sim$ , then*

$$b \sim b' \implies a O b \sim a O b'.$$

*Remark* (Standard quantified statement).

$$\forall a, b, b' \in A, b \sim b' \implies a O b \sim a O b'.$$

*Remark* (Interpretation). Full respect includes respect in the right coordinate.

*Remark* (Dependencies).

- [Operation Well-Definedness](#)

#### Theorem (Induced Operation)

**Theorem 3.2.8** (Induced Operation). *The operation*

$$[a] \overline{O} [b] := [a O b]$$

*on  $A/\sim$  is well-defined if and only if  $O$  respects  $\sim$ .*

*Remark* (Standard quantified statement).

$$\text{WellDefined}(\overline{O}) \iff \forall a, a', b, b' \in A, (a \sim a' \wedge b \sim b') \Rightarrow a O b \sim a' O b'.$$

*Remark* (Interpretation). This theorem is the abstract engine behind quotient constructions such as integers and rationals.

*Remark* (Dependencies).

- [Quotient Set](#)
- [Operation Well-Definedness](#)

### 3.3 Identity Elements

#### Definition (Left Identity)

**Definition 3.3.1** (Left Identity).

$$\forall x, e_L O x = x.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Right Identity)

**Definition 3.3.2** (Right Identity).

$$\forall x, x O e_R = x.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Two-Sided Identity)

**Definition 3.3.3** (Two-Sided Identity). An element  $e$  is a *two-sided identity* for  $O$  if it is both a left identity and a right identity.

*Remark* (Dependencies).

- [Left Identity](#)
- [Right Identity](#)

#### Definition (Absorbing Element)

**Definition 3.3.4** (Absorbing Element). An element  $z$  is an *absorbing element* for a binary operation  $O$  if

$$z O x = z \quad \text{and} \quad x O z = z$$

for every element  $x$  in the domain of the operation.

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Lemma (Left and Right Identities Coincide)

**Lemma 3.3.5** (Left and Right Identities Coincide). If  $e_L$  is a left identity and  $e_R$  is a right identity for the same operation, then  $e_L = e_R$ .

*Remark* (Standard quantified statement).

$$(\forall x, e_L O x = x) \wedge (\forall x, x O e_R = x) \implies e_L = e_R.$$

*Remark* (Dependencies).

- [Left Identity](#)
- [Right Identity](#)

#### Theorem (Uniqueness of Identity)

**Theorem 3.3.6** (Uniqueness of Identity). A two-sided identity for a binary operation is unique.

*Remark* (Standard quantified statement).

$$\text{Identity}(e) \wedge \text{Identity}(e') \implies e = e'.$$

*Remark* (Dependencies).

- [Left and Right Identities Coincide](#)

## 3.4 Inverse Operations

#### Definition (Left Inverse)

**Definition 3.4.1** (Left Inverse). An element  $x'$  is a left inverse of  $x$  with respect to identity  $e$  if

$$x' O x = e.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)
- [Two-Sided Identity](#)

#### Definition (Right Inverse)

**Definition 3.4.2** (Right Inverse). An element  $x''$  is a right inverse of  $x$  with respect to identity  $e$  if

$$x O x'' = e.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)
- [Two-Sided Identity](#)

#### Definition (Two-Sided Inverse)

**Definition 3.4.3** (Two-Sided Inverse). An element  $y$  is a two-sided inverse of  $x$  if

$$y O x = e \quad \text{and} \quad x O y = e.$$

*Remark* (Dependencies).

- [Left Inverse](#)
- [Right Inverse](#)

#### Lemma (Left and Right Inverses Coincide)

**Lemma 3.4.4** (Left and Right Inverses Coincide). *In an associative operation with identity, a left inverse and a right inverse of the same element are equal.*

*Remark* (Standard quantified statement).

$$(x' O x = e) \wedge (x O x'' = e) \implies x' = x''.$$

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Left Inverse](#)
- [Right Inverse](#)

#### Theorem (Uniqueness of Inverse)

**Theorem 3.4.5** (Uniqueness of Inverse). *In an associative operation with identity, inverses are unique.*

*Remark* (Dependencies).



- [Left and Right Inverses Coincide](#)

#### Definition (Right Solvability)

**Definition 3.4.6** (Right Solvability). Right solvability means

$$\forall x, y \exists z, x = y O z,$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Left Solvability)

**Definition 3.4.7** (Left Solvability). Left solvability means

$$\forall x, y \exists z, x = z O y.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Two-Sided Solvability)

**Definition 3.4.8** (Two-Sided Solvability). Two-sided solvability means both left solvability and right solvability hold.

*Remark* (Dependencies).

- [Left Solvability](#)
- [Right Solvability](#)

#### Proposition (Solvability and Inverses)

**Proposition 3.4.9** (Solvability and Inverses). *For an associative operation with a two-sided identity, solvability is equivalent to the existence of inverses.*

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Two-Sided Inverse](#)
- [Right Solvability](#)
- [Left Solvability](#)
- [Two-Sided Solvability](#)

## 3.5 Algebraic Laws

### Definition (Associativity)

**Definition 3.5.1** (Associativity).

$$\forall x, y, z, \quad x \, O \, (y \, O \, z) = (x \, O \, y) \, O \, z.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

### Theorem (General Associativity)

**Theorem 3.5.2** (General Associativity). *If  $O$  is associative, then every bracketing of  $x_1 \, O \cdots O x_n$  has the same value.*

*Remark* (Dependencies).

- [Associativity](#)

### Definition (Commutativity)

**Definition 3.5.3** (Commutativity).

$$\forall x, y, \quad x \, O \, y = y \, O \, x.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

### Theorem (General Commutativity)

**Theorem 3.5.4** (General Commutativity). *If  $O$  is associative and commutative, then any finite reordering of  $x_1 \, O \cdots O x_n$  has the same value.*

*Remark* (Dependencies).

- [Associativity](#)
- [Commutativity](#)

### Definition (Left Distributivity)

**Definition 3.5.5** (Left Distributivity). The operation  $\otimes$  is left distributive over  $\oplus$  if

$$x \otimes (y \oplus z) = (x \otimes y) \oplus (x \otimes z),$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Right Distributivity)

**Definition 3.5.6** (Right Distributivity). The operation  $\otimes$  is right distributive over  $\oplus$  if

$$(y \oplus z) \otimes x = (y \otimes x) \oplus (z \otimes x),$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Two-Sided Distributivity)

**Definition 3.5.7** (Two-Sided Distributivity). The operation  $\otimes$  is two-sided distributive over  $\oplus$  if it is both left distributive and right distributive.

*Remark* (Dependencies).

- [Left Distributivity](#)
- [Right Distributivity](#)

#### Definition (Left Cancellation)

**Definition 3.5.8** (Left Cancellation). An operation has left cancellation if

$$x O y = x O z \implies y = z,$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Right Cancellation)

**Definition 3.5.9** (Right Cancellation). An operation has right cancellation if

$$y O x = z O x \implies y = z.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Definition (Two-Sided Cancellation)

**Definition 3.5.10** (Two-Sided Cancellation). An operation has two-sided cancellation if it has both left cancellation and right cancellation.

*Remark* (Dependencies).

- [Left Cancellation](#)
- [Right Cancellation](#)

#### Proposition (Invertibility Implies Left Cancellation)

**Proposition 3.5.11** (Invertibility Implies Left Cancellation). *In an associative operation with identity, invertible elements satisfy left cancellation.*

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Two-Sided Inverse](#)
- [Left Cancellation](#)

#### Proposition (Invertibility Implies Cancellation)

**Proposition 3.5.12** (Invertibility Implies Cancellation). *In an associative operation with identity, invertible elements satisfy two-sided cancellation.*

*Remark* (Dependencies).

- [Invertibility Implies Left Cancellation](#)
- [Invertibility Implies Right Cancellation](#)
- [Two-Sided Cancellation](#)

#### Proposition (Invertibility Implies Right Cancellation)

**Proposition 3.5.13** (Invertibility Implies Right Cancellation). *In an associative operation with identity, invertible elements satisfy right cancellation.*

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Two-Sided Inverse](#)
- [Right Cancellation](#)

## 3.6 Derived Laws

### Proposition (Left Zero Absorption)

**Proposition 3.6.1** (Left Zero Absorption). *When multiplication distributes over addition and 0 is the additive identity,*

$$0 \cdot a = 0.$$

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Two-Sided Identity](#)
- [Absorbing Element](#)

### Proposition (Right Zero Absorption)

**Proposition 3.6.2** (Right Zero Absorption). *When multiplication distributes over addition and 0 is the additive identity,*

$$a \cdot 0 = 0.$$

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Two-Sided Identity](#)
- [Absorbing Element](#)

### Proposition (Zero Absorption)

**Proposition 3.6.3** (Zero Absorption). *When multiplication distributes over addition and 0 is the additive identity,*

$$0 \cdot a = 0 \quad \text{and} \quad a \cdot 0 = 0.$$

*Remark* (Dependencies).

- [Left Zero Absorption](#)
- [Right Zero Absorption](#)

### Proposition (Left Sign Rule)

**Proposition 3.6.4** (Left Sign Rule). *When multiplication distributes over addition and additive inverses exist,*

$$(-a) \cdot b = -(a \cdot b).$$

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Two-Sided Inverse](#)
- [Left Zero Absorption](#)

#### Proposition (Right Sign Rule)

**Proposition 3.6.5** (Right Sign Rule). *When multiplication distributes over addition and additive inverses exist,*

$$a \cdot (-b) = -(a \cdot b).$$

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Two-Sided Inverse](#)
- [Right Zero Absorption](#)

#### Proposition (Sign Rule)

**Proposition 3.6.6** (Sign Rule). *When multiplication distributes over addition and additive inverses exist,*

$$(-a) \cdot b = -(a \cdot b) \quad \text{and} \quad a \cdot (-b) = -(a \cdot b).$$

*Remark* (Dependencies).

- [Left Sign Rule](#)
- [Right Sign Rule](#)

#### Corollary (Negative One Rule)

**Corollary 3.6.7** (Negative One Rule). *When multiplication distributes over addition and additive inverses exist,*

$$(-1) \cdot a = -a.$$

*Remark* (Dependencies).

- [Left Sign Rule](#)

#### Proposition (Product of Negatives)

**Proposition 3.6.8** (Product of Negatives). *When multiplication distributes over addition and additive inverses exist,*

$$(-a) \cdot (-b) = a \cdot b.$$

*Remark* (Dependencies).

- [Left Sign Rule](#)
- [Right Sign Rule](#)

#### Definition (Zero Divisor)

**Definition 3.6.9** (Zero Divisor). Let  $K$  be an arithmetic system with multiplication and zero. A nonzero element  $a \in K$  is a *zero divisor* if there exists a nonzero element  $b \in K$  such that  $ab = 0$ .

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

#### Theorem (No Zero Divisors)

**Theorem 3.6.10** (No Zero Divisors). *In an arithmetic system with no nonzero zero divisors,*

$$a \cdot b = 0 \implies a = 0 \vee b = 0.$$

*Remark* (Dependencies).

- [Binary Arithmetic Operation](#)

## 3.7 Order Relations

#### Definition (Reflexive Relation)

**Definition 3.7.1** (Reflexive Relation).

$$\forall x, xRx.$$

*Remark* (Dependencies).

- [Relation](#)
- [Reflexive Relation](#)

#### Definition (Irreflexive Relation)

**Definition 3.7.2** (Irreflexive Relation).

$$\forall x, \neg(xRx).$$

*Remark* (Dependencies).

- [Relation](#)
- [Irreflexive Relation](#)

#### Definition (Symmetric Relation)

**Definition 3.7.3** (Symmetric Relation).

$$\forall x, y, xRy \implies yRx.$$

*Remark* (Dependencies).

- [Relation](#)
- [Symmetric Relation](#)

#### Definition (Antisymmetric Relation)

**Definition 3.7.4** (Antisymmetric Relation).

$$\forall x, y, (xRy \wedge yRx) \implies x = y.$$

*Remark* (Dependencies).

- [Relation](#)
- [Antisymmetric Relation](#)

#### Definition (Asymmetric Relation)

**Definition 3.7.5** (Asymmetric Relation).

$$\forall x, y, xRy \implies \neg(yRx).$$

*Remark* (Dependencies).

- [Relation](#)
- [Asymmetric Relation](#)

#### Definition (Transitive Relation)

**Definition 3.7.6** (Transitive Relation).

$$\forall x, y, z, (xRy \wedge yRz) \implies xRz.$$

*Remark* (Dependencies).

- [Relation](#)
- [Transitive Relation](#)



#### Definition (Total Relation)

**Definition 3.7.7** (Total Relation).

$$\forall x, y, \quad xRy \vee yRx.$$

*Remark* (Dependencies).

- [Relation](#)
- [Total Relation](#)

#### Definition (Trichotomy)

**Definition 3.7.8** (Trichotomy). The relation  $<$  satisfies *trichotomy* if exactly one of

$$x < y, \quad x = y, \quad y < x$$

holds for every pair  $x, y$ .

*Remark* (Dependencies).

- [Strict Order](#)
- [Total Relation](#)

## 3.8 Order Compatibility

#### Definition (Right Monotony)

**Definition 3.8.1** (Right Monotony). Right monotony means

$$yRz \implies (x \mathbin{O} y)R(x \mathbin{O} z),$$

*Remark* (Dependencies).

- [Relation](#)
- [Binary Arithmetic Operation](#)

#### Definition (Left Monotony)

**Definition 3.8.2** (Left Monotony). Left monotony means

$$yRz \implies (y \mathbin{O} x)R(z \mathbin{O} x).$$

*Remark* (Dependencies).

- [Relation](#)

- [Binary Arithmetic Operation](#)

#### Definition (Two-Sided Monotony)

**Definition 3.8.3** (Two-Sided Monotony). Two-sided monotony means both left monotony and right monotony hold.

*Remark* (Dependencies).

- [Left Monotony](#)
- [Right Monotony](#)

#### Definition (Right Order Reflection)

**Definition 3.8.4** (Right Order Reflection). Right reflection means

$$(x \mathbin{O} y)R(x \mathbin{O} z) \implies yRz,$$

*Remark* (Dependencies).

- [Relation](#)
- [Binary Arithmetic Operation](#)

#### Definition (Left Order Reflection)

**Definition 3.8.5** (Left Order Reflection). Left reflection means

$$(y \mathbin{O} x)R(z \mathbin{O} x) \implies yRz.$$

*Remark* (Dependencies).

- [Relation](#)
- [Binary Arithmetic Operation](#)

#### Definition (Two-Sided Order Reflection)

**Definition 3.8.6** (Two-Sided Order Reflection). Two-sided order reflection means both left order reflection and right order reflection hold.

*Remark* (Dependencies).

- [Left Order Reflection](#)
- [Right Order Reflection](#)

### Definition (Positive Cone)

**Definition 3.8.7** (Positive cone). Let  $K$  be an arithmetic system with addition, multiplication, and zero. A subset  $P \subset K$  is called a *positive cone* if it satisfies:

1. if  $a, b \in P$ , then  $a + b \in P$ ;
2. if  $a, b \in P$ , then  $ab \in P$ ;
3. for every  $a \in K$ , exactly one of the following holds:

$$a \in P, \quad a = 0, \quad -a \in P.$$

The strict order associated to  $P$  is

$$a < b \iff b - a \in P.$$

*Remark* (Dependencies).

- [Subset](#)
- [Trichotomy](#)
- [Binary Arithmetic Operation](#)

### Definition (Monotone Map)

**Definition 3.8.8** (Monotone Map).

$$x \leq y \implies f(x) \leq f(y).$$

*Remark* (Dependencies).

- [Function](#)
- [Relation](#)

### Definition (Strictly Monotone Map)

**Definition 3.8.9** (Strictly Monotone Map).

$$x < y \implies f(x) < f(y).$$

*Remark* (Dependencies).

- [Function](#)
- [Relation](#)

**Proposition (Addition Preserves Order on the Left)**

**Proposition 3.8.10** (Addition Preserves Order on the Left).

$$y < z \implies x + y < x + z$$

*in ordered arithmetic systems where addition is strictly order-preserving.*

*Remark* (Dependencies).

- [Right Monotony](#)

**Proposition (Addition Preserves Order on the Right)**

**Proposition 3.8.11** (Addition Preserves Order on the Right).

$$y < z \implies y + x < z + x$$

*in ordered arithmetic systems where addition is strictly order-preserving.*

*Remark* (Dependencies).

- [Left Monotony](#)

**Proposition (Addition Preserves Order)**

**Proposition 3.8.12** (Addition Preserves Order). *In ordered arithmetic systems where addition is strictly order-preserving, addition preserves order on both sides.*

*Remark* (Dependencies).

- [Addition Preserves Order on the Left](#)
- [Addition Preserves Order on the Right](#)

**Proposition (Multiplication Preserves Order on the Left)**

**Proposition 3.8.13** (Multiplication Preserves Order on the Left).

$$0 < x \wedge y < z \implies x \cdot y < x \cdot z$$

*in ordered arithmetic systems where multiplication by positive elements is strictly order-preserving.*

*Remark* (Dependencies).

- [Right Monotony](#)
- [Strict Total Order](#)

#### Proposition (Multiplication Preserves Order on the Right)

**Proposition 3.8.14** (Multiplication Preserves Order on the Right).

$$0 < x \wedge y < z \implies y \cdot x < z \cdot x$$

*in ordered arithmetic systems where multiplication by positive elements is strictly order-preserving.*

*Remark* (Dependencies).

- [Left Monotony](#)
- [Strict Total Order](#)

#### Proposition (Multiplication Preserves Order)

**Proposition 3.8.15** (Multiplication Preserves Order). *In ordered arithmetic systems where multiplication by positive elements is strictly order-preserving, multiplication preserves order on both sides.*

*Remark* (Dependencies).

- [Multiplication Preserves Order on the Left](#)
- [Multiplication Preserves Order on the Right](#)

## 3.9 Composed Relations

#### Definition (Preorder)

**Definition** (Preorder). A *preorder* is a reflexive and transitive relation.

#### Definition (Partial Order)

**Definition** (Partial Order). A *partial order* is a preorder that is antisymmetric.

#### Definition (Strict Partial Order)

**Definition 3.9.1** (Strict Partial Order). A *strict partial order* is an irreflexive and transitive relation.

*Remark* (Dependencies).

- [Irreflexive Relation](#)
- [Transitive Relation](#)

#### Definition (Total Order)

**Definition** (Total Order). A *total order* is a partial order whose relation is total.

#### Definition (Strict Total Order)

**Definition 3.9.2** (Strict Total Order). A *strict total order* is a strict partial order satisfying trichotomy.

*Remark* (Dependencies).

- [Strict Partial Order](#)
- [Trichotomy](#)

#### Definition (Well-Order)

**Definition** (Well-Order). A *well-order* is a total order in which every nonempty subset has a least element.

## 3.10 Structure-Preserving Maps

#### Definition (Homomorphism)

**Definition 3.10.1** (Homomorphism). A map  $f$  is a *homomorphism* for an operation  $O$  when

$$f(x \ O \ y) = f(x) \ O' \ f(y).$$

For ring-like structures it preserves both addition and multiplication, and preserves 1 when the language includes a distinguished multiplicative identity.

*Remark* (Dependencies).

- [Function](#)
- [Binary Arithmetic Operation](#)

#### Definition (Injective Homomorphism)

**Definition 3.10.2** (Injective Homomorphism). An *injective homomorphism* is a homomorphism whose underlying function is injective.

*Remark* (Dependencies).

- [Homomorphism](#)
- [Injective Function](#)

#### Definition (Order Embedding)

**Definition 3.10.3** (Order Embedding).

$$xRy \iff f(x)R'f(y).$$

*Remark* (Dependencies).

- [Function](#)
- [Relation](#)
- [Order Embedding](#)

#### Definition (Structure Embedding)

**Definition 3.10.4** (Structure Embedding). A *structure embedding* is an injective homomorphism that also embeds the relevant order relation.

*Remark* (Dependencies).

- [Injective Homomorphism](#)
- [Order Embedding](#)

#### Proposition (Composition of Homomorphisms)

**Proposition 3.10.5** (Composition of Homomorphisms). *The composition of two homomorphisms is a homomorphism.*

*Remark* (Dependencies).

- [Homomorphism](#)

#### Corollary (Identity Map Is a Homomorphism)

**Corollary 3.10.6** (Identity Map Is a Homomorphism). *The identity map on a structure is a homomorphism from that structure to itself.*

*Remark* (Dependencies).

- [Homomorphism](#)

## 3.11 Algebraic Structures

## 3.12 Field and Ordered-Field Laws

## Proofs

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Left Respect). *If a binary operation respects  $\sim$ , then*

$$a \sim a' \implies a \mathbin{O} b \sim a' \mathbin{O} b.$$

*Professional Standard Proof.*

TODO: professional standard proof for operation-left-respect. □

*Detailed Learning Proof.*

TODO: detailed learning proof for operation-left-respect. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Operation Well-Definedness](#)



*Remark* (Return). [Return to Theorem](#)

**Lemma** (Right Respect). *If a binary operation respects  $\sim$ , then*

$$b \sim b' \implies a \mathbin{O} b \sim a \mathbin{O} b'.$$

*Professional Standard Proof.*

TODO: professional standard proof for operation-right-respect. □

*Detailed Learning Proof.*

TODO: detailed learning proof for operation-right-respect. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Operation Well-Definedness](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induced Operation). *The operation*

$$[a] \overline{O} [b] := [a \ O \ b]$$

*on  $A/\sim$  is well-defined if and only if  $O$  respects  $\sim$ .*

*Professional Standard Proof.*

TODO: professional standard proof for induced-operation-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induced-operation-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Quotient Set](#)
- [Operation Well-Definedness](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Left and Right Identities Coincide). *If  $e_L$  is a left identity and  $e_R$  is a right identity for the same operation, then  $e_L = e_R$ .*

*Professional Standard Proof.*

TODO: professional standard proof for left-right-identities-coincide. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-right-identities-coincide. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Identity](#)
- [Right Identity](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Uniqueness of Identity). *A two-sided identity for a binary operation is unique.*

*Professional Standard Proof.*

TODO: professional standard proof for uniqueness-of-identity. □

*Detailed Learning Proof.*

TODO: detailed learning proof for uniqueness-of-identity. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left and Right Identities Coincide](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Left and Right Inverses Coincide). *In an associative operation with identity, a left inverse and a right inverse of the same element are equal.*

*Professional Standard Proof.*

TODO: professional standard proof for left-right-inverses-coincide. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-right-inverses-coincide. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Left Inverse](#)
- [Right Inverse](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Uniqueness of Inverse). *In an associative operation with identity, inverses are unique.*

*Professional Standard Proof.*

TODO: professional standard proof for uniqueness-of-inverse. □

*Detailed Learning Proof.*

TODO: detailed learning proof for uniqueness-of-inverse. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left and Right Inverses Coincide](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Solvability and Inverses). *In a monoid, solvability is equivalent to the existence of inverses.*

*Professional Standard Proof.*

TODO: professional standard proof for solvability-iff-inverses. □

*Detailed Learning Proof.*

TODO: detailed learning proof for solvability-iff-inverses. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Identity](#)
- [Two-Sided Inverse](#)
- [Right Solvability](#)
- [Left Solvability](#)
- [Two-Sided Solvability](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (General Associativity). *If  $O$  is associative, then every bracketing of  $x_1 O \cdots O x_n$  has the same value.*

*Professional Standard Proof.*

TODO: professional standard proof for general-associativity. □

*Detailed Learning Proof.*

TODO: detailed learning proof for general-associativity. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Associativity](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (General Commutativity). *If  $O$  is associative and commutative, then any finite reordering of  $x_1 O \cdots O x_n$  has the same value.*

*Professional Standard Proof.*

TODO: professional standard proof for general-commutativity. □

*Detailed Learning Proof.*

TODO: detailed learning proof for general-commutativity. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Associativity](#)
- [Commutativity](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Invertibility Implies Left Cancellation). *In an associative operation with identity, invertible elements satisfy left cancellation.*

*Professional Standard Proof.*

TODO: professional standard proof for invertibility-implies-left-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for invertibility-implies-left-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Two-Sided Inverse](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Invertibility Implies Cancellation). *In an associative operation with identity, invertible elements satisfy two-sided cancellation.*

*Professional Standard Proof.*

TODO: professional standard proof for invertibility-implies-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for invertibility-implies-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Invertibility Implies Left Cancellation](#)
- [Invertibility Implies Right Cancellation](#)
- [Two-Sided Cancellation](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Invertibility Implies Right Cancellation). *In an associative operation with identity, invertible elements satisfy right cancellation.*

*Professional Standard Proof.*

TODO: professional standard proof for invertibility-implies-right-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for invertibility-implies-right-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Associativity](#)
- [Two-Sided Identity](#)
- [Two-Sided Inverse](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Left Zero Absorption). *In a ring,*

$$0 \cdot a = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-zero-absorption. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-zero-absorption. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Right Zero Absorption). *In a ring,*

$$a \cdot 0 = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-zero-absorption. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-zero-absorption. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Zero Absorption). *In a ring,*

$$0 \cdot a = 0 \quad \text{and} \quad a \cdot 0 = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-absorption. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-absorption. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Zero Absorption](#)
- [Right Zero Absorption](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Left Sign Rule). *In a ring,*

$$(-a) \cdot b = -(a \cdot b).$$

*Professional Standard Proof.*

TODO: professional standard proof for left-sign-rule. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-sign-rule. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Left Zero Absorption](#)



*Remark* (Return). [Return to Theorem](#)

**Proposition** (Right Sign Rule). *In a ring,*

$$a \cdot (-b) = -(a \cdot b).$$

*Professional Standard Proof.*

TODO: professional standard proof for right-sign-rule. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-sign-rule. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Right Zero Absorption](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Sign Rule). *In a ring,*

$$(-a) \cdot b = -(a \cdot b) \quad \text{and} \quad a \cdot (-b) = -(a \cdot b).$$

*Professional Standard Proof.*

TODO: professional standard proof for sign-rule. □

*Detailed Learning Proof.*

TODO: detailed learning proof for sign-rule. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Sign Rule](#)
- [Right Sign Rule](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Negative One Rule). *In a ring,*

$$(-1) \cdot a = -a.$$

*Professional Standard Proof.*

TODO: professional standard proof for negative-one-rule. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negative-one-rule. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Sign Rule](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Product of Negatives). *In a ring,*

$$(-a) \cdot (-b) = a \cdot b.$$

*Professional Standard Proof.*

TODO: professional standard proof for product-of-negatives. □

*Detailed Learning Proof.*

TODO: detailed learning proof for product-of-negatives. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Sign Rule](#)
- [Right Sign Rule](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (No Zero Divisors). *In an integral domain,*

$$a \cdot b = 0 \implies a = 0 \vee b = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for no-zero-divisors. □

*Detailed Learning Proof.*

TODO: detailed learning proof for no-zero-divisors. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Divisor](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Addition Preserves Order on the Left).

$$y < z \implies x + y < x + z$$

*in ordered arithmetic systems where addition is strictly order-preserving.*

*Professional Standard Proof.*

TODO: professional standard proof for addition-order-compatibility-left. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-order-compatibility-left. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Right Monotony](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Addition Preserves Order on the Right).

$$y < z \implies y + x < z + x$$

*in ordered arithmetic systems where addition is strictly order-preserving.*

*Professional Standard Proof.*

TODO: professional standard proof for addition-order-compatibility-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-order-compatibility-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Monotony](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Addition Preserves Order). *In ordered arithmetic systems where addition is strictly order-preserving, addition preserves order on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for addition-order-compatibility. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-order-compatibility. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition Preserves Order on the Left](#)
- [Addition Preserves Order on the Right](#)



*Remark* (Return). [Return to Theorem](#)

**Proposition** (Multiplication Preserves Order on the Left).

$$0 < x \wedge y < z \implies x \cdot y < x \cdot z$$

*in ordered arithmetic systems where multiplication by positive elements is strictly order-preserving.*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-order-compatibility-left. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-order-compatibility-left. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Right Monotony](#)
- [Strict Total Order](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Multiplication Preserves Order on the Right).

$$0 < x \wedge y < z \implies y \cdot x < z \cdot x$$

*in ordered arithmetic systems where multiplication by positive elements is strictly order-preserving.*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-order-compatibility-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-order-compatibility-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Monotony](#)
- [Strict Total Order](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Multiplication Preserves Order). *In ordered arithmetic systems where multiplication by positive elements is strictly order-preserving, multiplication preserves order on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-order-compatibility. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-order-compatibility. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication Preserves Order on the Left](#)
- [Multiplication Preserves Order on the Right](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Composition of Homomorphisms). *The composition of two homomorphisms is a homomorphism.*

*Professional Standard Proof.*

TODO: professional standard proof for composition-of-homomorphisms. □

*Detailed Learning Proof.*

TODO: detailed learning proof for composition-of-homomorphisms. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Homomorphism](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Identity Map Is a Homomorphism). *The identity map on a structure is a homomorphism from that structure to itself.*

*Professional Standard Proof.*

TODO: professional standard proof for identity-map-is-homomorphism. □

*Detailed Learning Proof.*

TODO: detailed learning proof for identity-map-is-homomorphism. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Homomorphism](#)

*Remark* (Return). Return to Theorem

**Theorem** (Group Cancellation). *In a group, cancellation holds on both sides:*

$$a * c = b * c \implies a = b \quad \text{and} \quad c * a = c * b \implies a = b.$$

*Professional Standard Proof.*

TODO: professional standard proof for group-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for group-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Inverse](#)
- [Uniqueness of Inverse](#)

*Remark* (Return). Return to Theorem

**Theorem** (Group Uniqueness of Inverse). *In a group, each element has a unique inverse.*

*Professional Standard Proof.*

TODO: professional standard proof for group-uniqueness-of-inverse. □

*Detailed Learning Proof.*

TODO: detailed learning proof for group-uniqueness-of-inverse. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Inverse](#)
- [Uniqueness of Inverse](#)

*Remark* (Return). Return to Theorem

**Theorem** (Field Zero Product). *In a field,*

$$a \cdot b = 0 \quad \Longleftrightarrow \quad a = 0 \text{ or } b = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-zero-product. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-zero-product. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Inverse](#)
- [No Zero Divisors](#)



*Remark* (Return). Return to Theorem

**Theorem** (Ring Double Negation). *In a ring,*

$$-(-a) = a.$$

*Professional Standard Proof.*

TODO: professional standard proof for ring-double-negation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ring-double-negation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Distributivity](#)
- [Uniqueness of Inverse](#)

*Remark* (Return). Return to Theorem

**Theorem** (Ring Negation of Product). *In a ring,*

$$(-a) \cdot b = -(a \cdot b) \quad \text{and} \quad a \cdot (-b) = -(a \cdot b).$$

*Professional Standard Proof.*

TODO: professional standard proof for ring-negation-of-product. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ring-negation-of-product. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Sign Rule](#)

*Remark* (Return). Return to Theorem

**Theorem** (Ring Product of Negatives). *In a ring,*

$$(-a) \cdot (-b) = a \cdot b.$$

*Professional Standard Proof.*

TODO: professional standard proof for ring-product-of-negatives. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ring-product-of-negatives. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Product of Negatives](#)

*Remark* (Return). Return to Theorem

**Theorem** (Field Multiplicative Cancellation). *In a field,*

$$c \neq 0 \wedge a \cdot c = b \cdot c \implies a = b.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-multiplicative-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-multiplicative-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Inverse](#)
- Group Cancellation

*Remark* (Return). Return to Theorem

**Theorem** (Field Uniqueness of Inverse). *In a field, each nonzero element has a unique multiplicative inverse.*

*Professional Standard Proof.*

TODO: professional standard proof for field-uniqueness-of-inverse. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-uniqueness-of-inverse. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Sided Inverse](#)
- Group Uniqueness of Inverse

*Remark* (Return). Return to Theorem

**Theorem** (Field Inverse of One). *In a field,*

$$1^{-1} = 1.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-inverse-of-one.

□

*Detailed Learning Proof.*

TODO: detailed learning proof for field-inverse-of-one.

□

*Remark* (Proof structure).

*Remark* (Dependencies).

- Field Uniqueness of Inverse

*Remark* (Return). Return to Theorem

**Theorem** (Field Inverse of Product). *In a field, if  $a \neq 0$  and  $b \neq 0$ , then*

$$(a \cdot b)^{-1} = a^{-1} \cdot b^{-1}.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-inverse-of-product. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-inverse-of-product. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- Field Uniqueness of Inverse
- [Field Zero Product](#)

*Remark* (Return). Return to Theorem

**Theorem** (Field Inverse Involution). *In a field, if  $a \neq 0$ , then*

$$(a^{-1})^{-1} = a.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-inverse-involution. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-inverse-involution. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- Field Uniqueness of Inverse



*Remark* (Return). Return to Theorem

**Theorem** (Field Exponent Addition). *In a field, if  $a \neq 0$  and  $m, n \in \mathbb{Z}$ , then*

$$a^{m+n} = a^m \cdot a^n.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-exponent-addition. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-exponent-addition. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Field Integer Power](#)

*Remark* (Return). Return to Theorem

**Theorem** (Field Power of Power). *In a field, if  $a \neq 0$  and  $m, n \in \mathbb{Z}$ , then*

$$(a^m)^n = a^{m \cdot n}.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-power-of-power. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-power-of-power. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Field Integer Power](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Field Power of Product). *In a field, if  $a \neq 0$ ,  $b \neq 0$ , and  $n \in \mathbb{Z}$ , then*

$$(a \cdot b)^n = a^n \cdot b^n.$$

*Professional Standard Proof.*

TODO: professional standard proof for field-power-of-product. □

*Detailed Learning Proof.*

TODO: detailed learning proof for field-power-of-product. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Field Integer Power](#)
- Field Inverse of Product

*Remark* (Return). Return to Theorem

**Corollary** (Generic Field Laws). *Every field satisfies the zero-product law, cancellation laws, uniqueness of inverses, inverse laws, ring sign laws, and integer-exponent laws listed in this topic.*

*Professional Standard Proof.*

TODO: professional standard proof for generic-field-laws. □

*Detailed Learning Proof.*

TODO: detailed learning proof for generic-field-laws. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Field Zero Product](#)
- Field Multiplicative Cancellation
- Field Inverse of Product
- [Field Exponent Addition](#)

*Remark* (Return). Return to Theorem

**Theorem** (Ordered Field Transitivity). *In an ordered field, for all elements  $a$ ,  $b$ , and  $c$ ,*

$$a < b \wedge b < c \implies a < c.$$

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-transitivity. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-transitivity. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Total Order](#)

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Ordered Field Squares Are Nonnegative). *In an ordered field,*

$$0 \leq a^2.$$

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-squares-nonnegative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-squares-nonnegative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Total Order](#)

*Remark* (Return). Return to Theorem

**Theorem** (Ordered Field One Is Positive). *In an ordered field,*

$$0 < 1.$$

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-one-positive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-one-positive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Total Order](#)
- [Ordered Field Squares Are Nonnegative](#)

*Remark* (Return). Return to Theorem

**Theorem** (Ordered Field Positive Product). *In an ordered field,*

$$0 < a \wedge 0 < b \implies 0 < a \cdot b.$$

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-positive-product. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-positive-product. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Total Order](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Ordered Field Positive Inverses). *In an ordered field,*

$$0 < a \implies 0 < a^{-1}.$$

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-positive-inverses. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-positive-inverses. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- Ordered Field Positive Product
- Field Inverse of One

*Remark* (Return). Return to Theorem

**Theorem** (Ordered Field Positive Multiplication Preserves Order). *In an ordered field,*

$$0 < c \wedge a < b \implies c \cdot a < c \cdot b.$$

*The corresponding non-strict implication also holds.*

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-positive-multiplication-preserves-order.

□

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-positive-multiplication-preserves-order.

□

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Total Order](#)
- Ordered Field Positive Product

*Remark* (Return). Return to Theorem

**Theorem** (Ordered Field Negative Multiplication Reverses Order). *In an ordered field,*

$$c < 0 \wedge a < b \implies c \cdot b < c \cdot a.$$

*The corresponding non-strict implication also holds.*

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-negative-multiplication-reverses-order.

□

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-negative-multiplication-reverses-order.

□

*Remark* (Proof structure).

*Remark* (Dependencies).

- Ordered Field Positive Multiplication Preserves Order
- Ring Negation of Product

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Ordered Field Fraction Comparison). *In an ordered field, if  $b \cdot d > 0$ , then*

$$\frac{a}{b} < \frac{c}{d} \iff a \cdot d < c \cdot b.$$

*Professional Standard Proof.*

TODO: professional standard proof for ordered-field-fraction-comparison. □

*Detailed Learning Proof.*

TODO: detailed learning proof for ordered-field-fraction-comparison. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- Ordered Field Positive Multiplication Preserves Order
- Field Multiplicative Cancellation

*Remark* (Return). Return to Theorem

**Corollary** (Generic Ordered-Field Laws). *Every ordered field satisfies the standard positivity, order-compatibility, inverse-order, sign, and comparison laws listed in this topic.*

*Professional Standard Proof.*

TODO: professional standard proof for generic-ordered-field-laws. □

*Detailed Learning Proof.*

TODO: detailed learning proof for generic-ordered-field-laws. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Ordered Field One Is Positive](#)
- Ordered Field Positive Product
- [Ordered Field Positive Inverses](#)
- Ordered Field Positive Multiplication Preserves Order

# Capstone

# Chapter 4

## Constructing a Number System



### 4.1 Ground Rules

#### Toolkit: Construction Obligations

- **Carrier:** the underlying collection of objects.
- **Operations:** the arithmetic rules defined on the carrier.
- **Relations:** equality, equivalence, order, or congruence relations.
- **Verification:** closure, well-definedness, laws, and embeddings.

### Construction Obligations

*Remark* (Exposition). A number-system construction is a structured promise. The construction must say what the objects are, how arithmetic is performed, which objects are identified, and which algebraic or order laws the resulting system satisfies.

#### Definition

**Definition 4.1.1** (Number-System Construction Target). A *number-system construction target* is the specified algebraic and order structure that a proposed system is meant to satisfy. The target fixes the required carrier, operations, relations, distinguished elements, and laws.

*Remark* (Standard quantified statement). A construction target specifies the structure and laws the new arithmetic system is intended to satisfy.

*Remark* (Interpretation). The target determines the proof obligations; without it, the phrase number system has no fixed mathematical content.

*Remark* (Dependencies).

#### Definition

**Definition 4.1.2** (Construction Data). The *construction data* for a number system consists of the raw carrier objects, the intended equality or equivalence relation, the distinguished elements, and the operation formulas from which the system is built.

*Remark* (Standard quantified statement). Construction data specifies the raw objects, identifications, distinguished elements, and operation formulas used in the construction.

*Remark* (Interpretation). The data is the material being tested against the target; it is not yet a verified number system.

*Remark* (Dependencies).

- [Number-System Construction Target](#)

#### Definition

**Definition 4.1.3** (Construction Verification). *Construction verification* is the proof that the construction data actually satisfies the construction target. It includes, as applicable, closure of operations, equivalence-relation proofs, well-definedness of operations on classes, algebraic laws, order laws, and compatibility of embeddings with earlier systems.

*Remark* (Standard quantified statement). Construction verification proves that the construction data satisfies the chosen construction target.

*Remark* (Interpretation). Verification is the step that turns proposed arithmetic into a justified structure.

*Remark* (Dependencies).

- [Number-System Construction Target](#)
- [Construction Data](#)

## 4.2 Finite Modular Number Systems

#### Toolkit: Residue Class Arithmetic

- **Modulus:** a positive integer  $n$ .
- **Congruence:**  $a \equiv b \pmod{n}$  when  $n \mid (a - b)$ .
- **Elements:** equivalence classes of integers modulo  $n$ .
- **Operations:** class addition and multiplication inherited from  $\mathbb{Z}$ .



## Congruence and Residue Classes

*Remark* (Exposition). Modular arithmetic is the quotient version of the two-element example. The carrier is obtained by identifying integers that have the same remainder modulo a fixed positive integer.

### Definition

**Definition 4.2.1** (Congruence Modulo an Integer). Let  $n \in \mathbb{Z}$  be positive. For  $a, b \in \mathbb{Z}$ , define

$$a \equiv b \pmod{n} \quad :\Longleftrightarrow \quad n \mid (a - b).$$

*Remark* (Standard quantified statement). For positive integer  $n$ , two integers are congruent modulo  $n$  exactly when  $n$  divides their difference.

*Remark* (Interpretation). Congruence modulo  $n$  records sameness of remainder using divisibility in  $\mathbb{Z}$ .

*Remark* (Dependencies).

- [Integers](#)
- [Positive Integer](#)
- [Divisibility in  \$\mathbb{Z}\$](#)
- [Subtraction on  \$\mathbb{Z}\$](#)

### Theorem

**Theorem 4.2.2** (Congruence Modulo an Integer Is an Equivalence Relation). *For each positive integer  $n$ , congruence modulo  $n$  is an equivalence relation on  $\mathbb{Z}$ .*

*Remark* (Standard quantified statement). For each positive integer  $n$ , congruence modulo  $n$  is reflexive, symmetric, and transitive on  $\mathbb{Z}$ .

*Remark* (Interpretation). This permits the quotient set  $\mathbb{Z}/n\mathbb{Z}$  to be formed from congruence classes.

*Remark* (Dependencies).

- [Congruence Modulo an Integer](#)
- [Divisibility in  \$\mathbb{Z}\$](#)

### Definition

**Definition 4.2.3** (Integers Modulo  $n$ ). Let  $n \in \mathbb{Z}$  be positive. The *integers modulo  $n$*  are the quotient set

$$\mathbb{Z}/n\mathbb{Z} := \mathbb{Z}/\equiv \pmod{n}.$$

*Remark* (Standard quantified statement). For positive integer  $n$ ,  $\mathbb{Z}/n\mathbb{Z}$  is the quotient of  $\mathbb{Z}$  by congruence modulo  $n$ .

*Remark* (Interpretation). The elements are not single integers but equivalence classes of integers.

*Remark* (Dependencies).

- [Quotient Set](#)
- [Integers](#)
- [Congruence Modulo an Integer](#)

#### Definition

**Definition 4.2.4** (Residue Class Modulo  $n$ ). For  $a \in \mathbb{Z}$ , the *residue class of  $a$  modulo  $n$*  is

$$[a]_n := \{b \in \mathbb{Z} : b \equiv a \pmod{n}\}.$$

*Remark* (Standard quantified statement). The set  $[a]_n$  contains exactly the integers congruent to  $a$  modulo  $n$ .

*Remark* (Interpretation). Each such set is one element of the quotient number system.

*Remark* (Dependencies).

- [Integers Modulo  \$n\$](#)
- [Equivalence Class](#)
- [Congruence Modulo an Integer](#)

## Operations

#### Definition

**Definition 4.2.5** (Addition Modulo  $n$ ). For residue classes modulo a positive integer  $n$ , define

$$[a]_n + [b]_n := [a + b]_n.$$

*Remark* (Standard quantified statement). The sum  $[a]_n + [b]_n$  is  $[a + b]_n$ .

*Remark* (Interpretation). This formula becomes a valid operation on classes only after well-definedness is proved.

*Remark* (Dependencies).

- [Residue Class Modulo  \$n\$](#)
- [Addition on  \$\mathbb{Z}\$ -Classes](#)

#### Definition

**Definition 4.2.6** (Multiplication Modulo  $n$ ). For residue classes modulo a positive integer  $n$ , define

$$[a]_n \cdot [b]_n := [ab]_n.$$

*Remark* (Standard quantified statement). The product  $[a]_n \cdot [b]_n$  is  $[ab]_n$ .

*Remark* (Interpretation). This formula inherits multiplication from  $\mathbb{Z}$ , subject to the same well-definedness obligation.

*Remark* (Dependencies).

- [Residue Class Modulo  \$n\$](#)
- [Multiplication on  \$\mathbb{Z}\$ -Classes](#)

#### Theorem

**Theorem 4.2.7** (Modular Addition and Multiplication Are Well-Defined). *For each positive integer  $n$ , addition and multiplication modulo  $n$  are well-defined operations on  $\mathbb{Z}/n\mathbb{Z}$ .*

*Remark* (Standard quantified statement). For each positive integer  $n$ , the representative formulas for modular addition and multiplication determine unique outputs.

*Remark* (Interpretation). Changing representatives does not change the output, so the operations belong to  $\mathbb{Z}/n\mathbb{Z}$  itself.

*Remark* (Dependencies).

- [Addition Modulo  \$n\$](#)
- [Multiplication Modulo  \$n\$](#)
- [Congruence Modulo an Integer](#)

#### Definition

**Definition 4.2.8** (Finite Modular Number System). For a positive integer  $n$ , the *finite modular number system modulo  $n$*  is

$$\mathbb{Z}/n\mathbb{Z}$$

with addition and multiplication modulo  $n$ , additive identity  $[0]_n$ , and multiplicative identity  $[1]_n$ .

*Remark* (Standard quantified statement). For positive integer  $n$ , the finite modular number system modulo  $n$  is  $\mathbb{Z}/n\mathbb{Z}$  with modular addition, modular multiplication,  $[0]_n$ , and  $[1]_n$ .

*Remark* (Interpretation). This is the quotient arithmetic system obtained by reducing integer arithmetic modulo  $n$ .

*Remark* (Dependencies).

- [Integers Modulo  \$n\$](#)
- [Addition Modulo  \$n\$](#)
- [Multiplication Modulo  \$n\$](#)
- [Zero in  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)

## Basic Example

### Theorem

**Theorem 4.2.9** ( $\mathbb{Z}/4\mathbb{Z}$  Has Zero Divisors). In  $\mathbb{Z}/4\mathbb{Z}$ , the nonzero class  $[2]_4$  satisfies

$$[2]_4 \cdot [2]_4 = [0]_4.$$

Therefore  $\mathbb{Z}/4\mathbb{Z}$  is not a field.

*Remark* (Standard quantified statement). In  $\mathbb{Z}/4\mathbb{Z}$ ,  $[2]_4 \neq [0]_4$  and  $[2]_4 \cdot [2]_4 = [0]_4$ .

*Remark* (Interpretation). This shows that finite modular number systems need not be fields.

*Remark* (Dependencies).

- [Finite Modular Number System](#)
- [Multiplication Modulo  \$n\$](#)

## 4.3 The $\{0, 1\}$ Number System

### Toolkit: The System with Two Elements

- **Carrier:** the set  $\{0, 1\}$ .
- **Addition:** addition with  $1 + 1 = 0$ .
- **Multiplication:** multiplication with  $1 \cdot 1 = 1$ .
- **Purpose:** a complete finite model of the construction checklist.

## The Carrier and Operations

### Definition

**Definition 4.3.1** (Two-Element Carrier). The *two-element carrier* is the set

$$\mathbb{B}_2 := \{0, 1\}.$$

*Remark* (Standard quantified statement). The carrier of the two-element system is the set containing exactly the integer elements 0 and 1.

*Remark* (Interpretation). This fixes the underlying finite set before any arithmetic is assigned to it.

*Remark* (Dependencies).

- [Zero in  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)

### Definition

**Definition 4.3.2** (Addition on the Two-Element System). Addition on  $\mathbb{B}_2$  is the binary operation  $+_2$  determined by

| $+_2$ | 0 | 1 |
|-------|---|---|
| 0     | 0 | 1 |
| 1     | 1 | 0 |

*Remark* (Standard quantified statement). Addition on the two-element system is the binary operation whose values are given by the displayed table.

*Remark* (Interpretation). The table is the definition of addition; in particular, 1 plus 1 equals 0 in this system.

*Remark* (Dependencies).

- [Two-Element Carrier](#)

### Definition

**Definition 4.3.3** (Multiplication on the Two-Element System). Multiplication on  $\mathbb{B}_2$  is the binary operation  $\cdot_2$  determined by

| $\cdot_2$ | 0 | 1 |
|-----------|---|---|
| 0         | 0 | 0 |
| 1         | 0 | 1 |

*Remark* (Standard quantified statement). Multiplication on the two-element system is the binary operation whose values are given by the displayed table.

*Remark* (Interpretation). The table is the definition of multiplication; 1 is multiplicative identity and 0 is absorbing.

*Remark* (Dependencies).

- [Two-Element Carrier](#)

### Definition

**Definition 4.3.4** (Two-Element Number System). The *two-element number system* is

$$\mathbb{F}_2 := (\mathbb{B}_2, +_2, \cdot_2, 0, 1).$$

*Remark* (Standard quantified statement). The two-element number system is the carrier  $\mathbb{B}_2$  equipped with its addition, multiplication, zero, and one.

*Remark* (Interpretation). This packages the carrier and operations as the arithmetic structure to be verified.

*Remark* (Dependencies).

- [Two-Element Carrier](#)
- [Addition on the Two-Element System](#)
- [Multiplication on the Two-Element System](#)

## Verification

### Theorem

**Theorem 4.3.5** (The Two-Element System Is a Field). *The two-element number system  $\mathbb{F}_2$  is a field.*

*Remark* (Standard quantified statement). The structure  $\mathbb{F}_2$  satisfies the field axioms.

*Remark* (Interpretation). The two tables make the smallest field visible by direct finite verification.

*Remark* (Dependencies).

- [Two-Element Number System](#)

### Theorem

**Theorem 4.3.6** (The Two-Element System Is Not an Ordered Field). *There is no order on  $\mathbb{B}_2$  that makes  $\mathbb{F}_2$  an ordered field.*

*Remark* (Standard quantified statement). No order on  $\mathbb{B}_2$  makes  $\mathbb{F}_2$  satisfy the ordered-field axioms.

*Remark* (Interpretation). The equation  $1 + 1 = 0$  is compatible with field arithmetic but incompatible with ordered-field arithmetic.

*Remark* (Dependencies).

- [The Two-Element System Is a Field](#)

## As Modular Arithmetic

### Theorem

**Theorem 4.3.7** ( $\mathbb{Z}/2\mathbb{Z}$  Is the Two-Element System). *The finite modular number system  $\mathbb{Z}/2\mathbb{Z}$  has exactly two classes,  $[0]_2$  and  $[1]_2$ , and its addition and multiplication tables are the tables of  $\mathbb{F}_2$ .*

*Remark* (Standard quantified statement).  $\mathbb{Z}/2\mathbb{Z}$  has the same carrier size and operation tables as  $\mathbb{F}_2$ .

*Remark* (Interpretation). The two-element table system is the first modular arithmetic system.

*Remark* (Dependencies).

- [Finite Modular Number System](#)
- [Two-Element Number System](#)
- [Modular Addition and Multiplication Are Well-Defined](#)

## Proofs

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Congruence Modulo an Integer Is an Equivalence Relation). *For each positive integer  $n$ , congruence modulo  $n$  is an equivalence relation on  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for congruence-modulo-integer-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for congruence-modulo-integer-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Congruence Modulo an Integer](#)
- [Divisibility in  \$\mathbb{Z}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Modular Addition and Multiplication Are Well-Defined). *For each positive integer  $n$ , addition and multiplication modulo  $n$  are well-defined operations on  $\mathbb{Z}/n\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for modular-operations-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for modular-operations-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition Modulo  \$n\$](#)
- [Multiplication Modulo  \$n\$](#)
- [Congruence Modulo an Integer](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}/4\mathbb{Z}$  Has Zero Divisors). *In  $\mathbb{Z}/4\mathbb{Z}$ , the nonzero class  $[2]_4$  satisfies  $[2]_4 \cdot [2]_4 = [0]_4$ . Therefore  $\mathbb{Z}/4\mathbb{Z}$  is not a field.*

*Professional Standard Proof.*

TODO: professional standard proof for z-mod-four-has-zero-divisors. □

*Detailed Learning Proof.*

TODO: detailed learning proof for z-mod-four-has-zero-divisors. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Finite Modular Number System](#)
- [Multiplication Modulo  \$n\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (The Two-Element System Is a Field). *The two-element number system  $\mathbb{F}_2$  is a field.*

*Professional Standard Proof.*

TODO: professional standard proof for two-element-system-is-field. □

*Detailed Learning Proof.*

TODO: detailed learning proof for two-element-system-is-field. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Two-Element Number System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (The Two-Element System Is Not an Ordered Field). *There is no order on  $\mathbb{B}_2$  that makes  $\mathbb{F}_2$  an ordered field.*

*Professional Standard Proof.*

TODO: professional standard proof for two-element-system-not-ordered-field. □

*Detailed Learning Proof.*

TODO: detailed learning proof for two-element-system-not-ordered-field. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [The Two-Element System Is a Field](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}/2\mathbb{Z}$  Is the Two-Element System). *The finite modular number system  $\mathbb{Z}/2\mathbb{Z}$  has exactly two classes,  $[0]_2$  and  $[1]_2$ , and its addition and multiplication tables are the tables of  $\mathbb{F}_2$ .*

*Professional Standard Proof.*

TODO: professional standard proof for z-mod-two-is-two-element-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for z-mod-two-is-two-element-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Finite Modular Number System](#)
- [Two-Element Number System](#)
- [Modular Addition and Multiplication Are Well-Defined](#)

# Capstone

# Chapter 5

## Peano Systems



### 5.1 Presburger Arithmetic

#### Definition (Presburger Signature)

**Definition 5.1.1** (Presburger Signature). The *Presburger signature* is the first-order signature

$$\mathcal{L}_{\text{Pres}} = \{0, S, +, <, =\},$$

where 0 is a constant symbol,  $S$  is a unary function symbol,  $+$  is a binary function symbol, and  $<$  and  $=$  are binary relation symbols.

*Remark* (Standard quantified statement).

$\text{PresburgerSignature}(\mathcal{L}) \iff \mathcal{L} = \{0, S, +, <, =\} \wedge \text{ar}(S) = 1 \wedge \text{ar}(+) = 2 \wedge \text{ar}(<) = 2 \wedge \text{ar}(=) = 2.$  ■

*Remark* (Interpretation). The Presburger signature names addition and order, but it deliberately omits multiplication as a function symbol. This makes Presburger arithmetic a theory of the additive structure of the natural numbers rather than a theory of full arithmetic.

#### Definition (Presburger Arithmetic)

**Definition 5.1.2** (Presburger Arithmetic). *Presburger arithmetic* is the first-order theory of the natural numbers in the Presburger signature. Its intended structure is

$$(\mathbb{N}, 0, S, +, <, =),$$

and its axioms describe the behavior of zero, successor, addition, order, and induction for formulas written in the language  $\mathcal{L}_{\text{Pres}}$ .

*Remark* (Standard quantified statement).

$\text{PresburgerArithmetic}(T) \iff T \subseteq \text{Sentences}(\mathcal{L}_{\text{Pres}}) \wedge T \text{ axiomatizes } (\mathbb{N}, 0, S, +, <, =).$

*Remark* (Interpretation). Presburger arithmetic isolates the part of arithmetic governed by addition. It is strong enough to express linear equations, congruences, and order properties of natural numbers, but it cannot directly express multiplication as a primitive operation. The later Peano-system material supplies the more general recursive framework from which addition, multiplication, order, and induction are developed structurally.

## 5.2 Defining Peano Systems

### defining-peano-systems

Peano Systems  $\rightarrow$  **Peano Axioms**  $\rightarrow$  Recursion on Peano Systems

### The Peano Axioms

The Peano axioms make precise what it means for a system to behave like the natural numbers. They isolate the first element, the successor map, and the induction principle. These axioms do not define addition, multiplication, or order. They fix the structure that supports those later constructions. The formulation used here belongs to the Dedekind–Peano line of axiomatizing the natural-number structure [Dedekind, 1901, Peano, 1889].

### Definition (Peano System)

**Definition 5.2.1** (Peano System). A triple  $(P, S, 1)$  is called a *Peano system* exactly when  $P$  is a set,  $1 \in P$ ,  $S : P \rightarrow P$  is a function, and the following axioms hold:

- P1.**  $1 \in P$ .
- P2.** For every  $n \in P$ ,  $S(n) \in P$ .
- P3.** For every  $n \in P$ ,  $S(n) \neq 1$ .
- P4.** For all  $m, n \in P$ , if  $S(m) = S(n)$  then  $m = n$ .
- P5.** If  $A \subseteq P$ ,  $1 \in A$ , and

$$\forall n \in P (n \in A \implies S(n) \in A),$$

then  $A = P$ .

*Remark* (Standard quantified statement).

$$\begin{aligned}
& (P, S, 1) \text{ is a Peano system} \\
& \iff \left( P \text{ is a set} \wedge 1 \in P \wedge S : P \rightarrow P \right. \\
& \quad \wedge \forall n \in P (S(n) \in P) \\
& \quad \wedge \forall n \in P (S(n) \neq 1) \\
& \quad \wedge \forall m, n \in P (S(m) = S(n) \implies m = n) \\
& \quad \left. \wedge \forall A \subseteq P ((1 \in A \wedge \forall n \in P (n \in A \implies S(n) \in A)) \implies A = P) \right).
\end{aligned}$$

*Remark* (Failure modes). A triple  $(P, S, 1)$  fails to be a Peano system if the distinguished element does not lie in the underlying set, if the successor map leaves the set, if the successor hits 1, if the successor identifies two distinct elements, or if there exists a proper subset of  $P$  that contains 1 and is closed under successor.

*Remark* (Failure mode decomposition).

$$\begin{aligned}
\neg (P, S, 1) \text{ is a Peano system} & \implies 1 \notin P \\
& \vee \exists n \in P (S(n) \notin P) \\
& \vee \exists n \in P (S(n) = 1) \\
& \vee \exists m, n \in P (m \neq n \wedge S(m) = S(n)) \\
& \vee \exists A \subseteq P (1 \in A \wedge \forall n \in P (n \in A \implies S(n) \in A) \wedge A \neq P).
\end{aligned}$$

*Remark* (Interpretation). A Peano system is the abstract data of counting: an underlying set, a first element, and a successor operation satisfying the usual non-collision and induction principles. The purpose of the definition is to isolate the ambient object before the individual axioms are recorded and used one by one.

*Remark* (Source alignment). The five displayed clauses are the house-style Peano-system presentation of Peano's axiomatization of arithmetic [Peano, 1889]. They are stated here in structural form so that later definitions and proofs can refer directly to the successor and induction clauses.

*Remark* (Comparison with Feferman). This definition corresponds closely to Definition 3.1 in Feferman [1964], where Peano systems are introduced as the abstract structural framework underlying the positive integers. The present notes expand the definition into explicit quantified, failure-mode, and decomposition forms in order to support later logical analysis and dependency extraction.

*Remark* (Dependencies).

- [Function](#)
- [Subset](#)



#### Axiom (Base Element Lies in the Underlying Set)

**Axiom 5.2.2** (Distinguished Element Lies in the Underlying Set).

$$1 \in P.$$

*Remark* (Standard quantified statement).

$$1 \in P.$$

*Remark* (Interpretation). The counting system begins from a distinguished first element of the underlying set. This is the base point from which the remaining elements are generated by the successor operation.

#### Axiom (Closure Under Successor)

**Axiom 5.2.3** (Closure Under Successor). If  $n \in P$ , then  $S(n) \in P$ .

*Remark* (Standard quantified statement).

$$\forall n \in P (S(n) \in P).$$

*Remark* (Interpretation). Applying the successor operation to an element of the underlying set never leaves that set. Successor is therefore an internal operation on the counting system.

#### Axiom (Base Element Is Not a Successor)

**Axiom 5.2.4** (Distinguished Element Is Not a Successor). For every  $n \in P$ ,  $S(n) \neq 1$ .

*Remark* (Standard quantified statement).

$$\forall n \in P (S(n) \neq 1).$$

*Remark* (Interpretation). The first natural number does not arise by taking the successor of an earlier natural number. This keeps the counting system from folding backward onto its starting point.

#### Axiom (Injectivity of Successor)

**Axiom 5.2.5** (Injectivity of Successor). If  $m, n \in P$  and  $S(m) = S(n)$ , then  $m = n$ .

*Remark* (Standard quantified statement).

$$\forall m, n \in P (S(m) = S(n) \implies m = n).$$

*Remark* (Interpretation). Different natural numbers do not collapse to the same successor. The successor operation preserves distinctness and lets predecessor reasoning work whenever a successor is known.

### Axiom (Induction Axiom)

**Axiom 5.2.6** (Induction Axiom). Let  $A \subseteq P$ . If  $1 \in A$  and if

$$\forall n \in P (n \in A \implies S(n) \in A),$$

then  $A = P$ .

*Remark* (Standard quantified statement).

$$\forall A \subseteq P \left( (1 \in A \wedge \forall n \in P (n \in A \implies S(n) \in A)) \implies A = P \right).$$

*Remark* (Interpretation). Any subset of the underlying set that contains the distinguished element and is closed under successor must already contain the entire set. This axiom rules out rogue elements and is the logical source of mathematical induction.

### Axiom (Existence of a Peano System)

**Axiom 5.2.7** (Existence of a Peano System). There exists a Peano system.

*Remark* (Standard quantified statement).

$$\exists P \exists S ((P, S, 1) \text{ is a Peano system}).$$

*Remark* (Interpretation). The Peano axioms are not merely a formal pattern; at least one structure satisfies them. This axiom guarantees that the abstract counting system is available before we give it the conventional name  $\mathbb{N}$ .

*Remark* (Historical note). This axiom corresponds to Axiom 3.2 of [Feferman \[1964\]](#), where the existence of at least one Peano system is taken as a foundational assumption.

*Remark* (Dependencies).

- [Peano System](#)
- [Language](#)

## First Theorems on Peano Systems

### Induction as a Working Principle

The induction axiom for a Peano system is formulated in terms of inductive subsets of the underlying set. Arithmetic arguments are usually written in terms of predicates rather than subsets. This theorem identifies the predicate-form induction principle derived from the Peano axioms. It is the standard form of induction used in the rest of the chapter. Historically, this is the working proof principle attached to Peano's axiomatization of the

natural numbers [Peano, 1889].

#### Theorem (Induction Principle for a Peano System)

**Theorem 5.2.8** (Induction Principle for a Peano System). *Let  $(P, S, 1)$  be a Peano system, and let  $\varphi$  be a predicate on  $P$ . If*

$$\varphi(1) \quad \text{and} \quad \forall n \in P (\varphi(n) \implies \varphi(S(n))),$$

*then*

$$\forall n \in P \varphi(n).$$

*Go to proof.*

#### Lean 4 Verification Record

**Source anchor:** Peano, *Arithmetices principia*; Feferman, *The Number Systems*, Section 3.1.

**Formalizes:** [Theorem 5.2.8](#).

**Lean module:** `LRA.VolumeII.PeanoSystems.Induction`.

**Lean declaration:** `induction_principle`.

**Status:** checked against `lra-lean`.

```
theorem induction_principle
  (ps : PeanoSystem)
  (predicate : ps.carrier -> Prop)
  (base_case : predicate ps.one)
  (successor_step :
    forall element : ps.carrier,
      predicate element ->
        predicate (ps.successor element)) :
    forall element : ps.carrier,
      predicate element :=
  ps.induction predicate base_case successor_step
```

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $\varphi$  be a predicate on  $P$ .

$$\left( \varphi(1) \wedge \forall n \in P (\varphi(n) \implies \varphi(S(n))) \right) \implies \forall n \in P \varphi(n).$$

*Remark* (Contrapositive quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $\varphi$  be a predicate on  $P$ .

$$\neg \forall n \in P \varphi(n) \implies \neg \left( \varphi(1) \wedge \forall n \in P (\varphi(n) \implies \varphi(S(n))) \right).$$

*Remark* (Interpretation). This theorem converts the subset form of induction into the predicate form used in ordinary arithmetic arguments. A property propagates through the entire Peano system once it holds at the distinguished first element and is preserved by successor. The theorem therefore turns the structural induction axiom into a proof principle for arbitrary predicates on  $P$ .

*Remark* (Comparison with Feferman). Peano’s axiomatization includes induction as part of the natural-number structure [Peano, 1889]. Feferman derives the working form of mathematical induction from the subset formulation of the Peano axioms immediately after Theorem 3.3 in Chapter 3 of Feferman [1964]. The present notes formalize that induction principle as an explicit reusable theorem.

*Remark* (Dependencies).

- [Peano System](#)
- [Induction Axiom](#)

## Strong Induction

Ordinary induction propagates a property from each element to its immediate successor. Strong induction allows the inductive step to use the property at every predecessor, not just the immediate one. This broader hypothesis makes strong induction convenient when the successor step needs information from the entire earlier segment of the Peano system.

### Theorem (Strong Induction on a Peano System)

**Theorem 5.2.9** (Strong Induction on a Peano System). *Let  $(P, S, 1)$  be a Peano system, and let  $\varphi$  be a predicate on  $P$ . If*

$$\forall n \in P \left( \forall k \in P (k < n \implies \varphi(k)) \right) \implies \varphi(n),$$

*then*

$$\forall n \in P \varphi(n).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $\varphi$  be a predicate on  $P$ .

$$\left( \forall n \in P \left( \forall k \in P (k < n \implies \varphi(k)) \right) \implies \varphi(n) \right) \implies \forall n \in P \varphi(n).$$

*Remark* (Interpretation). Strong induction grants the inductive step access to the entire history below  $n$ , not just the single predecessor. It is the proof form behind least counterexample arguments.

*Remark* (Dependencies).

- [Induction Principle for a Peano System](#)
- [Strict Less Than on a Peano System](#)
- [Trichotomy on a Peano System](#)

## Inductive Subsets and Predecessors

Induction is expressed in terms of subsets of a Peano system. Two structural notions appear repeatedly in that setting: closure under successor and membership generated from the distinguished first element. Predecessor language is also convenient once it is known that every non-initial element arises as a successor.

### Definition (Successor-Closed Subset)

**Definition 5.2.10** (Successor-Closed Subset). Let  $(P, S, 1)$  be a Peano system, and let  $A \subseteq P$ . The subset  $A$  is called *successor-closed* exactly when

$$\forall n \in P (n \in A \implies S(n) \in A).$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $A \subseteq P$ .

$A$  is successor-closed  $\iff \forall n \in P (n \in A \implies S(n) \in A)$ .

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $A \subseteq P$ .

$A$  is not successor-closed  $\iff \exists n \in P (n \in A \wedge S(n) \notin A)$ .

*Remark* (Failure modes). A subset fails to be successor-closed exactly when it contains some element of  $P$  but fails to contain that element's successor.

*Remark* (Failure mode decomposition).

$$A \text{ is not successor-closed } \iff \exists n \in P \underbrace{(n \in A \wedge S(n) \notin A)}_{\text{closure fails at } n}.$$

*Remark* (Interpretation). A successor-closed subset is stable under one step of counting. Once an element of  $A$  is present, its successor remains in  $A$  as well.

*Remark* (Comparison with Feferman). The notion of a successor-closed subset is implicit in condition (iii) of Feferman's Definition 3.1 [Feferman, 1964]. The present notes isolate it as a separate definition so that induction arguments can refer to the closure condition directly.

*Remark* (Dependencies).

- [Peano System](#)

### Definition (Inductive Subset of a Peano System)

**Definition 5.2.11** (Inductive Subset of a Peano System). Let  $(P, S, 1)$  be a Peano system, and let  $A \subseteq P$ . The subset  $A$  is called *inductive* exactly when  $1 \in A$  and  $A$  is successor-closed.

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $A \subseteq P$ .

$A$  is inductive  $\iff (1 \in A \wedge \forall n \in P (n \in A \implies S(n) \in A))$ .

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $A \subseteq P$ .

$A$  is not inductive  $\iff (1 \notin A \vee \exists n \in P (n \in A \wedge S(n) \notin A))$ .

*Remark* (Failure modes). A subset fails to be inductive in exactly two ways: it may fail to contain the distinguished first element, or it may contain some element while failing to contain that element's successor.

*Remark* (Failure mode decomposition).

$$A \text{ is not inductive } \iff \underbrace{1 \notin A}_{\text{base point missing}} \vee \underbrace{\exists n \in P (n \in A \wedge S(n) \notin A)}_{\text{successor closure fails}}.$$

*Remark* (Interpretation). An inductive subset contains the distinguished first element and is closed under successor. Such a subset contains every stage obtained by starting at 1 and applying successor repeatedly.

*Remark* (Comparison with Feferman). The notion of an inductive subset packages the two hypotheses appearing in the induction clause of Feferman's Definition 3.1 [Feferman, 1964]. The present notes name this package so that the internal structure of induction proofs remains explicit.

*Remark* (Dependencies).

- [Peano System](#)
- [Successor-Closed Subset](#)

#### Theorem (Minimality of a Peano System)

**Theorem 5.2.12** (Minimality of a Peano System). *Let  $(P, S, 1)$  be a Peano system. If  $A \subseteq P$  contains 1 and is closed under successor, then  $A = P$ .*

$$(1 \in A \wedge \forall n \in P (n \in A \implies S(n) \in A)) \implies A = P.$$

[Go to proof.](#)

*Remark* (Interpretation). A Peano system has no elements outside the successor chain generated by its base element. Any subset that contains the base element and is closed under successor already exhausts the whole carrier.

*Remark* (Dependencies).

- [Induction Axiom](#)
- [Inductive Subset of a Peano System](#)

#### Definition (Predecessor in a Peano System)

**Definition 5.2.13** (Predecessor in a Peano System). Let  $(P, S, 1)$  be a Peano system, and let  $x, u \in P$ . The element  $u$  is called a *predecessor* of  $x$  exactly when

$$S(u) = x.$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $x, u \in P$ .  
 $u$  is a predecessor of  $x \iff S(u) = x$ .

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $x, u \in P$ .  
 $u$  is not a predecessor of  $x \iff S(u) \neq x$ .

*Remark* (Interpretation). A predecessor of  $x$  is an element whose successor is  $x$ . This language isolates the inverse-looking question attached to the successor map without assuming that successor is globally invertible.

*Remark* (Dependencies).

- [Peano System](#)

### Generation by Successor

The Peano axioms describe a counting system generated from a distinguished first element by the successor operation. The next theorems make that generation principle explicit: every element is either the first element or a successor, every non-initial element has a unique predecessor, and no element can be its own successor.

#### Theorem (Every Element Is Either 1 or a Successor)

**Theorem 5.2.14** (Every Element Is Either 1 or a Successor). *Let  $(P, S, 1)$  be a Peano system. For every  $x \in P$ , either  $x = 1$  or there exists  $u \in P$  such that  $S(u) = x$ . [Go to proof.](#)*

## Lean 4 Verification Record

**Source anchor:** Landau, *Foundations of Analysis*, Section 1; Feferman, *The Number Systems*, Theorem 3.3.

**Formalizes:** [Theorem 5.2.14](#).

**Lean module:** LRA.VolumeII.PeanoSystems.BasicTheorems.

**Lean declaration:** every\_element\_is\_one\_or\_successor.

**Status:** checked against lra-lean.

```
theorem every_element_is_one_or_successor
  (ps : PeanoSystem) :
  forall element : ps.carrier,
    Or (element = ps.one)
      (Exists (fun predecessor : ps.carrier =>
        ps.successor predecessor = element)) := by
  let D : ps.carrier -> Prop :=
    fun candidate_element =>
      Or (candidate_element = ps.one)
        (Exists (fun predecessor : ps.carrier =>
          ps.successor predecessor = candidate_element))

  apply induction_principle ps D

  case base_case =>
    exact Or.inl rfl

  case successor_step =>
    intro element induction_hypothesis
    exact Or.inr (Exists.intro element rfl)
```

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall x \in P \ (x = 1 \vee \exists u \in P \ (S(u) = x)).$$

*Remark* (Interpretation). Every element of a Peano system is reached by starting at the distinguished first element and moving forward by successor. The only possible exception to having a predecessor is the element 1 itself.

*Remark* (Historical note). This theorem is one of the basic successor consequences of Peano's axiomatization [Peano, 1889]. In the present organization it corresponds directly to Theorem 3.3 in Feferman [1964]. Feferman uses this result to show that the distinguished initial element is the only possible non-successor in a Peano system.

*Remark* (Dependencies).

- [Peano System](#)
- [Distinguished Element Lies in the Underlying Set](#)
- [Closure Under Successor](#)
- [Induction Principle for a Peano System](#)



### Theorem (Successor Inequality Reflection)

**Theorem 5.2.15** (Successor Inequality Reflection). *Let  $(P, S, 1)$  be a Peano system. For all  $a, b \in P$ , if  $a \neq b$ , then  $S(a) \neq S(b)$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall a, b \in P (a \neq b \implies S(a) \neq S(b)).$

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\exists a, b \in P (a \neq b \wedge S(a) = S(b)).$

*Remark* (Failure modes). Successor inequality reflection fails exactly when two distinct elements have the same successor.

*Remark* (Failure mode decomposition).

$$\exists a, b \in P \underbrace{(a \neq b \wedge S(a) = S(b))}_{\text{distinct elements with a common successor}}.$$

*Remark* (Contrapositive quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall a, b \in P (S(a) = S(b) \implies a = b).$

*Remark* (Interpretation). The successor operation reflects inequality: distinct inputs remain distinct after one successor step. This is the contrapositive form of successor injectivity, and it is the form used when an induction step must preserve a negative equality statement.

*Remark* (Dependencies).

- [Peano System](#)
- [Injectivity of Successor](#)

### Corollary (Non-One Elements Have a Predecessor)

**Corollary 5.2.16** (Non-One Elements Have a Predecessor). *Let  $(P, S, 1)$  be a Peano system. For every  $x \in P$ , if  $x \neq 1$ , then there exists  $u \in P$  such that  $S(u) = x$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall x \in P (x \neq 1 \implies \exists u \in P (S(u) = x)).$

*Remark* (Negated quantified statement).

$$\begin{aligned} &\text{Let } (P, S, 1) \text{ be a Peano system.} \\ &\exists x \in P \left( x \neq 1 \wedge \forall u \in P (S(u) \neq x) \right). \end{aligned}$$

*Remark* (Failure modes). The statement fails exactly when some element distinct from 1 has no predecessor.

*Remark* (Failure mode decomposition).

$$\exists x \in P \underbrace{\left( x \neq 1 \wedge \forall u \in P (S(u) \neq x) \right)}_{\text{non-initial element with no predecessor}}.$$

*Remark* (Interpretation). Every non-initial element of a Peano system is reached by one successor step from an earlier element. This separates predecessor existence from predecessor uniqueness so that later arguments can cite only the part they need.

*Remark* (Comparison with Feferman). Feferman's Theorem 3.3 in [Feferman \[1964\]](#) contains this predecessor-existence fact as part of the structural analysis of Peano systems. The present notes record it separately before adding uniqueness.

*Remark* (Dependencies).

- [Peano System](#)
- [Every Element Is Either 1 or a Successor](#)

#### Corollary (Predecessor Existence and Uniqueness Away from 1)

**Corollary 5.2.17** (Predecessor Existence and Uniqueness Away from 1). *Let  $(P, S, 1)$  be a Peano system, and let  $x \in P$  with  $x \neq 1$ . Then there exists a unique  $u \in P$  such that  $S(u) = x$ . [Go to proof](#).*

*Remark* (Standard quantified statement).

$$\begin{aligned} &\text{Let } (P, S, 1) \text{ be a Peano system.} \\ &\forall x \in P \left( x \neq 1 \implies \exists! u \in P (S(u) = x) \right). \end{aligned}$$

*Remark* (Interpretation). Every element distinct from 1 has exactly one predecessor. The successor map therefore organizes the non-initial part of a Peano system into one-step extensions of earlier elements.

*Remark* (Comparison with Feferman). Feferman proves predecessor existence and uniqueness together inside Theorem 3.3 of [Feferman \[1964\]](#). The present notes separate the uniqueness statement into an independent corollary so that it can be cited directly in later proofs.

*Remark* (Dependencies).

- [Peano System](#)

- [Non-One Elements Have a Predecessor](#)
- [Injectivity of Successor](#)

#### Corollary (Unique Predecessor Characterization Away from 1)

**Corollary 5.2.18** (Unique Predecessor Characterization Away from 1). *Let  $(P, S, 1)$  be a Peano system. For every  $x \in P$ ,  $x \neq 1$  if and only if there exists a unique  $u \in P$  such that  $S(u) = x$ . [Go to proof](#).*

*Remark* (Standard quantified statement).

$$\begin{aligned} &\text{Let } (P, S, 1) \text{ be a Peano system.} \\ &\forall x \in P \left( x \neq 1 \iff \exists! u \in P (S(u) = x) \right). \end{aligned}$$

*Remark* (Negated quantified statement).

$$\begin{aligned} &\text{Let } (P, S, 1) \text{ be a Peano system.} \\ &\exists x \in P \left( (x \neq 1 \wedge (\forall u \in P (S(u) \neq x) \right. \\ &\quad \left. \vee \exists u, v \in P (S(u) = x \wedge S(v) = x \wedge u \neq v))) \right. \\ &\quad \left. \vee (x = 1 \wedge \exists u \in P (S(u) = x \wedge \forall v \in P (S(v) = x \implies v = u))) \right). \end{aligned}$$

*Remark* (Failure modes). The biconditional can fail in two ways: a non-initial element can fail to have a unique predecessor, or the distinguished first element can have a unique predecessor.

*Remark* (Failure mode decomposition).

$$\begin{aligned} &\exists x \in P \left( \underbrace{(x \neq 1 \wedge (\forall u \in P (S(u) \neq x) \vee \exists u, v \in P (S(u) = x \wedge S(v) = x \wedge u \neq v)))}_{\text{non-initial element lacks a unique predecessor}} \right. \\ &\quad \left. \vee \underbrace{(x = 1 \wedge \exists u \in P (S(u) = x \wedge \forall v \in P (S(v) = x \implies v = u)))}_{\text{the first element has a unique predecessor}} \right). \end{aligned}$$

*Remark* (Interpretation). Being different from the distinguished first element is exactly the same as having a unique predecessor. This turns the informal phrase "away from 1" into an internal characterization of the successor structure.

*Remark* (Comparison with Feferman). This corollary packages the predecessor analysis surrounding Feferman's Theorem 3.3 in [Feferman \[1964\]](#). The forward direction is the predecessor existence-and-uniqueness result, while the reverse direction uses the axiom that 1 is not a successor.

*Remark* (Dependencies).

- [Peano System](#)

- [Distinguished Element Is Not a Successor](#)
- [Predecessor Existence and Uniqueness Away from 1](#)

#### Corollary (The Distinguished Element Is the Unique Non-Successor)

**Corollary 5.2.19** (The Distinguished Element Is the Unique Non-Successor). *Let  $(P, S, 1)$  be a Peano system. An element  $x \in P$  is not a successor of any element of  $P$  if and only if  $x = 1$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall x \in P \left( \neg \exists u \in P (S(u) = x) \iff x = 1 \right).$$

*Remark* (Interpretation). The distinguished first element is characterized internally as the unique element with no predecessor. This gives a structural description of 1 that does not depend on external notation.

*Remark* (Comparison with Feferman). This corollary extracts a structural characterization of the distinguished initial element that is implicit in the discussion surrounding Theorem 3.3 of [Feferman \[1964\]](#).

*Remark* (Dependencies).

- [Distinguished Element Is Not a Successor](#)
- [Predecessor Existence and Uniqueness Away from 1](#)

#### Theorem (No Object Is Its Own Successor)

**Theorem 5.2.20** (No Object Is Its Own Successor). *Let  $(P, S, 1)$  be a Peano system. For every  $n \in P$ ,*

$$S(n) \neq n.$$

[Go to proof.](#)

## Lean 4 Verification Record

**Source anchor:** Landau, *Foundations of Analysis*, Section 2, Theorem 2.

**Formalizes:** [Theorem 5.2.20](#).

**Lean module:** LRA.VolumeII.PeanoSystems.BasicTheorems.

**Lean declaration:** successor\_not\_self.

**Status:** checked against lra-lean.

```
theorem successor_not_self
  (ps : PeanoSystem) :
  forall element : ps.carrier,
    Not (ps.successor element = element) := by
  apply induction_principle ps

  case base_case =>
    exact ps.one_not_successor ps.one

  case successor_step =>
    intro element induction_hypothesis
    exact
      successor_preserves_inequality
        ps
        (ps.successor element)
        element
        induction_hypothesis
```

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$\forall n \in P \ (S(n) \neq n)$ .

*Remark* (Interpretation). Successor always moves forward in the counting structure. No element can be recovered by applying successor to itself.

*Remark* (Historical note). This theorem is a standard consequence of the Peano axioms in the classical development of the natural numbers [Peano, 1889]. Feferman includes the corresponding result as an exercise in Chapter 3 of Feferman [1964].

*Example 5.2.1 (Why the base case is forced).*

For the first element, the theorem says

$$S(1) \neq 1.$$

This is exactly the axiom that the distinguished element is not a successor: there is no element  $x \in P$  with  $S(x) = 1$ . In particular, taking  $x = 1$  would be impossible, so  $S(1) \neq 1$ .

*Example 5.2.2 (Why the induction step uses injectivity).*

Suppose the induction hypothesis gives  $S(n) \neq n$ . To prove the next case, we must show

$$S(S(n)) \neq S(n).$$

If equality held, injectivity of  $S$  would let us cancel the outer successor and conclude  $S(n) = n$ , contradicting the induction hypothesis. Thus the property propagates from  $n$  to  $S(n)$ .

*Remark* (Dependencies).

- [Distinguished Element Is Not a Successor](#)
- [Injectivity of Successor](#)
- [Induction Principle for a Peano System](#)

## **Peano System Figure Plates**

The following figure plates collect the visual diagrams for the Peano system and iterator material. Each plate is presented separately so that the diagram can be read as a full-page reference.

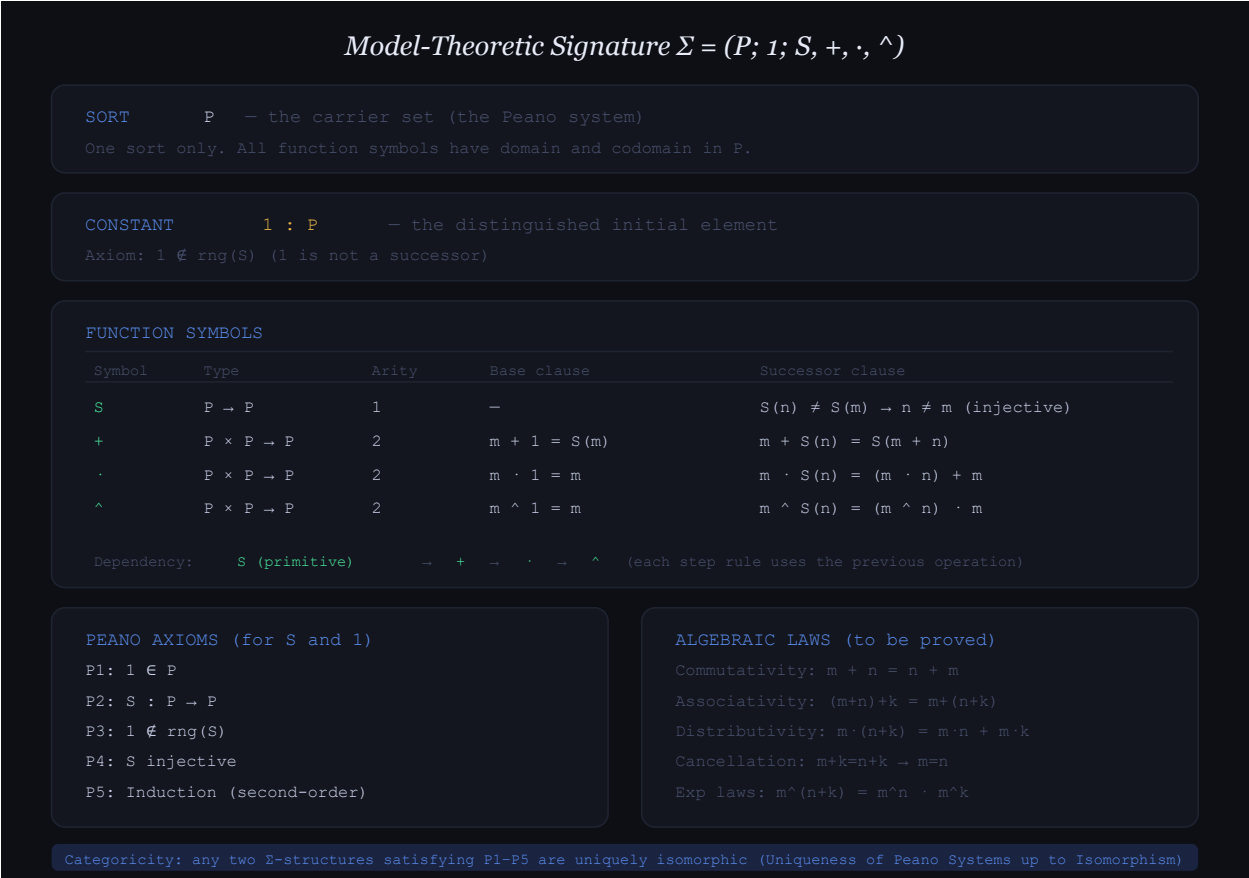


Figure 5.1: The model-theoretic signature for a Peano system and the later arithmetic operations generated from it.

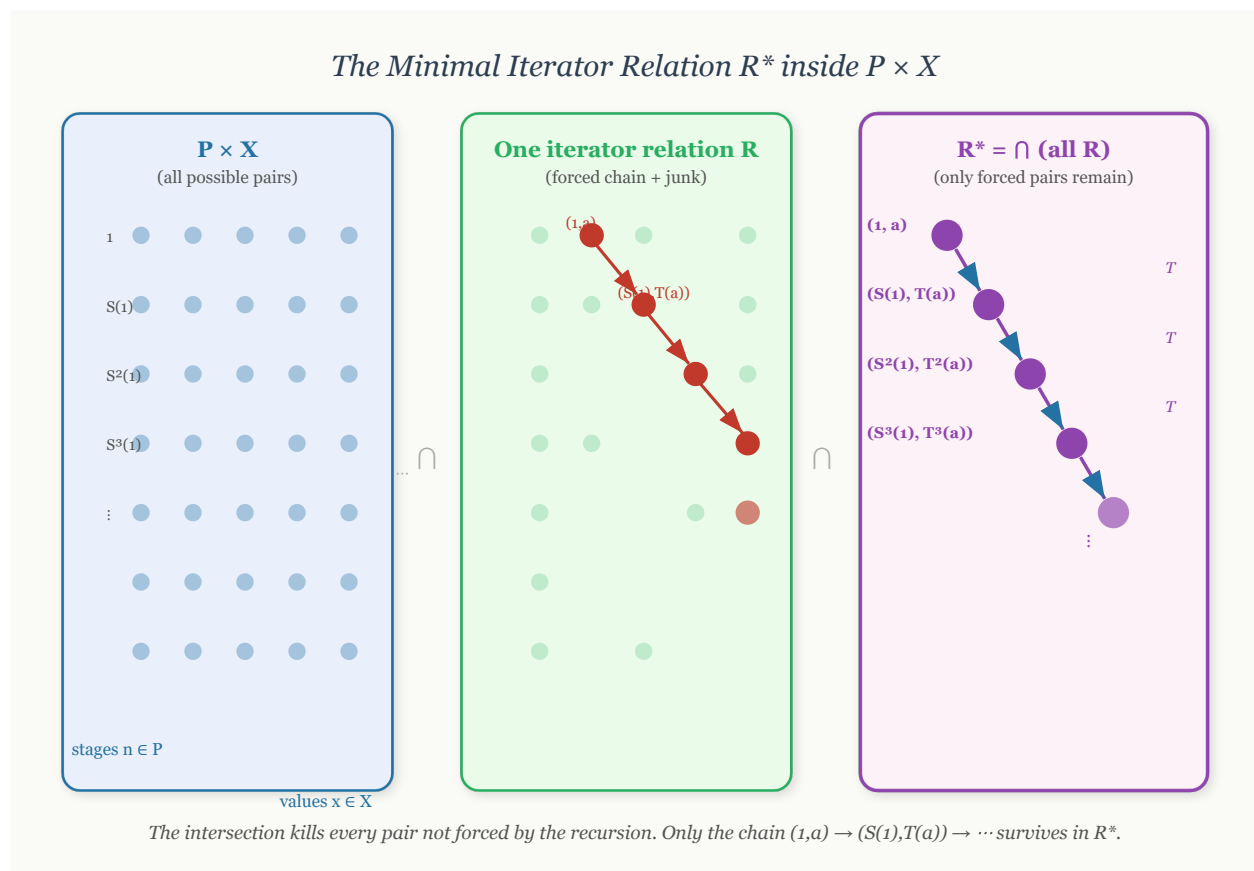


Figure 5.2: The intuitive construction of the minimal iterator relation  $R^*$  inside  $P \times W$ .



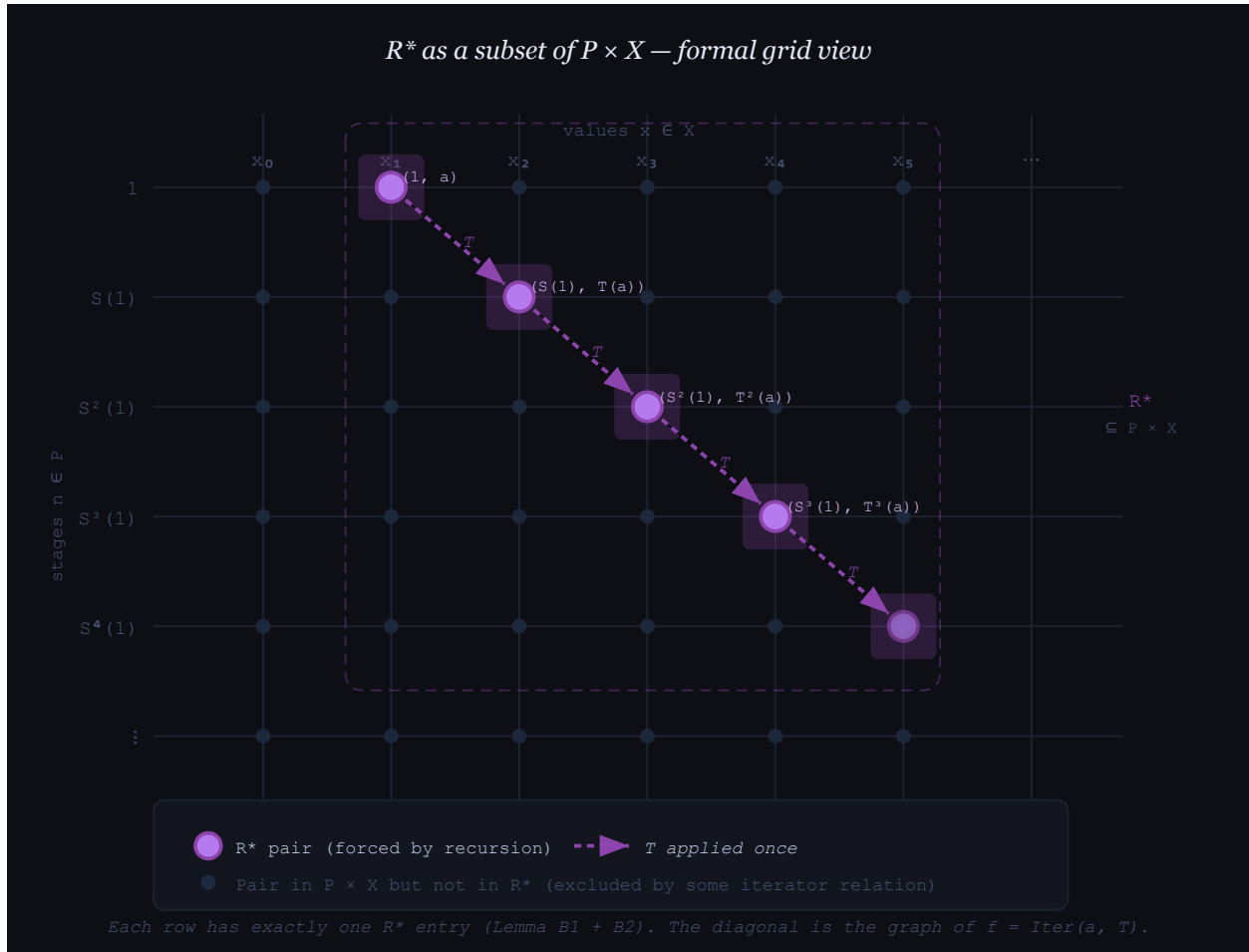
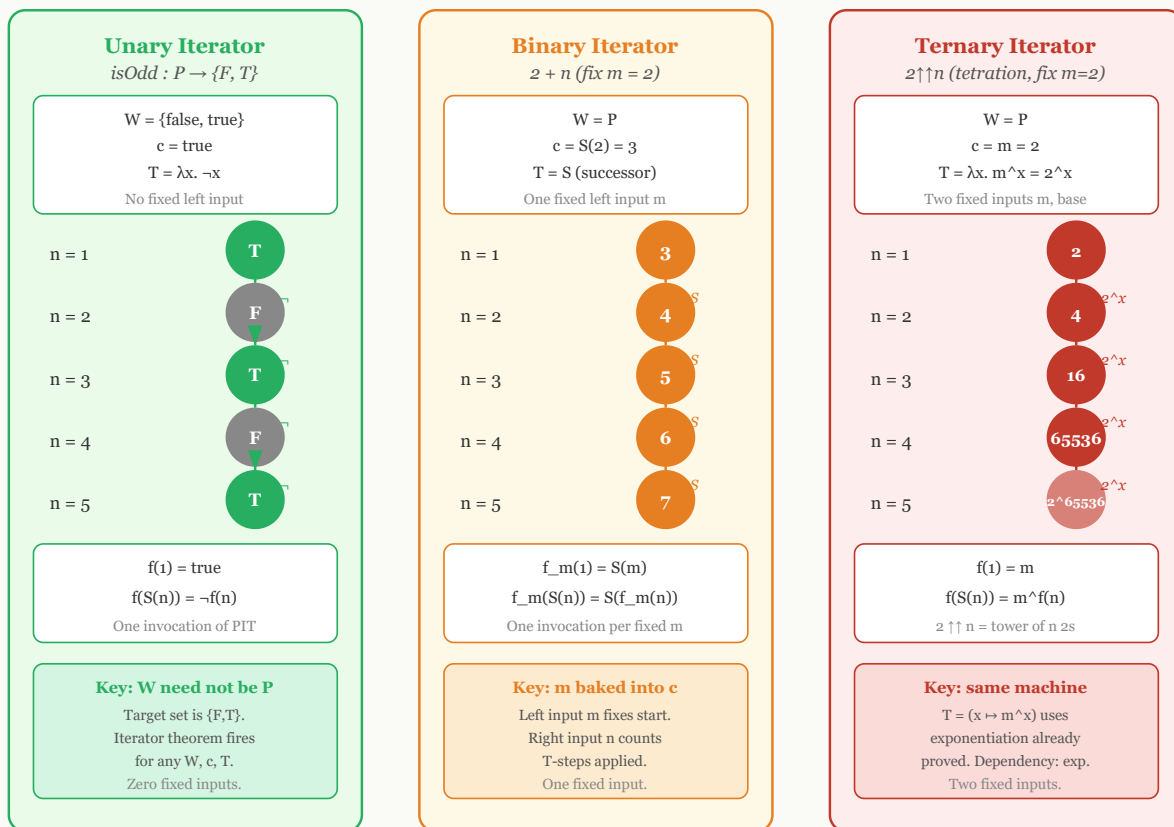


Figure 5.3: A formal grid view of  $R^*$  as the chain of forced ordered pairs in  $P \times W$ .



All three are configurations of  $(W, c, T)$ . The arity of the output operation = number of fixed inputs.

Figure 5.4: Unary, binary, and higher-arity iterator configurations after fixing the relevant parameters.

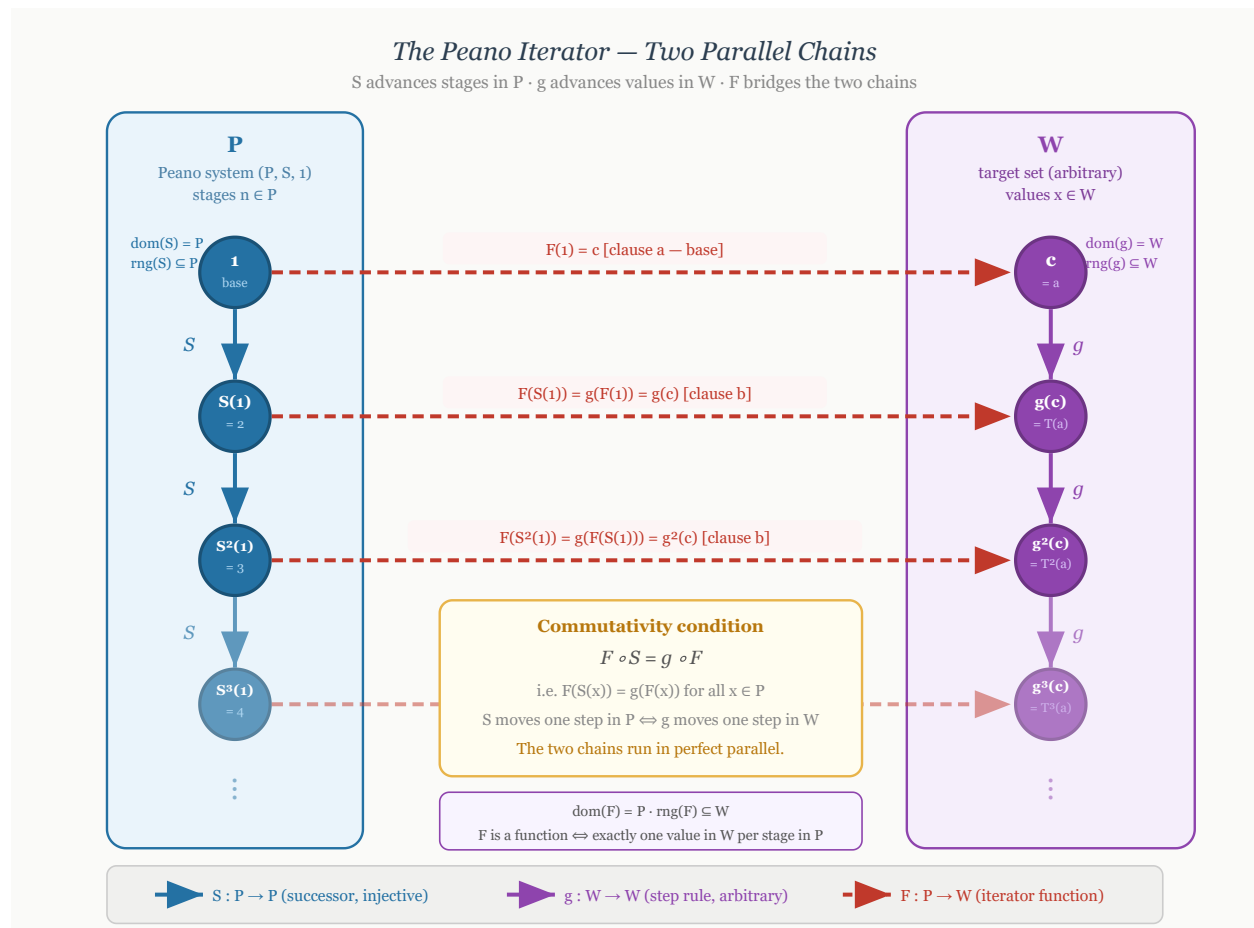


Figure 5.5: The Peano iterator as two parallel chains, one in  $P$  and one in  $W$ , connected by the iterator function.

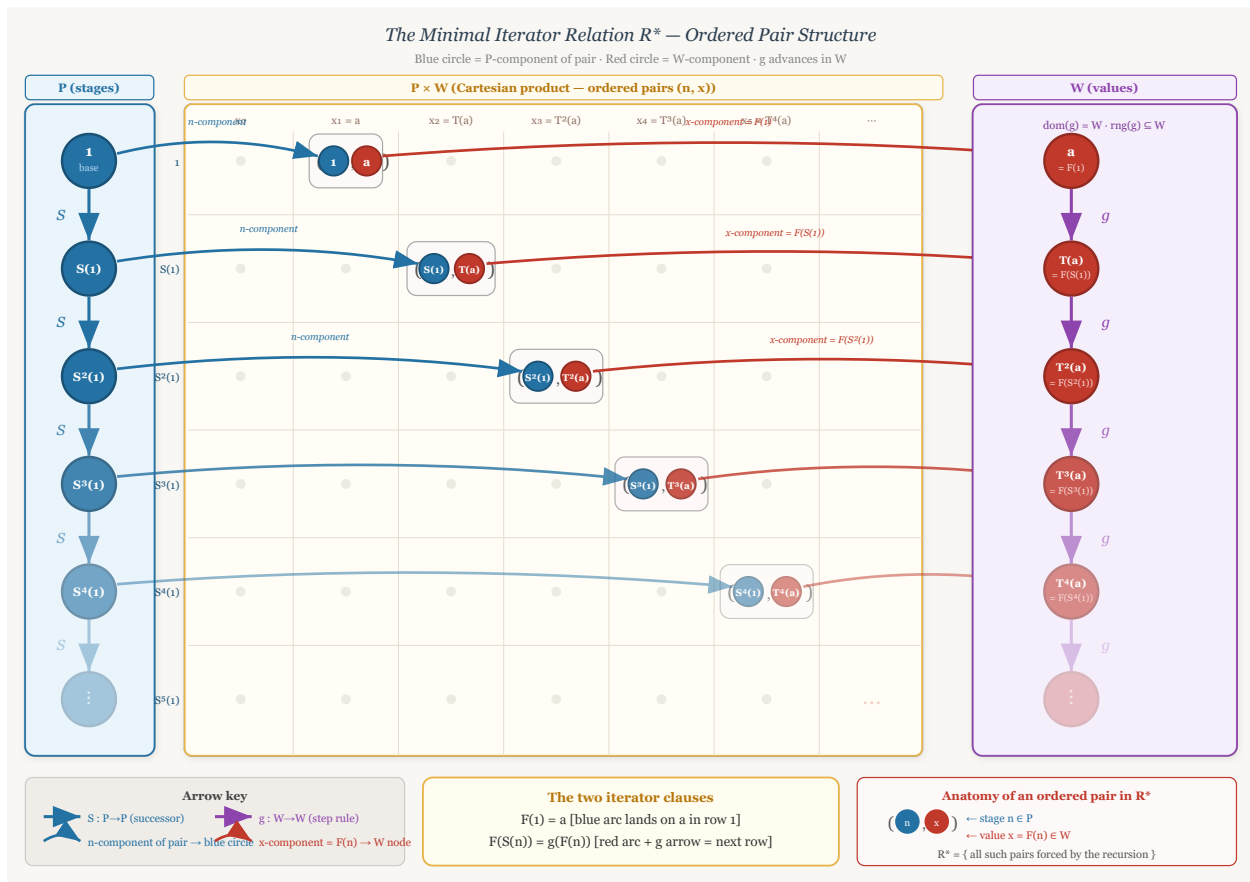


Figure 5.6: The three-zone view of  $R^*$ , separating stages in  $P$ , ordered pairs in  $P \times W$ , and values in  $W$ .

## 5.3 Examples of Peano Systems

examples-of-peano-systems

Peano Systems Defining Peano Systems → **Examples of Peano Systems** →  
Recursion on Peano Systems

### Peano Systems as Abstract Structures

The examples in this section separate the abstract idea of a Peano system from any particular representation of the natural numbers. A Peano system is not a specific set such as  $\mathbb{N}$ . It is a structure consisting of a carrier set, a distinguished initial element, and a successor operation satisfying the Peano conditions. Different carriers can therefore realize the same abstract successor structure.

*Remark* (Formal reference). The formal definition used throughout this section is [Peano System](#). In each example, the data are displayed as a triple  $(P, S, e)$  consisting of a carrier  $P$ , a successor map  $S : P \rightarrow P$ , and a distinguished first element  $e \in P$ .

*Remark* (Interpretation). The axioms say that the system has one starting point, the successor operation never merges two different elements, and induction reaches every element of the carrier. Thus the carrier is an infinite one-way chain generated from its initial element.

### The Standard One-Based System

#### The Usual One-Based Carrier

The familiar natural-number model is the reference example, but it is only one realization of the Peano axioms. The carrier is the usual one-based set of natural numbers, the distinguished element is 1, and successor is the usual next-number operation.

*Example* (The Usual Natural Numbers). Let

$$P = \mathbb{N}, \quad e = 1,$$

and define

$$S : P \rightarrow P, \quad S(n) = n + 1.$$

Then  $(P, S, e) = (\mathbb{N}, S, 1)$  is the usual one-based Peano system.

*Remark* (Structure). The Peano data are

$$(P, S, e) = (\mathbb{N}, n \mapsto n + 1, 1).$$

*Remark* (Interpretation). This is the familiar model, but it is not the only possible carrier for a Peano system. The remaining examples use different elements while preserving the same abstract successor pattern.

## Von Neumann Ordinals

### Set-Theoretic Carriers

Von Neumann ordinals realize natural-number structure inside set theory. In this representation, each stage is a set, and successor is formed by adjoining the current stage as a new element.

*Example* (The Zero-Based Von Neumann Chain). Let

$$0 = \varnothing, \quad 1 = \{0\}, \quad 2 = \{0, 1\}, \quad 3 = \{0, 1, 2\}, \quad \dots$$

Let  $P = \omega$ , let  $e = 0$ , and define

$$S : P \rightarrow P, \quad S(n) = n \cup \{n\}.$$

Then  $(P, S, e) = (\omega, S, 0)$  is a zero-based Peano system.

*Remark* (Structure). The Peano data are

$$(P, S, e) = (\omega, n \mapsto n \cup \{n\}, 0).$$

This example uses the zero-based convention common in set-theoretic foundations. The house convention for arithmetic in this volume remains one-based.

*Example* (The One-Based Von Neumann Chain). Let

$$P = \omega \setminus \{0\}, \quad e = 1,$$

and define

$$S : P \rightarrow P, \quad S(n) = n \cup \{n\}.$$

Then  $(P, S, e)$  is a one-based Peano system.

*Remark* (Structure). The Peano data are

$$(P, S, e) = (\omega \setminus \{0\}, n \mapsto n \cup \{n\}, 1).$$

*Remark* (Interpretation). In the Von Neumann model, each natural number is literally the set of all smaller natural numbers. This is not merely notation; it is a concrete set-theoretic realization of the abstract Peano structure.

## Nonnumeric Peano Systems

### Different Carriers, Same Successor Shape

A Peano carrier need not be the usual natural-number set, and its successor operation need not agree with the ambient order or arithmetic of a larger structure. What matters is the internally generated successor chain.

*Example* (The Powers of Ten). Let

$$P = \{10^{n-1} : n \in \mathbb{N}\}, \quad e = 1,$$

and define

$$S : P \rightarrow P, \quad S(10^{n-1}) = 10^n \quad (n \in \mathbb{N}).$$

Equivalently, for every  $x \in P$ ,

$$S(x) = 10x.$$

Then  $(P, S, e)$  is a Peano system.

*Remark* (Structure). The Peano data are

$$(P, S, e) = (\{1, 10, 10^2, 10^3, \dots\}, x \mapsto 10x, 1).$$

*Remark* (Interpretation). The elements are not all natural numbers. They are only powers of ten. Nevertheless, starting from 1 and repeatedly multiplying by 10 produces a successor chain with exactly the Peano shape:

$$1 \mapsto 10 \mapsto 10^2 \mapsto 10^3 \mapsto \dots$$

*Example* (Strings of Repeated Letters). Let  $\Sigma = \{a\}$  and let

$$P = \{a^n : n \in \mathbb{N}\}, \quad e = a,$$

where  $a^n$  denotes the string consisting of  $n$  copies of  $a$ . Define

$$S : P \rightarrow P, \quad S(a^n) = a^{n+1} \quad (n \in \mathbb{N}).$$

Then  $(P, S, e)$  is a Peano system.

*Remark* (Structure). The Peano data are

$$(P, S, e) = (\{a, aa, aaa, aaaa, \dots\}, a^n \mapsto a^{n+1}, a).$$

*Remark* (Interpretation). This example is useful because the elements are not numbers at all. They are finite strings. The Peano structure is carried by the successor relation, not by the surface appearance of the elements.

*Example* (The Reciprocal Chain). Let

$$P = \left\{ \frac{1}{n} : n \in \mathbb{N} \right\}, \quad e = 1,$$

and define

$$S : P \rightarrow P, \quad S\left(\frac{1}{n}\right) = \frac{1}{n+1} \quad (n \in \mathbb{N}).$$

Then  $(P, S, e)$  is a Peano system.

*Remark* (Structure). The Peano data are

$$(P, S, e) = \left( \left\{ 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots \right\}, \frac{1}{n} \mapsto \frac{1}{n+1}, 1 \right).$$

*Remark* (Interpretation). This is a good example precisely because the carrier is a decreasing sequence in the usual order on  $\mathbb{R}$ :

$$1 > \frac{1}{2} > \frac{1}{3} > \frac{1}{4} > \dots.$$

The Peano successor does not have to mean “larger than.” It only means the next element in the generated successor chain. The order inherited from  $\mathbb{R}$  is external structure, not part of the Peano-system data.

## A Finite Alphabet Is Not a Peano System

### A Failed Carrier

Finite chains help isolate what the Peano axioms forbid. A finite alphabet can support a partial next-letter operation, or it can support a cyclic successor operation, but neither choice gives a Peano system.

*Example* (The Finite Alphabet Fails). Let

$$P = \{A, B, C, \dots, Z\}.$$

If one tries to define

$$S(A) = B, \quad S(B) = C, \quad \dots,$$

then there is no value of  $S(Z)$  inside the same finite alphabet that extends the one-way chain. If instead one defines  $S(Z) = A$ , then  $S : P \rightarrow P$  is total but the successor operation forms a cycle.

*Remark* (Failure modes). The finite alphabet fails in one of two ways:

either  $S$  is not a total map  $P \rightarrow P$ ,  
or  $S$  loops back into a cycle.

The first failure violates closure of successor. The second failure violates the one-way successor structure forced by the Peano axioms.

*Remark* (Interpretation). A finite alphabet cannot be a Peano system. Peano systems are necessarily infinite because induction and successor closure generate an unending chain from the distinguished first element.



## Summary

### The Core Lesson

The carrier of a Peano system can be numerical, set-theoretic, symbolic, or embedded in another ordered structure. The common content is the same abstract successor pattern.

*Remark* (Examples). The following are Peano systems once their successor operations are specified:

$$\begin{aligned} &(\mathbb{N}, n \mapsto n + 1, 1), \\ &(\omega, n \mapsto n \cup \{n\}, 0), \\ &(\omega \setminus \{0\}, n \mapsto n \cup \{n\}, 1), \\ &(\{1, 10, 10^2, 10^3, \dots\}, x \mapsto 10x, 1), \\ &(\{a, aa, aaa, aaaa, \dots\}, a^n \mapsto a^{n+1}, a), \\ &\left( \left\{ 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots \right\}, \frac{1}{n} \mapsto \frac{1}{n+1}, 1 \right). \end{aligned}$$

They look different, but they have the same abstract Peano shape. Each is an infinite chain generated from a distinguished initial element by repeated successor.

## 5.4 Recursion on Peano Systems

### Why Recursion Requires a Theorem

The Peano axioms describe the shape of a Peano system: there is a first element, a successor operation, no predecessor of the first element, no successor collisions, and an induction principle. They do not, by themselves, say that every recursive specification determines a function. A recursive display such as

$$f(1) = c \quad \text{and} \quad f(S(n)) = g(f(n))$$

is only a specification. It becomes a definition only after one proves that there exists a unique function satisfying it. This is the “specification is not existence” point emphasized in Feferman’s treatment of recursive definition [Feferman, 1964, p. 67].

The proof used here follows the function-as-graph viewpoint. A function  $f : P \rightarrow W$  is a subset of  $P \times W$  whose elements are ordered pairs  $(n, w)$ , one for each stage  $n \in P$ . The construction therefore moves through three zones: the stage set  $P$ , the Cartesian product grid  $P \times W$ , and the value set  $W$ . The recursion theorem is the result that identifies the right graph inside  $P \times W$  and proves that it is the graph of exactly one function.

*Remark* (Prerequisite definition). An *inductive subset* of a Peano system is defined in [Inductive Subset of a Peano System](#). In this section, induction is used only through the earlier [Induction Principle for a Peano System](#).

## Arity Examples Before the Theorem

### Iterator Configurations

The iterator theorem has one formal shape but many uses. In every example below, the stage variable is the Peano stage  $n \in P$ , the value set is  $W$ , the first value is  $c$ , and the update rule is  $g : W \rightarrow W$ . These examples are motivating configurations only; the theorem establishing their legitimacy is proved afterward.

*Remark* (Example: isOdd).

| component | choice                          |        |
|-----------|---------------------------------|--------|
| $W$       | $\{\text{false}, \text{true}\}$ | $n$    |
| $c$       | true                            | $h(n)$ |
| $g$       | $x \mapsto \neg x$              |        |

|      |       |      |       |      |
|------|-------|------|-------|------|
| 1    | 2     | 3    | 4     | 5    |
| true | false | true | false | true |

The defining clauses are

$$h(1) = \text{true} \quad \text{and} \quad h(S(n)) = \neg h(n).$$

This example shows that the value set  $W$  need not be the Peano system  $P$ . The stages are natural-number-like, but the values are truth values.

*Remark* (Example: addition with the left parameter fixed). Fix the left parameter at  $m = 2$ .

| component | choice     |        |
|-----------|------------|--------|
| $W$       | $P$        | $n$    |
| $c$       | $S(2) = 3$ | $h(n)$ |
| $g$       | $S$        |        |

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| 3 | 4 | 5 | 6 | 7 |

The defining clauses are

$$h(1) = S(2) \quad \text{and} \quad h(S(n)) = S(h(n)).$$

This is the iterator pattern behind  $2 + n$  in the one-based convention. One parameter is fixed, and recursion runs in the remaining argument.

*Remark* (Example: tetration with the base fixed). Fix the base at  $m = 2$ .

| component | choice          |        |
|-----------|-----------------|--------|
| $W$       | $P$             | $n$    |
| $c$       | 2               | $h(n)$ |
| $g$       | $x \mapsto 2^x$ |        |

|   |       |           |               |                   |
|---|-------|-----------|---------------|-------------------|
| 1 | 2     | 3         | 4             | 5                 |
| 2 | $2^2$ | $2^{2^2}$ | $2^{2^{2^2}}$ | $2^{2^{2^{2^2}}}$ |

The defining clauses are

$$h(1) = 2 \quad \text{and} \quad h(S(n)) = 2^{h(n)}.$$

This example shows that the iterator theorem is not limited to the first arithmetic operations. Higher arithmetic still has the same formal configuration once the relevant operation on  $W$  is available.

*Remark* (Example: string repetition). Fix a string  $s \in \Sigma^*$ .

| component | choice                |                                                                                                                                                                               |
|-----------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $W$       | $\Sigma^*$            | $\frac{n}{h(n)} \mid \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \\ s & s \cdot s & s \cdot s \cdot s & s \cdot s \cdot s \cdot s & s \cdot s \cdot s \cdot s \cdot s \end{array}$ |
| $c$       | $s$                   |                                                                                                                                                                               |
| $g$       | $x \mapsto x \cdot s$ |                                                                                                                                                                               |

The defining clauses are

$$h(1) = s \quad \text{and} \quad h(S(n)) = h(n) \cdot s.$$

This example is not arithmetic. It illustrates that recursion on  $P$  can generate values in any set equipped with an appropriate update rule.

## Formal Development

### Minimal Relations and Iterator Functions

The formal proof is organized in Feferman's graph style. First define the admissible relations in  $P \times W$ . Then take their intersection. The resulting minimal relation is shown to be consistent, deterministic, and complete, hence it is the graph of the required function. This follows the minimal-relation construction used in Feferman, Hrbacek–Jech, and related lecture-note presentations of the recursion theorem [Feferman, 1964, Hrbacek and Jech, 1999, Freiwald, 2009].

#### Definition (Iterator Data)

**Definition 5.4.1** (Iterator Data). Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $g : W \rightarrow W$  be a function. The triple  $(W, c, g)$  is called *iterator data* for  $(P, S, 1)$ .

*Remark* (Standard quantified statement).

$$(W, c, g) \text{ is iterator data for } (P, S, 1) \iff (W \text{ is a set} \wedge c \in W \wedge g : W \rightarrow W).$$

*Remark* (Definition predicate reading).

$$\text{IteratorData}(W, c, g) \iff (W \text{ is a set} \wedge c \in W \wedge g : W \rightarrow W).$$

*Remark* (Negated quantified statement).

$$(W, c, g) \text{ is not iterator data for } (P, S, 1) \iff (W \text{ is not a set} \vee c \notin W \vee g \text{ is not a function } W \rightarrow W).$$

*Remark* (Failure modes). Iterator data fails if the proposed value collection is not a set, if the initial value does not lie in the value set, or if the proposed update rule is not a self-map of the value set.

*Remark* (Failure mode decomposition).

$$\begin{aligned}
 & (W, c, g) \text{ is not iterator data for } (P, S, 1) \\
 \iff & \underbrace{W \text{ is not a set}}_{\text{no value set}} \\
 & \vee \underbrace{c \notin W}_{\text{invalid initial value}} \\
 & \vee \underbrace{g \text{ is not a function } W \rightarrow W}_{\text{invalid update rule}}.
 \end{aligned}$$

*Remark* (Interpretation). Iterator data records the value set, the first value, and the fixed rule used to pass from each value to the next value.

*Remark* (Dependencies).

- [Peano System](#)

#### Definition (Iterator Relation)

**Definition 5.4.2** (Iterator Relation). Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. A relation  $R \subseteq P \times W$  is called an *iterator relation* for  $(W, c, g)$  exactly when

$$(1, c) \in R$$

and

$$\forall n \in P \forall w \in W ((n, w) \in R \implies (S(n), g(w)) \in R).$$

*Remark* (Standard quantified statement).

$$\begin{aligned}
 R \text{ is an iterator relation for } (W, c, g) \iff & R \subseteq P \times W \wedge (1, c) \in R \\
 & \wedge \forall n \in P \forall w \in W ((n, w) \in R \implies (S(n), g(w)) \in R).
 \end{aligned}$$

*Remark* (Definition predicate reading).

$$\begin{aligned}
 \text{IteratorRelation}(R, (P, S, 1), W, c, g) \iff & \text{Relation}(R, P, W) \wedge (1, c) \in R \\
 & \wedge \forall n \in P \forall w \in W ((n, w) \in R \implies (S(n), g(w)) \in R).
 \end{aligned}$$

*Remark* (Negated quantified statement).

$$\begin{aligned}
 R \text{ is not an iterator relation for } (W, c, g) \iff & R \not\subseteq P \times W \vee (1, c) \notin R \\
 & \vee \exists n \in P \exists w \in W ((n, w) \in R \wedge (S(n), g(w)) \notin R).
 \end{aligned}$$

*Remark* (Failure modes). An iterator relation fails if it is not a relation inside  $P \times W$ , if it omits the required base pair, or if it is not closed under the iterator step.

*Remark* (Failure mode decomposition).

$$R \text{ is not an iterator relation for } (W, c, g) \iff \underbrace{R \not\subseteq P \times W}_{\text{wrong ambient relation}} \vee \underbrace{(1, c) \notin R}_{\text{base pair missing}} \vee \underbrace{\exists n \in P \exists w \in W ((n, w) \in R \wedge (S(n), g(w)) \notin R)}_{\text{step closure fails}}.$$

*Remark* (Interpretation). An iterator relation is any graph-like set of possible stage-value pairs that contains the required first pair and is closed under the recursive step.

*Remark* (Dependencies).

- [Iterator Data](#)

#### Definition (Minimal Iterator Relation)

**Definition 5.4.3** (Minimal Iterator Relation). Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. The *minimal iterator relation* for  $(W, c, g)$  is

$$R^* = \bigcap \{R \subseteq P \times W : R \text{ is an iterator relation for } (W, c, g)\}.$$

#### Lean 4 Verification Record

**Source anchor:** Feferman, *The Number Systems*, Section 3.4; Mendelson, *Number Systems and the Foundations of Analysis*, Section 2.2.

**Formalizes:** [Definition 5.4.3](#).

**Lean module:** LRA.VolumeII.PeanoSystems.Recursion.

**Lean declaration:** minimal\_iterator\_relation.

**Status:** checked against lra-lean.

```
def minimal_iterator_relation
  (ps : PeanoSystem)
  (data : IteratorData ps)
  (element : ps.carrier)
  (value : data.target) : Prop :=
  forall relation : ps.carrier -> data.target -> Prop,
    iterator_relation ps data relation ->
    relation element value
```

*Remark* (Standard quantified statement).

$$(n, w) \in R^* \iff \forall R \subseteq P \times W (R \text{ is an iterator relation for } (W, c, g) \implies (n, w) \in R).$$

*Remark* (Definition predicate reading). Let  $\mathcal{C}_{\text{it}}$  be the closure condition

$$\mathcal{C}_{\text{it}}(R) \iff \text{IteratorRelation}(R, (P, S, 1), W, c, g).$$

Then the definition says

$$\text{MinimalRelation}(R^*, \mathcal{C}_{\text{it}}).$$

*Remark* (Negated quantified statement).

$$(n, w) \notin R^* \iff \exists R \subseteq P \times W \text{ (} R \text{ is an iterator relation for } (W, c, g) \wedge (n, w) \notin R \text{)}.$$

*Remark* (Failure modes). A pair fails to belong to the minimal iterator relation exactly when some iterator relation omits that pair. Such a pair is not forced by the base and successor-closure clauses.

*Remark* (Interpretation). The minimal iterator relation consists exactly of the stage-value pairs forced by the base pair and the successor closure rule. This is the  $R^*$  construction used in Feferman's proof of recursive definition, and in the related set-theoretic presentations cited here [Feferman, 1964, Hrbacek and Jech, 1999, Freiwald, 2009].

*Remark* (Dependencies).

- [Iterator Relation](#)

#### Lemma (Consistency of the Minimal Iterator Relation)

**Lemma 5.4.4** (Consistency of the Minimal Iterator Relation). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation for  $(W, c, g)$ . Then  $R^*$  is an iterator relation for  $(W, c, g)$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$R^* \text{ is an iterator relation for } (W, c, g).$$

*Remark* (Predicate reading).

$$\text{MinimalRelation}(R^*, \mathcal{C}_{\text{it}}) \implies \text{IteratorRelation}(R^*, (P, S, 1), W, c, g).$$

*Remark* (Negated quantified statement).

$$R^* \text{ is not an iterator relation for } (W, c, g).$$

*Remark* (Contrapositive quantified statement).

$$R^* \text{ is not an iterator relation for } (W, c, g) \implies R^* \text{ is not the minimal iterator relation for } (W, c, g). \blacksquare$$

*Remark* (Interpretation). The intersection construction does not lose the base pair or the successor closure condition. The minimal relation is therefore still one of the admissible iterator relations.

*Remark* (Dependencies).

- [Minimal Iterator Relation](#)

#### Lemma (Forced Values Are Unique)

**Lemma 5.4.5** (Forced Values Are Unique). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation. If  $(n, w_0) \in R^*$  and  $(n, w_1) \in R^*$ , then  $w_0 = w_1$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\forall n \in P \forall w_0, w_1 \in W ((n, w_0) \in R^* \wedge (n, w_1) \in R^* \implies w_0 = w_1).$$

*Remark* (Predicate reading).

$$\text{FunctionalRelation}(R^*, P, W).$$

*Remark* (Negated quantified statement).

$$\exists n \in P \exists w_0, w_1 \in W ((n, w_0) \in R^* \wedge (n, w_1) \in R^* \wedge w_0 \neq w_1).$$

*Remark* (Failure modes). Forced value uniqueness fails exactly when one stage is assigned two distinct values by the minimal iterator relation.

*Remark* (Contrapositive quantified statement).

$$\exists n \in P \exists w_0, w_1 \in W ((n, w_0) \in R^* \wedge (n, w_1) \in R^* \wedge w_0 \neq w_1) \implies R^* \text{ is not functional at every stage.} \blacksquare$$

*Remark* (Interpretation). No stage can be forced to carry two different values. This is the key determinacy step in the minimal-relation construction used to turn recursive specifications into functions [Feferman, 1964].

*Remark* (Dependencies).

- [Consistency of the Minimal Iterator Relation](#)

#### Lemma (Determinism of the Minimal Iterator Relation)

**Lemma 5.4.6** (Determinism of the Minimal Iterator Relation). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation. For each  $n \in P$ , there is at most one  $w \in W$  such that  $(n, w) \in R^*$ . [Go to proof.](#)*

## Lean 4 Verification Record

**Source anchor:** Feferman, *The Number Systems*, Section 3.4; Mendelson, *Number Systems and the Foundations of Analysis*, Section 2.2.

**Formalizes:** [Lemma 5.4.6](#).

**Lean module:** LRA.VolumeII.PeanoSystems.Recursion.

**Lean declaration:** minimal\_iterator\_relation\_deterministic.

**Status:** checked against lra-lean.

```
theorem minimal_iterator_relation_deterministic
  (ps : PeanoSystem)
  (data : IteratorData ps) :
  forall element : ps.carrier,
    forall first_value second_value : data.target,
      minimal_iterator_relation ps data element first_value ->
      minimal_iterator_relation ps data element second_value ->
      first_value = second_value := by
    apply induction_principle ps

  case base_case =>
    intro first_value second_value first_forced second_forced
    exact
      Eq.trans
        (minimal_iterator_relation_base_unique ps data first_value first_forced)
        (minimal_iterator_relation_base_unique ps data second_value second_forced).symm

  case successor_step =>
    intro element induction_hypothesis
    intro first_successor_value second_successor_value
    intro first_forced second_forced
    cases minimal_iterator_relation_complete ps data element with
    | intro current_value current_forced =>
      exact
        Eq.trans
          (forced_successor_values_are_unique
            ps data element current_value first_successor_value
            induction_hypothesis current_forced first_forced)
          (forced_successor_values_are_unique
            ps data element current_value second_successor_value
            induction_hypothesis current_forced second_forced).symm
```

*Remark* (Standard quantified statement).

$$\forall n \in P \forall w_0, w_1 \in W \left( (n, w_0) \in R^* \wedge (n, w_1) \in R^* \implies w_0 = w_1 \right).$$

*Remark* (Predicate reading).

$$\text{FunctionalRelation}(R^*, P, W).$$

*Remark* (Negated quantified statement).

$$\exists n \in P \exists w_0, w_1 \in W \left( (n, w_0) \in R^* \wedge (n, w_1) \in R^* \wedge w_0 \neq w_1 \right).$$

*Remark* (Failure modes). Determinism fails exactly when a single stage has two distinct values in the minimal iterator relation.

*Remark* (Dependencies).



- [Forced Values Are Unique](#)

#### Lemma (Completeness of the Minimal Iterator Relation)

**Lemma 5.4.7** (Completeness of the Minimal Iterator Relation). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation. For each  $n \in P$ , there exists  $w \in W$  such that  $(n, w) \in R^*$ . [Go to proof.](#)*

#### Lean 4 Verification Record

**Source anchor:** Feferman, *The Number Systems*, Section 3.4; Mendelson, *Number Systems and the Foundations of Analysis*, Section 2.2.

**Formalizes:** [Lemma 5.4.7](#).

**Lean module:** LRA.VolumeII.PeanoSystems.Recursion.

**Lean declaration:** minimal\_iterator\_relation\_complete.

**Status:** checked against lra-lean.

```
theorem minimal_iterator_relation_complete
  (ps : PeanoSystem)
  (data : IteratorData ps) :
  forall element : ps.carrier,
    Exists (fun value : data.target =>
      minimal_iterator_relation ps data element value) := by
  apply induction_principle ps

  case base_case =>
    exact
      Exists.intro
        data.initial_value
        (minimal_iterator_relation_is_iterator_relation ps data).left

  case successor_step =>
    intro element induction_hypothesis
    cases induction_hypothesis with
    | intro value value_is_forced =>
      exact
        Exists.intro
          (data.step_rule value)
          ((minimal_iterator_relation_is_iterator_relation ps data).right
            element value value_is_forced)
```

*Remark* (Standard quantified statement).

$$\forall n \in P \exists w \in W ((n, w) \in R^*).$$

*Remark* (Predicate reading).

$$\text{TotalRelation}(R^*, P, W).$$

*Remark* (Negated quantified statement).

$$\exists n \in P \forall w \in W ((n, w) \notin R^*).$$

*Remark* (Failure modes). Completeness fails exactly when some stage of  $P$  receives no value in the minimal iterator relation.

*Remark* (Source alignment). This is the totality half of Feferman’s minimal-relation construction for recursive definition: after determinacy gives at most one value at each stage, completeness gives at least one value at each stage [Feferman, 1964].

*Remark* (Dependencies).

- Consistency of the Minimal Iterator Relation
- Induction Principle for a Peano System

#### Lemma (Uniqueness of Iterator Functions)

**Lemma 5.4.8** (Uniqueness of Iterator Functions). *Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. If  $f, h : P \rightarrow W$  satisfy*

$$f(1) = c, \quad h(1) = c,$$

*and*

$$\forall n \in P (f(S(n)) = g(f(n))) \quad \text{and} \quad \forall n \in P (h(S(n)) = g(h(n))),$$

*then  $f = h$ . Go to proof.*

*Remark* (Standard quantified statement).

$$\begin{aligned} \forall f, h : P \rightarrow W \Big( & f(1) = c \wedge h(1) = c \\ & \wedge \forall n \in P (f(S(n)) = g(f(n))) \\ & \wedge \forall n \in P (h(S(n)) = g(h(n))) \implies f = h \Big). \end{aligned}$$

*Remark* (Predicate reading).

$$\forall f, h : P \rightarrow W \left( \text{IteratorFunction}(f, (P, S, 1), W, c, g) \wedge \text{IteratorFunction}(h, (P, S, 1), W, c, g) \implies f = h \right). \blacksquare$$

*Remark* (Negated quantified statement).

$$\begin{aligned} \exists f, h : P \rightarrow W \Big( & f(1) = c \wedge h(1) = c \\ & \wedge \forall n \in P (f(S(n)) = g(f(n))) \\ & \wedge \forall n \in P (h(S(n)) = g(h(n))) \wedge f \neq h \Big). \end{aligned}$$

*Remark* (Failure modes). Uniqueness fails exactly when two distinct functions satisfy the same base clause and the same successor clause.

*Remark* (Contrapositive quantified statement).

$$\begin{aligned} f \neq h \implies & f(1) \neq c \vee h(1) \neq c \\ & \vee \exists n \in P (f(S(n)) \neq g(f(n))) \\ & \vee \exists n \in P (h(S(n)) \neq g(h(n))). \end{aligned}$$

*Remark* (Interpretation). The iterator clauses determine at most one function. Any difference between two candidate functions must show up as a failure of the base clause or a failure of one of the successor clauses.

*Remark* (Dependencies).

- [Iterator Data](#)
- [Induction Principle for a Peano System](#)

#### Lemma (Existence of an Iterator Function)

**Lemma 5.4.9** (Existence of an Iterator Function). *Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. Then there exists a function  $f : P \rightarrow W$  such that*

$$f(1) = c \quad \text{and} \quad \forall n \in P (f(S(n)) = g(f(n))).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\exists f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n)))).$$

*Remark* (Predicate reading).

$$\exists f : P \rightarrow W \text{ IteratorFunction}(f, (P, S, 1), W, c, g).$$

*Remark* (Negated quantified statement).

$$\forall f : P \rightarrow W (f(1) \neq c \vee \exists n \in P (f(S(n)) \neq g(f(n)))).$$

*Remark* (Failure modes). Existence fails exactly when every candidate function misses the base value or misses the recursive successor equation at some stage.

*Remark* (Contrapositive quantified statement).

$$\neg \exists f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n)))) \implies R^* \text{ is not both deterministic and complete.} \blacksquare$$

*Remark* (Interpretation). Existence is the assertion that the minimal relation is not merely graph-like locally, but supplies a value at every stage and hence determines an actual function.

*Remark* (Dependencies).

- [Determinism of the Minimal Iterator Relation](#)
- [Completeness of the Minimal Iterator Relation](#)

### Theorem (Peano Iterator Theorem)

**Theorem 5.4.10** (Peano Iterator Theorem). *Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. Then there exists a unique function  $f : P \rightarrow W$  such that*

$$f(1) = c \quad \text{and} \quad \forall n \in P (f(S(n)) = g(f(n))).$$

*Go to proof.*

### Lean 4 Verification Record

**Source anchor:** Feferman, *The Number Systems*, Theorem 3.4; Mendelson, *Number Systems and the Foundations of Analysis*, Section 2.2.

**Formalizes:** [Theorem 5.4.10](#).

**Lean module:** `LRA.VolumeII.PeanoSystems.Recursion`.

**Lean declaration:** `peano_iterator_theorem`.

**Status:** checked against `lra-lean`.

```
theorem peano_iterator_theorem
  (ps : PeanoSystem)
  (target : Type)
  (initial_value : target)
  (step_rule : target -> target) :
  Exists (fun iterator_function : ps.carrier -> target =>
    And
      (satisfies_iterator_clauses
        ps target initial_value step_rule iterator_function)
      (forall other_iterator : ps.carrier -> target,
        satisfies_iterator_clauses
          ps target initial_value step_rule other_iterator ->
          forall element : ps.carrier,
            other_iterator element = iterator_function element)) := by
  refine
    Exists.intro
      (iter ps target initial_value step_rule)
    ?_
  constructor
  exact iter_satisfies_iterator_clauses ps target initial_value step_rule
  intro other_iterator other_satisfies element
  exact
    Eq.symm
      (iterator_function_unique
        ps target initial_value step_rule
        (iter ps target initial_value step_rule)
        other_iterator
        (iter_satisfies_iterator_clauses ps target initial_value step_rule)
        other_satisfies
        element)
```

*Remark* (Standard quantified statement).

$$\exists! f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n)))).$$

*Remark* (Predicate reading).

$$\exists! f : P \rightarrow W \text{ IteratorFunction}(f, (P, S, 1), W, c, g).$$

*Remark* (Negated quantified statement).

$$\neg \exists! f : P \rightarrow W \left( f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n))) \right).$$

*Remark* (Failure modes). The iterator theorem fails only if no function satisfies the iterator clauses or if more than one function satisfies them.

*Remark* (Failure mode decomposition).

$$\begin{aligned} & \neg \exists! f : P \rightarrow W \left( f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n))) \right) \\ \iff & \underbrace{\neg \exists f : P \rightarrow W \left( f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n))) \right)}_{\text{existence fails}} \\ & \vee \underbrace{\exists f, h : P \rightarrow W \left( f \neq h \wedge f(1) = c \wedge h(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n))) \wedge \forall n \in P (h(S(n)) = g(h(n))) \right)}_{\text{uniqueness fails}} \end{aligned}$$

*Remark* (Interpretation). The theorem says that iterator data is sufficient to turn a recursive specification into a genuine definition: there is exactly one graph and hence exactly one function satisfying the base and successor clauses.

*Remark* (Historical note). This is the one-based Peano-system version of the iterator theorem in [Feferman \[1964\]](#), [Mendelson \[1982\]](#), [Simpson \[2009\]](#). It is also the set-theoretic instance of the natural numbers object universal property [\[Lawvere, 1969\]](#); Dedekind's recursion theorem is the classical ancestor [\[Dedekind, 1901\]](#).

*Remark* (Dependencies).

- [Uniqueness of Iterator Functions](#)
- [Existence of an Iterator Function](#)

#### Definition (Iterator-Generated Function)

**Definition 5.4.11** (Iterator-Generated Function). Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. The unique function  $f : P \rightarrow W$  satisfying

$$f(1) = c \quad \text{and} \quad \forall n \in P (f(S(n)) = g(f(n)))$$

is called the *iterator-generated function determined by*  $(W, c, g)$ .

*Remark* (Standard quantified statement).

$f$  is the iterator-generated function determined by  $(W, c, g) \iff (f : P \rightarrow W \wedge f(1) = c \wedge \forall n \in P (f(S(n)) = g(f(n))))$

*Remark* (Definition predicate reading).

$\text{IteratorGeneratedFunction}(f, (P, S, 1), W, c, g) \iff \text{IteratorFunction}(f, (P, S, 1), W, c, g) \wedge \exists! h : P \rightarrow W \text{ It}$

*Remark* (Negated quantified statement).

$f$  is not the iterator-generated function determined by  $(W, c, g) \iff f$  is not a function  $P \rightarrow W$   
 $\vee f(1) \neq c \vee \exists n \in P (f(S(n)) \neq g(f(n)))$

*Remark* (Failure modes). A candidate fails to be the iterator-generated function if it is not a function from  $P$  to  $W$ , if it has the wrong base value, or if it violates the successor clause.

*Remark* (Interpretation). This definition packages the unique function supplied by the Peano Iterator Theorem as a named object available for later constructions.

*Remark* (Dependencies).

- [Peano Iterator Theorem](#)

#### Theorem (Iterator Base Clause)

**Theorem 5.4.12** (Iterator Base Clause). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $f$  be the iterator-generated function determined by  $(W, c, g)$ . Then*

$$f(1) = c.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$f$  is the iterator-generated function determined by  $(W, c, g) \implies f(1) = c$ .

*Remark* (Predicate reading).

$$\text{IteratorGeneratedFunction}(f, (P, S, 1), W, c, g) \implies f(1) = c.$$

*Remark* (Contrapositive quantified statement).

$f(1) \neq c \implies f$  is not the iterator-generated function determined by  $(W, c, g)$ .

*Remark* (Interpretation). The base clause is not an additional theorem about the generated function; it is one of the clauses built into the unique recursive specification.

*Remark* (Dependencies).

- [Iterator-Generated Function](#)

#### Theorem (Iterator Successor Clause)

**Theorem 5.4.13** (Iterator Successor Clause). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $f$  be the iterator-generated function determined by  $(W, c, g)$ . Then*

$$\forall n \in P (f(S(n)) = g(f(n))).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$f$  is the iterator-generated function determined by  $(W, c, g) \implies \forall n \in P (f(S(n)) = g(f(n)))$ . ■

*Remark* (Predicate reading).

$\text{IteratorGeneratedFunction}(f, (P, S, 1), W, c, g) \implies \forall n \in P (f(S(n)) = g(f(n)))$ .

*Remark* (Contrapositive quantified statement).

$\exists n \in P (f(S(n)) \neq g(f(n))) \implies f$  is not the iterator-generated function determined by  $(W, c, g)$ . ■

*Remark* (Interpretation). The successor clause says that the generated function advances by applying the update rule to the previous value at every Peano stage.

*Remark* (Dependencies).

- [Iterator-Generated Function](#)

#### Definition (Iteration of a Self-Map)

**Definition 5.4.14** (Iteration of a Self-Map). Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $g : W \rightarrow W$ . The iterator-generated function determined by  $(W, c, g)$  is called the *iteration of  $g$  from  $c$  along  $P$* .

*Remark* (Standard quantified statement).

$f$  is the iteration of  $g$  from  $c$  along  $P \iff f$  is the iterator-generated function determined by  $(W, c, g)$ . ■

*Remark* (Definition predicate reading).

$f$  is the iteration of  $g$  from  $c$  along  $P \iff \text{IteratorGeneratedFunction}(f, (P, S, 1), W, c, g)$ .

*Remark* (Negated quantified statement).

$f$  is not the iteration of  $g$  from  $c$  along  $P \iff f$  is not the iterator-generated function determined by  $(W, c, g)$ .

*Remark* (Interpretation). Iteration is the special case of recursion in which the same self-map is used at every stage.

*Remark* (Dependencies).

- [Iterator-Generated Function](#)

#### Corollary (Uniqueness of Binary Iterator Operations)

**Corollary 5.4.15** (Uniqueness of Binary Iterator Operations). Let  $(P, S, 1)$  be a Peano system, let  $A$  be a set, let  $W$  be a set, and suppose each  $a \in A$  determines iterator data  $(W, c_a, g_a)$  for  $(P, S, 1)$ . If  $F, G : A \times P \rightarrow W$  satisfy

$$F(a, 1) = c_a, \quad G(a, 1) = c_a$$

and

$$F(a, S(n)) = g_a(F(a, n)), \quad G(a, S(n)) = g_a(G(a, n))$$

for all  $a \in A$  and all  $n \in P$ , then  $F = G$ . [Go to proof](#).

*Remark* (Standard quantified statement).

$$\forall a \in A \forall n \in P \left( F(a, 1) = c_a \wedge G(a, 1) = c_a \right. \\ \left. \wedge F(a, S(n)) = g_a(F(a, n)) \wedge G(a, S(n)) = g_a(G(a, n)) \right) \implies F = G.$$

*Remark* (Predicate reading). For each  $a \in A$ , write  $F_a(n) = F(a, n)$  and  $G_a(n) = G(a, n)$ . Then

$$\forall a \in A \left( \text{IteratorFunction}(F_a, (P, S, 1), W, c_a, g_a) \wedge \text{IteratorFunction}(G_a, (P, S, 1), W, c_a, g_a) \implies F_a = G_a \right).$$

*Remark* (Negated quantified statement).

$$\exists a \in A \exists n \in P \left( F(a, 1) = c_a \wedge G(a, 1) = c_a \right. \\ \left. \wedge F(a, S(n)) = g_a(F(a, n)) \wedge G(a, S(n)) = g_a(G(a, n)) \wedge F \neq G \right).$$

*Remark* (Failure modes). Binary iterator uniqueness fails if two different binary operations satisfy the same one-parameter iterator specification for every fixed parameter.

*Remark* (Interpretation). For each fixed first argument, the binary operation is an ordinary iterator in the second argument. Uniqueness for the binary operation is therefore pointwise uniqueness of iterator functions.

*Remark* (Historical note). This isolates the binary-operation uniqueness pattern used by Thurston before existence is invoked [Thurston, 1956].

*Remark* (Dependencies).

- [Uniqueness of Iterator Functions](#)

## General Recursion

Pure iteration uses a fixed self-map  $g : W \rightarrow W$ . General recursion allows the successor rule to depend on the current stage as well as on the current value. The state-encoding theorem below reduces that stage-dependent case to the Peano Iterator Theorem.

### Definition (Stage-Dependent Step Rule)

**Definition 5.4.16** (Stage-Dependent Step Rule). Let  $(P, S, 1)$  be a Peano system and let  $W$  be a set. A *stage-dependent step rule* on  $W$  over  $P$  is a function

$$\Phi : P \times W \rightarrow W.$$

*Remark* (Standard quantified statement).

$$\Phi \text{ is a stage-dependent step rule on } W \text{ over } P \iff \Phi : P \times W \rightarrow W.$$



*Remark* (Definition predicate reading).

$$\text{StageDependentStepRule}(\Phi, P, W) \iff \Phi : P \times W \rightarrow W.$$

*Remark* (Negated quantified statement).

$\Phi$  is not a stage-dependent step rule on  $W$  over  $P \iff \Phi$  is not a function  $P \times W \rightarrow W$ .

*Remark* (Failure modes). A stage-dependent step rule fails precisely when it is not a well-defined function from stage-value pairs back into the value set.

*Remark* (Interpretation). A stage-dependent step rule allows the next value to depend on both the current Peano stage and the current value.

*Remark* (Dependencies).

- [Peano System](#)

#### Definition (General Recursive Function)

**Definition 5.4.17** (General Recursive Function). Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. A function  $f : P \rightarrow W$  is called the *general recursive function determined by*  $(W, c, \Phi)$  exactly when

$$f(1) = c \quad \text{and} \quad \forall n \in P (f(S(n)) = \Phi(n, f(n))).$$

*Remark* (Standard quantified statement).

$f$  is the general recursive function determined by  $(W, c, \Phi) \iff f : P \rightarrow W \wedge f(1) = c$   
 $\wedge \forall n \in P (f(S(n)) = \Phi(n, f(n))).$

*Remark* (Definition predicate reading).

$$\text{GeneralRecursiveFunction}(f, (P, S, 1), W, c, \Phi) \iff f : P \rightarrow W \wedge f(1) = c$$

$$\wedge \forall n \in P (f(S(n)) = \Phi(n, f(n))).$$

*Remark* (Negated quantified statement).

$f$  is not the general recursive function determined by  $(W, c, \Phi) \iff f$  is not a function  $P \rightarrow W$   
 $\vee f(1) \neq c \vee \exists n \in P (f(S(n)) \neq \Phi(n, f(n))).$

*Remark* (Failure modes). A candidate fails to be the general recursive function if it has the wrong domain or codomain, the wrong base value, or a failed stage-dependent successor equation.

*Remark* (Interpretation). General recursive functions are the stage-dependent analogues of iterator-generated functions.

Remark (Dependencies).

- [Stage-Dependent Step Rule](#)

**Lemma (Uniqueness of General Recursive Functions)**

**Lemma 5.4.18** (Uniqueness of General Recursive Functions). *Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. If  $f, h : P \rightarrow W$  satisfy*

$$f(1) = c, \quad h(1) = c,$$

*and*

$$\forall n \in P (f(S(n)) = \Phi(n, f(n))) \quad \text{and} \quad \forall n \in P (h(S(n)) = \Phi(n, h(n))),$$

*then  $f = h$ . [Go to proof](#).*

Remark (Standard quantified statement).

$$\begin{aligned} \forall f, h : P \rightarrow W \Big( & f(1) = c \wedge h(1) = c \\ & \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n))) \\ & \wedge \forall n \in P (h(S(n)) = \Phi(n, h(n))) \implies f = h \Big). \end{aligned}$$

Remark (Predicate reading).

$\forall f, h : P \rightarrow W \left( \text{GeneralRecursiveFunction}(f, (P, S, 1), W, c, \Phi) \wedge \text{GeneralRecursiveFunction}(h, (P, S, 1), W, c, \Phi) \implies f = h \right)$

Remark (Negated quantified statement).

$$\begin{aligned} \exists f, h : P \rightarrow W \Big( & f(1) = c \wedge h(1) = c \\ & \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n))) \\ & \wedge \forall n \in P (h(S(n)) = \Phi(n, h(n))) \wedge f \neq h \Big). \end{aligned}$$

Remark (Failure modes). Uniqueness of general recursive functions fails if two distinct functions satisfy the same base clause and the same stage-dependent successor clause.

Remark (Contrapositive quantified statement).

$$\begin{aligned} f \neq h \implies & f(1) \neq c \vee h(1) \neq c \\ & \vee \exists n \in P (f(S(n)) \neq \Phi(n, f(n))) \\ & \vee \exists n \in P (h(S(n)) \neq \Phi(n, h(n))). \end{aligned}$$

Remark (Interpretation). Once the initial value and stage-dependent rule are fixed, two recursive candidates cannot diverge without violating one of the defining clauses.

*Remark* (Dependencies).

- [General Recursive Function](#)
- [Induction Principle for a Peano System](#)

#### Theorem (General Recursion by State Encoding)

**Theorem 5.4.19** (General Recursion by State Encoding). *Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. There exists a function  $H : P \rightarrow P \times W$  such that*

$$H(1) = (1, c)$$

and

$$\forall n \in P \left( H(S(n)) = (S(\pi_1(H(n))), \Phi(\pi_1(H(n)), \pi_2(H(n)))) \right).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\exists H : P \rightarrow P \times W \left( H(1) = (1, c) \wedge \forall n \in P \ H(S(n)) = (S(\pi_1(H(n))), \Phi(\pi_1(H(n)), \pi_2(H(n)))) \right).$$

*Remark* (Predicate reading). Let  $\Gamma_\Phi : P \times W \rightarrow P \times W$  be defined by

$$\Gamma_\Phi(p, w) = (S(p), \Phi(p, w)).$$

Then state encoding is the iterator assertion

$$\exists H : P \rightarrow P \times W \text{ IteratorFunction}(H, (P, S, 1), P \times W, (1, c), \Gamma_\Phi).$$

*Remark* (Negated quantified statement).

$$\forall H : P \rightarrow P \times W \left( H(1) \neq (1, c) \vee \exists n \in P \ H(S(n)) \neq (S(\pi_1(H(n))), \Phi(\pi_1(H(n)), \pi_2(H(n)))) \right).$$

*Remark* (Failure modes). State encoding fails if every candidate state function has the wrong initial state or violates the encoded successor step at some stage.

*Remark* (Interpretation). The theorem converts stage-dependent recursion into ordinary iteration by carrying the current stage together with the current value as the recursive state.

*Remark* (Dependencies).

- [Peano Iterator Theorem](#)
- [Stage-Dependent Step Rule](#)

### Theorem (General Recursion Theorem)

**Theorem 5.4.20** (General Recursion Theorem). *Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. Then there exists a unique function  $f : P \rightarrow W$  such that*

$$f(1) = c \quad \text{and} \quad \forall n \in P (f(S(n)) = \Phi(n, f(n))).$$

*Go to proof.*

*Remark* (Standard quantified statement).

$$\exists! f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n)))).$$

*Remark* (Predicate reading).

$$\exists! f : P \rightarrow W \text{ GeneralRecursiveFunction}(f, (P, S, 1), W, c, \Phi).$$

*Remark* (Negated quantified statement).

$$\neg \exists! f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n)))).$$

*Remark* (Failure modes). General recursion fails if no function satisfies the general recursive clauses or if two distinct functions satisfy them.

*Remark* (Failure mode decomposition).

$$\begin{aligned} & \neg \exists! f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n)))) \\ \iff & \underbrace{\neg \exists f : P \rightarrow W (f(1) = c \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n))))}_{\text{existence fails}} \\ & \underbrace{\vee \exists f, h : P \rightarrow W (f \neq h \wedge f(1) = c \wedge h(1) = c \wedge \forall n \in P (f(S(n)) = \Phi(n, f(n))) \wedge \forall n \in P (h(S(n)) = \Phi(n, h(n))))}_{\text{uniqueness fails}} \end{aligned}$$

*Remark* (Interpretation). General recursion justifies recursive definitions whose successor rule depends on the current stage as well as the current value.

*Remark* (Historical note). This is the stage-dependent form corresponding to Feferman's Theorem 3.4' [Feferman, 1964]. It is the form used later for arithmetic definitions; those definitions are deferred to their own sections [Feferman, 1964, Mendelson, 1982].

*Remark* (Dependencies).

- [Uniqueness of General Recursive Functions](#)
- [General Recursion by State Encoding](#)

*Remark* (Deferred arithmetic definitions). Definitions of addition, multiplication, exponentiation, and later arithmetic operations are not part of this section. They begin after the recursion principles have been established.

### Theorem (Uniqueness of Peano Systems up to Isomorphism)

**Theorem 5.4.21** (Uniqueness of Peano Systems up to Isomorphism). *Let  $(P, S_P, 1_P)$  and  $(Q, S_Q, 1_Q)$  be Peano systems. Then there exists a unique bijection  $\theta : P \rightarrow Q$  such that*

$$\theta(1_P) = 1_Q$$

*and*

$$\forall n \in P (\theta(S_P(n)) = S_Q(\theta(n))).$$

*Thus any two Peano systems are isomorphic. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\exists! \theta : P \rightarrow Q (\theta \text{ is bijective} \wedge \theta(1_P) = 1_Q \wedge \forall n \in P (\theta(S_P(n)) = S_Q(\theta(n)))).$$

*Remark* (Predicate reading). Let  $A = (P, S_P, 1_P)$  and  $B = (Q, S_Q, 1_Q)$ . The theorem asserts

$$\text{Isomorphism}(A, B),$$

with unique witness  $\theta : P \rightarrow Q$  satisfying the displayed base and successor-preservation clauses.

*Remark* (Negated quantified statement).

$$\neg \exists! \theta : P \rightarrow Q (\theta \text{ is bijective} \wedge \theta(1_P) = 1_Q \\ \wedge \forall n \in P (\theta(S_P(n)) = S_Q(\theta(n)))).$$

*Remark* (Failure modes). Categoricity fails if no structure-preserving bijection exists between the two Peano systems or if more than one such bijection exists.

*Remark* (Interpretation). All Peano systems have the same structure up to a unique isomorphism. The elements may be named differently, but the distinguished first element and successor structure determine the translation.

*Remark* (Historical note). This is the categoricity theorem for Peano systems, following Feferman's Theorem 3.5 and Simpson's Peano-system presentation [[Feferman, 1964](#), [Simpson, 2009](#)].

*Remark* (Dependencies).

- [Peano System](#)
- [General Recursion Theorem](#)

## Proofs

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induction Principle for a Peano System). *Let  $(P, S, 1)$  be a Peano system, and let  $\varphi$  be a predicate on  $P$ . If*

$$\varphi(1) \quad \text{and} \quad \forall n \in P (\varphi(n) \implies \varphi(S(n))),$$

*then*

$$\forall n \in P \varphi(n).$$

*Professional Standard Proof.*

TODO: professional standard proof for induction-principle-for-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induction-principle-for-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Induction Axiom \(P5\)](#)
- [Inductive Subset of a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Strong Induction on a Peano System). *Let  $(P, S, 1)$  be a Peano system, and let  $\varphi$  be a predicate on  $P$ . If*

$$\forall n \in P \left( \forall k \in P (k < n \implies \varphi(k)) \right) \implies \varphi(n),$$

*then*

$$\forall n \in P \varphi(n).$$

*Professional Standard Proof.*

TODO: professional standard proof for strong-induction-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for strong-induction-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Induction Principle for a Peano System](#)
- [Strict Less Than on a Peano System](#)
- [Trichotomy on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Minimality of a Peano System). *Let  $(P, S, 1)$  be a Peano system. If  $A \subseteq P$  contains 1 and is closed under successor, then  $A = P$ .*

*Professional Standard Proof.*

TODO: professional standard proof for peano-minimality. □

*Detailed Learning Proof.*

TODO: detailed learning proof for peano-minimality. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Induction Axiom](#)
- [Inductive Subset of a Peano System](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Every Element Is Either 1 or a Successor). *Let  $(P, S, 1)$  be a Peano system. For every  $x \in P$ , either  $x = 1$  or there exists  $u \in P$  such that  $S(u) = x$ .*

*Professional Standard Proof.*

TODO: professional standard proof for every-element-is-one-or-a-successor. □

*Detailed Learning Proof.*

TODO: detailed learning proof for every-element-is-one-or-a-successor. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Distinguished Element Lies in the Underlying Set](#)
- [Closure Under Successor](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Successor Inequality Reflection). *Let  $(P, S, 1)$  be a Peano system. For all  $a, b \in P$ , if  $a \neq b$ , then  $S(a) \neq S(b)$ .*

*Professional Standard Proof.*

TODO: professional standard proof for successor-inequality-reflection. □

*Detailed Learning Proof.*

TODO: detailed learning proof for successor-inequality-reflection. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Injectivity of Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Non-One Elements Have a Predecessor). *Let  $(P, S, 1)$  be a Peano system. For every  $x \in P$ , if  $x \neq 1$ , then there exists  $u \in P$  such that  $S(u) = x$ .*

*Professional Standard Proof.*

TODO: professional standard proof for non-one-elements-have-a-predecessor. □

*Detailed Learning Proof.*

TODO: detailed learning proof for non-one-elements-have-a-predecessor. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Every Element Is Either 1 or a Successor](#)

*Remark* (Return). [Return to Corollary](#)

**Corollary** (Predecessor Existence and Uniqueness Away from 1). *Let  $(P, S, 1)$  be a Peano system, and let  $x \in P$  with  $x \neq 1$ . Then there exists a unique  $u \in P$  such that  $S(u) = x$ .*

*Professional Standard Proof.*

TODO: professional standard proof for predecessor-exists-unique-away-from-one. □

*Detailed Learning Proof.*

TODO: detailed learning proof for predecessor-exists-unique-away-from-one. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Every Element Is Either 1 or a Successor](#)
- [Injectivity of Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Unique Predecessor Characterization Away from 1). *Let  $(P, S, 1)$  be a Peano system. For every  $x \in P$ ,  $x \neq 1$  if and only if there exists a unique  $u \in P$  such that  $S(u) = x$ .*

*Professional Standard Proof.*

TODO: professional standard proof for unique-predecessor-characterization-away-from-one. □

*Detailed Learning Proof.*

TODO: detailed learning proof for unique-predecessor-characterization-away-from-one. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Distinguished Element Is Not a Successor](#)
- [Predecessor Existence and Uniqueness Away from 1](#)

*Remark* (Return). [Return to Corollary](#)

**Corollary** (The Distinguished Element Is the Unique Non-Successor). *Let  $(P, S, 1)$  be a Peano system. An element  $x \in P$  is not a successor of any element of  $P$  if and only if  $x = 1$ .*

*Professional Standard Proof.*

TODO: professional standard proof for one-is-unique-non-successor. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-is-unique-non-successor. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Distinguished Element Is Not a Successor](#)
- [Every Element Is Either 1 or a Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (No Object Is Its Own Successor). *Let  $(P, S, 1)$  be a Peano system. For every  $n \in P$ ,*

$$S(n) \neq n.$$

*Professional Standard Proof.*

TODO: professional standard proof for no-object-is-its-own-successor. □

*Detailed Learning Proof.*

TODO: detailed learning proof for no-object-is-its-own-successor. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Distinguished Element Is Not a Successor](#)
- [Injectivity of Successor](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Lemma](#)

*Remark* (Proof status).

*Remark* (Proof vault). [Handwritten proof record](#)

**Lemma** (Consistency of the Minimal Iterator Relation). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation for  $(W, c, g)$ . Then  $R^*$  is an iterator relation for  $(W, c, g)$ .*

*Professional Standard Proof.*

TODO: professional standard proof for iterator-relation-consistency. □

*Detailed Learning Proof.*

TODO: detailed learning proof for iterator-relation-consistency. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Iterator Data on a Peano System](#)
- [Iterator Relation](#)
- [Minimal Iterator Relation](#)



*Remark* (Return). [Return to Theorem](#)

**Lemma** (Forced Values Are Unique). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation. If  $(n, w_0) \in R^*$  and  $(n, w_1) \in R^*$ , then  $w_0 = w_1$ .*

*Professional Standard Proof.*

TODO: professional standard proof for forced-values-are-unique. □

*Detailed Learning Proof.*

TODO: detailed learning proof for forced-values-are-unique. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Consistency of the Minimal Iterator Relation](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Determinism of the Minimal Iterator Relation). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation. For each  $n \in P$ , there is at most one  $w \in W$  such that  $(n, w) \in R^*$ .*

*Professional Standard Proof.*

TODO: professional standard proof for minimal-iterator-relation-deterministic. □

*Detailed Learning Proof.*

TODO: detailed learning proof for minimal-iterator-relation-deterministic. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Forced Values Are Unique](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Completeness of the Minimal Iterator Relation). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $R^*$  be the minimal iterator relation. For each  $n \in P$ , there exists  $w \in W$  such that  $(n, w) \in R^*$ .*

*Professional Standard Proof.*

TODO: professional standard proof for minimal-iterator-relation-complete. □

*Detailed Learning Proof.*

TODO: detailed learning proof for minimal-iterator-relation-complete. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Consistency of the Minimal Iterator Relation](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Uniqueness of Iterator Functions). *Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. If  $f, h : P \rightarrow W$  satisfy*

$$f(1) = c, \quad h(1) = c,$$

*and*

$$\forall n \in P (f(S(n)) = g(f(n))) \quad \text{and} \quad \forall n \in P (h(S(n)) = g(h(n))),$$

*then  $f = h$ .*

*Professional Standard Proof.*

TODO: professional standard proof for uniqueness-of-iterator-functions. □

*Detailed Learning Proof.*

TODO: detailed learning proof for uniqueness-of-iterator-functions. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Iterator Data](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Existence of an Iterator Function). *Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. Then there exists a function  $f : P \rightarrow W$  such that*

$$f(1) = c \quad \text{and} \quad \forall n \in P \ (f(S(n)) = g(f(n))).$$

*Professional Standard Proof.*

TODO: professional standard proof for existence-of-iterator-function. □

*Detailed Learning Proof.*

TODO: detailed learning proof for existence-of-iterator-function. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Determinism of the Minimal Iterator Relation](#)
- [Completeness of the Minimal Iterator Relation](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Peano Iterator Theorem). *Let  $(P, S, 1)$  be a Peano system, and let  $(W, c, g)$  be iterator data for it. Then there exists a unique function  $f : P \rightarrow W$  such that*

$$f(1) = c \quad \text{and} \quad \forall n \in P \ (f(S(n)) = g(f(n))).$$

*Professional Standard Proof.*

TODO: professional standard proof for peano-iterator-theorem. □

*Detailed Learning Proof.*

TODO: detailed learning proof for peano-iterator-theorem. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Uniqueness of Iterator Functions](#)
- [Existence of an Iterator Function](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Iterator Base Clause). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $f$  be the iterator-generated function determined by  $(W, c, g)$ . Then*

$$f(1) = c.$$

*Professional Standard Proof.*

TODO: professional standard proof for iterator-base-value. □

*Detailed Learning Proof.*

TODO: detailed learning proof for iterator-base-value. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Iterator-Generated Function](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Iterator Successor Clause). *Let  $(P, S, 1)$  be a Peano system, let  $(W, c, g)$  be iterator data for it, and let  $f$  be the iterator-generated function determined by  $(W, c, g)$ . Then*

$$\forall n \in P \ (f(S(n)) = g(f(n))).$$

*Professional Standard Proof.*

TODO: professional standard proof for iterator-successor-step. □

*Detailed Learning Proof.*

TODO: detailed learning proof for iterator-successor-step. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Iterator-Generated Function](#)



*Remark* (Return). [Return to Theorem](#)

**Corollary** (Uniqueness of Binary Iterator Operations). *Let  $(P, S, 1)$  be a Peano system, let  $A$  be a set, let  $W$  be a set, and suppose each  $a \in A$  determines iterator data  $(W, c_a, g_a)$  for  $(P, S, 1)$ . If  $F, G : A \times P \rightarrow W$  satisfy*

$$F(a, 1) = c_a, \quad G(a, 1) = c_a$$

*and*

$$F(a, S(n)) = g_a(F(a, n)), \quad G(a, S(n)) = g_a(G(a, n))$$

*for all  $a \in A$  and all  $n \in P$ , then  $F = G$ .*

*Professional Standard Proof.*

TODO: professional standard proof for uniqueness-of-binary-iterator-operations. □

*Detailed Learning Proof.*

TODO: detailed learning proof for uniqueness-of-binary-iterator-operations. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Uniqueness of Iterator Functions](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Uniqueness of General Recursive Functions). *Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. If  $f, h : P \rightarrow W$  satisfy*

$$f(1) = c, \quad h(1) = c,$$

*and*

$$\forall n \in P (f(S(n)) = \Phi(n, f(n))) \quad \text{and} \quad \forall n \in P (h(S(n)) = \Phi(n, h(n))),$$

*then  $f = h$ .*

*Professional Standard Proof.*

TODO: professional standard proof for uniqueness-of-general-recursive-functions. □

*Detailed Learning Proof.*

TODO: detailed learning proof for uniqueness-of-general-recursive-functions. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [General Recursive Function](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (General Recursion by State Encoding). *Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. There exists a function  $H : P \rightarrow P \times W$  such that*

$$H(1) = (1, c)$$

*and*

$$\forall n \in P \left( H(S(n)) = (S(\pi_1(H(n))), \Phi(\pi_1(H(n)), \pi_2(H(n)))) \right).$$

*Professional Standard Proof.*

TODO: professional standard proof for general-recursion-by-state-encoding. □

*Detailed Learning Proof.*

TODO: detailed learning proof for general-recursion-by-state-encoding. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano Iterator Theorem](#)
- [Stage-Dependent Step Rule](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (General Recursion Theorem). *Let  $(P, S, 1)$  be a Peano system, let  $W$  be a set, let  $c \in W$ , and let  $\Phi : P \times W \rightarrow W$  be a stage-dependent step rule. Then there exists a unique function  $f : P \rightarrow W$  such that*

$$f(1) = c \quad \text{and} \quad \forall n \in P \ (f(S(n)) = \Phi(n, f(n))).$$

*Professional Standard Proof.*

TODO: professional standard proof for general-recursion-theorem-for-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for general-recursion-theorem-for-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Uniqueness of General Recursive Functions](#)
- [General Recursion by State Encoding](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Uniqueness of Peano Systems up to Isomorphism). *Let  $(P, S_P, 1_P)$  and  $(Q, S_Q, 1_Q)$  be Peano systems. Then there exists a unique bijection  $\theta : P \rightarrow Q$  such that*

$$\theta(1_P) = 1_Q$$

*and*

$$\forall n \in P \ (\theta(S_P(n)) = S_Q(\theta(n))).$$

*Thus any two Peano systems are isomorphic.*

*Professional Standard Proof.*

TODO: professional standard proof for uniqueness-of-peano-systems-up-to-isomorphism.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for uniqueness-of-peano-systems-up-to-isomorphism.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [General Recursion Theorem](#)

# Capstone

# Chapter 6

## Natural Numbers ( $\mathbb{N}$ )



### 6.1 Axioms for $\mathbb{N}$

#### 6.1.1 Axioms for $\mathbb{N}$

##### The Canonical Peano System

The natural numbers are now fixed as the canonical one-based Peano system. The preceding Peano chapter developed the abstract interface  $(X, e, S)$ ; in this chapter,  $\mathbb{N}$  is the distinguished instance obtained by taking the base element to be 1. The symbol 0 is not part of this system. It is adjoined in the next chapter when the whole numbers are constructed.

##### Definition (The Natural Numbers)

**Definition 6.1.1** (The Natural Numbers). The natural numbers  $\mathbb{N}$  are the canonical Peano system

$$(\mathbb{N}, 1, S),$$

where  $1 \in \mathbb{N}$  is the distinguished base element and  $S : \mathbb{N} \rightarrow \mathbb{N}$  is the successor operation.

*Remark* (Standing convention). Throughout this chapter, variables  $m, n, k$  range over  $\mathbb{N}$  unless a larger ambient set is explicitly named. The element 1 is the base natural number, and  $S(n)$  is the successor of  $n$ .

*Remark* (Dependencies).

- [Peano System](#)

## Peano Axioms Specialized to $\mathbb{N}$

The following axioms are not new principles beyond the Peano-system interface; they are the abstract Peano axioms specialized to the canonical natural-number system. Naming them here makes later references inside the arithmetic chapter point directly to  $\mathbb{N}$  rather than to an arbitrary Peano system.

### Axiom (N1, Base)

**Axiom 6.1.2** (N1, Base).

$$1 \in \mathbb{N}.$$

### Axiom (N2, Successor Closure)

**Axiom 6.1.3** (N2, Successor Closure).

$$\forall n \in \mathbb{N} (S(n) \in \mathbb{N}).$$

### Axiom (N3, Base Has No Predecessor)

**Axiom 6.1.4** (N3, Base Has No Predecessor).

$$\forall n \in \mathbb{N} (S(n) \neq 1).$$

### Axiom (N4, Successor Injectivity)

**Axiom 6.1.5** (N4, Successor Injectivity).

$$\forall m, n \in \mathbb{N} (S(m) = S(n) \implies m = n).$$

### Axiom (N5, Induction)

**Axiom 6.1.6** (N5, Induction). For every predicate  $\varphi$  on  $\mathbb{N}$ ,

$$\left( \varphi(1) \wedge \forall n \in \mathbb{N} (\varphi(n) \implies \varphi(S(n))) \right) \implies \forall n \in \mathbb{N} \varphi(n).$$

## 6.2 Canonical Natural Number System

### Working in $\mathbb{N}$

The preceding chapter established the Peano-system facts needed for arithmetic: induction, successor analysis, recursion, and categoricity. From this point forward,  $\mathbb{N}$  denotes the canonical natural-number Peano system, with distinguished first element 1 and successor map  $S$ . The arithmetic operations introduced in this chapter are defined inside that system by the recursion principles already proved for Peano systems.



*Remark* (Standing convention). Unless otherwise stated, variables  $m, n, k$  range over  $\mathbb{N}$ , the symbol 1 denotes the distinguished first natural number, and  $S(n)$  denotes the successor of  $n$ .

## 6.3 Addition on $\mathbb{N}$

### Recursive Definition of Addition

Addition on a Peano system is defined from the successor primitive by recursion. For each fixed left input  $m$ , the map  $n \mapsto m + n$  is generated from the initial value  $S(m)$  and the successor operator on the same Peano system. This definition makes addition a derived operation rather than an independent axiom.

#### Definition (Addition on a Peano System)

**Definition 6.3.1** (Addition on a Peano System). Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , define the function

$$f_m : P \rightarrow P$$

to be the unique function satisfying

$$f_m(1) = S(m) \quad \text{and} \quad \forall n \in P (f_m(S(n)) = S(f_m(n))).$$

For  $m, n \in P$ , define

$$m + n := f_m(n).$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , let  $f_m : P \rightarrow P$  satisfy  
 $f_m(1) = S(m) \quad \text{and} \quad \forall n \in P (f_m(S(n)) = S(f_m(n))).$   
 Then for all  $m, n \in P$ ,  $m + n := f_m(n)$ .

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , let  $f_m : P \rightarrow P$  satisfy  
 $f_m(1) = S(m) \quad \text{and} \quad \forall n \in P (f_m(S(n)) = S(f_m(n))).$   
 Then for all  $m, n, x \in P$ ,  $x \neq m + n \iff x \neq f_m(n)$ .

*Remark* (Failure modes). The definition fails only when the designated value does not agree with the unique recursively generated function attached to the fixed left input  $m$ . Once the recursive map  $f_m$  is fixed, every incorrect candidate for  $m + n$  is simply a value of  $P$  different from  $f_m(n)$ .

*Remark* (Failure mode decomposition).

Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , let  $f_m : P \rightarrow P$  satisfy  
 $f_m(1) = S(m)$       and       $\forall n \in P (f_m(S(n)) = S(f_m(n)))$ .  
 $x \neq m + n \iff \underbrace{x \neq f_m(n)}_{\text{candidate value disagrees with the recursive output}}.$

*Remark* (Interpretation). Addition is introduced by fixing the left input and recursively generating the right-input map. The value  $m + 1$  is set to be the successor of  $m$ , and each further successor step in the right input advances the output by one more successor. This makes addition repeated succession.

*Remark* (Dependencies).

- [Peano System](#)
- [Peano Iterator Theorem](#)

#### Theorem (Addition Is Well-Defined on a Peano System)

**Theorem 6.3.2** (Addition Is Well-Defined on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ , there exists a unique function*

$$f_m : P \rightarrow P$$

*such that*

$$f_m(1) = S(m) \quad \text{and} \quad \forall n \in P (f_m(S(n)) = S(f_m(n))).$$

*Consequently the rule*

$$(m, n) \longmapsto m + n := f_m(n)$$

*defines a unique binary operation on  $P$ . [Go to proof](#).*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m \in P \exists! f_m : P \rightarrow P \left( f_m(1) = S(m) \wedge \forall n \in P (f_m(S(n)) = S(f_m(n))) \right).$$

*Remark* (Interpretation). The theorem shows that the recursive clauses for addition determine exactly one right-input map for each fixed left input. The addition operation is therefore not assumed; it is constructed from the successor map by recursion.

*Remark* (Source alignment). This is the house-style well-definedness form of addition obtained by applying the Peano recursion theorem in the manner of Feferman's construction of arithmetic operations [[Feferman, 1964](#)].

*Remark* (Dependencies).

- [Addition on a Peano System](#)
- [Peano Iterator Theorem](#)

## Recursive Clauses as Reusable Equalities

The recursive definition of addition is given by the base value and successor step for the right-input map  $n \mapsto m + n$ . Those clauses are part of the definition, but they become substantially more useful once isolated as named theorems. Later algebraic arguments repeatedly cite these equalities rather than reopening the recursive construction each time.

### Theorem (Addition With 1)

**Theorem 6.3.3** (Addition With 1). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ ,*

$$m + 1 = S(m).$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m \in P (m + 1 = S(m)).$$

*Remark* (Interpretation). This theorem records the base clause of the recursive definition in the operation notation itself. Adding the distinguished first element applies one successor step to the left input.

*Remark* (Dependencies).

- [Addition on a Peano System](#)

### Theorem (Addition Successor on the Right)

**Theorem 6.3.4** (Addition Successor on the Right). *Let  $(P, S, 1)$  be a Peano system. For every  $m, n \in P$ ,*

$$m + S(n) = S(m + n).$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m, n \in P (m + S(n) = S(m + n)).$$

*Remark* (Interpretation). This theorem records the recursive step of addition in operation notation. Moving one successor step forward in the right input moves the sum one successor step forward as well.

*Remark* (Dependencies).

- [Addition on a Peano System](#)

## Left Input Initialization

The recursive definition controls addition when the right input is advanced by successor. To use addition symmetrically, the first structural task is to understand what happens when the distinguished first element appears on the left. The next theorem shows that left addition by 1 also produces a single successor step.

### Theorem (One Plus $n$ )

**Theorem 6.3.5** (One Plus  $n$ ). *Let  $(P, S, 1)$  be a Peano system. For every  $n \in P$ ,*

$$1 + n = S(n).$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall n \in P (1 + n = S(n)).$$

*Remark* (Interpretation). The theorem shows that the distinguished first element acts on either side as a single successor step. This is the first sign that the recursively defined operation is not tied permanently to the asymmetry of its definition.

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [Induction Principle for a Peano System](#)

## Associativity and Commutativity

Once the recursive clauses are available as reusable theorems, addition can be shown to satisfy the two fundamental algebraic symmetries of a binary operation. Associativity makes repeated addition unambiguous, and commutativity shows that the recursively defined operation does not depend on which input is treated as the evolving one.

### Theorem (Addition Is Associative)

**Theorem 6.3.6** (Addition Is Associative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a + b) + c = a + (b + c).$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P ((a + b) + c = a + (b + c)).$$

*Remark* (Interpretation). Associativity shows that successive additions can be regrouped without changing the value. This makes longer sums structurally coherent and prepares addition for later interaction with order and multiplication.

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [Induction Principle for a Peano System](#)

#### Theorem (Addition Is Commutative)

**Theorem 6.3.7** (Addition Is Commutative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a + b = b + a.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b \in P (a + b = b + a).$$

*Remark* (Interpretation). Although addition is defined by recursion on the right input, the resulting operation is symmetric in its two arguments. Commutativity shows that the apparent asymmetry of the defining clauses is an artifact of construction rather than a feature of the operation itself.

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [One Plus  \$n\$](#)
- [Addition Is Associative](#)
- [Induction Principle for a Peano System](#)

## Cancellation Laws

The next structural property shows that addition preserves enough information to allow one to remove a common summand from an equation. These cancellation laws are the first substitute for subtraction inside a Peano system and are indispensable when order is later defined through additive comparison.

### Theorem (Left Cancellation for Addition)

**Theorem 6.3.8** (Left Cancellation for Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a + b = a + c \implies b = c.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P \ ((a + b = a + c) \implies b = c).$$

*Remark* (Contrapositive quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P \ (b \neq c \implies a + b \neq a + c).$$

*Remark* (Interpretation). Left cancellation shows that adding the same element to two inputs cannot erase a difference between them. This is the algebraic core that later makes additive definitions of order behave well.

*Remark* (Dependencies).

- [Injectivity of Successor](#)
- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [One Plus  \$n\$](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Induction Principle for a Peano System](#)

### Theorem (Right Cancellation for Addition)

**Theorem 6.3.9** (Right Cancellation for Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$b + a = c + a \implies b = c.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall a, b, c \in P \ ((b + a = c + a) \implies b = c).$

*Remark* (Contrapositive quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall a, b, c \in P \ (b \neq c \implies b + a \neq c + a).$

*Remark* (Interpretation). Right cancellation is the companion to left cancellation. Once commutativity is known, the placement of the common summand no longer matters, so equality can be tested after adding the same element on either side.

*Remark* (Dependencies).

- [Addition Is Commutative](#)
- [Left Cancellation for Addition](#)

#### Theorem (Cancellation for Addition)

**Theorem 6.3.10** (Cancellation for Addition). *Let  $(P, S, 1)$  be a Peano system. Addition on  $P$  satisfies cancellation on both sides:*

$$a + b = a + c \implies b = c \quad \text{and} \quad b + a = c + a \implies b = c.$$

*Go to proof.*

*Remark* (Interpretation). The left and right cancellation laws together say that adding a common natural number never destroys equality information, regardless of which side contains the common summand.

*Remark* (Dependencies).

- [Left Cancellation for Addition](#)
- [Right Cancellation for Addition](#)

### Additive Non-Return

Addition on  $\mathbb{N}$  always moves forward by at least one successor step in its right input. The next theorem records the resulting non-return law: adding a natural number to  $m$  never returns the added natural number itself.

### Theorem (No Natural Number Equals Itself Plus a Natural Number)

**Theorem 6.3.11** (No Natural Number Equals Itself Plus a Natural Number). *Let  $(P, S, 1)$  be a Peano system. For every  $m, n \in P$ ,*

$$n \neq m + n.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m, n \in P (n \neq m + n).$$

*Remark* (Interpretation). Because every element of  $P$  is at least one successor step, the sum  $m + n$  lies strictly beyond  $n$ . The theorem rules out additive cycles and is a standard tool for order arguments on the natural numbers.

*Remark* (Source alignment). This is the house-style form of the additive non-return theorem in [Feferman, 1964].

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [No Object Is Its Own Successor](#)
- [Induction Principle for a Peano System](#)

## 6.4 Multiplication on $\mathbb{N}$

### Recursive Definition of Multiplication

Multiplication on a Peano system is defined from addition by recursion. For each fixed left input  $m$ , the map  $n \mapsto m \cdot n$  is generated by starting at  $m$  and adding one further copy of  $m$  at each successor stage of the right input. This makes multiplication repeated addition rather than an independent primitive operation.



### Definition (Multiplication on a Peano System)

**Definition 6.4.1** (Multiplication on a Peano System). Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , define the function

$$g_m : P \rightarrow P$$

to be the unique function satisfying

$$g_m(1) = m \quad \text{and} \quad \forall n \in P (g_m(S(n)) = g_m(n) + m).$$

For  $m, n \in P$ , define

$$m \cdot n := g_m(n).$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , let  $g_m : P \rightarrow P$  satisfy  
 $g_m(1) = m$  and  $\forall n \in P (g_m(S(n)) = g_m(n) + m)$ .  
 Then for all  $m, n \in P$ ,  $m \cdot n := g_m(n)$ .

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , let  $g_m : P \rightarrow P$  satisfy  
 $g_m(1) = m$  and  $\forall n \in P (g_m(S(n)) = g_m(n) + m)$ .  
 Then for all  $m, n, x \in P$ ,  $x \neq m \cdot n \iff x \neq g_m(n)$ .

*Remark* (Failure modes). The definition fails only when a proposed value for  $m \cdot n$  does not agree with the recursively generated function attached to the fixed left factor  $m$ . Once  $g_m$  is determined, every incorrect candidate product is simply a value different from  $g_m(n)$ .

*Remark* (Failure mode decomposition).

Let  $(P, S, 1)$  be a Peano system. For each  $m \in P$ , let  $g_m : P \rightarrow P$  satisfy  
 $g_m(1) = m$  and  $\forall n \in P (g_m(S(n)) = g_m(n) + m)$ .  
 $x \neq m \cdot n \iff \underbrace{x \neq g_m(n)}_{\text{candidate value disagrees with the recursive output}}.$

*Remark* (Interpretation). Multiplication is introduced by fixing the left input and recursively generating the right-input map. The value  $m \cdot 1$  is initialized at  $m$ , and each successor step in the right input adds one further copy of  $m$ . Multiplication is therefore repeated addition.

*Remark* (Dependencies).

- [Peano System](#)
- [General Recursion Theorem for a Peano System](#)

- [Addition on a Peano System](#)

#### Theorem (Multiplication Is Well-Defined on a Peano System)

**Theorem 6.4.2** (Multiplication Is Well-Defined on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ , there exists a unique function*

$$g_m : P \rightarrow P$$

*such that*

$$g_m(1) = m \quad \text{and} \quad \forall n \in P (g_m(S(n)) = g_m(n) + m).$$

*Consequently the rule*

$$(m, n) \mapsto m \cdot n := g_m(n)$$

*defines a unique binary operation on  $P$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m \in P \exists! g_m : P \rightarrow P \left( g_m(1) = m \wedge \forall n \in P (g_m(S(n)) = g_m(n) + m) \right).$$

*Remark* (Interpretation). The recursive clauses for multiplication determine exactly one right-input map for each fixed left input. The multiplication operation is therefore constructed from recursion and addition rather than assumed at the outset.

*Remark* (Source alignment). This is the house-style well-definedness form of multiplication obtained by applying the Peano recursion theorem in the manner of Feferman's construction of arithmetic operations [Feferman, 1964].

*Remark* (Dependencies).

- [Multiplication on a Peano System](#)
- [General Recursion Theorem for a Peano System](#)
- [Addition on a Peano System](#)

### Recursive Clauses as Reusable Equalities

The recursive definition of multiplication is given by a base value and a successor step. As with addition, these clauses become more useful once they are isolated as named theorems that can be cited directly in later algebraic arguments.

### Theorem (Multiplication With 1)

**Theorem 6.4.3** (Multiplication With 1). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ ,*

$$m \cdot 1 = m.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m \in P (m \cdot 1 = m).$$

*Remark* (Interpretation). This theorem records the base clause of the recursive definition in operation notation. Multiplying by the distinguished first element leaves the left input unchanged.

*Remark* (Dependencies).

- [Multiplication on a Peano System](#)

### Theorem (Multiplication Successor on the Right)

**Theorem 6.4.4** (Multiplication Successor on the Right). *Let  $(P, S, 1)$  be a Peano system. For every  $m, n \in P$ ,*

$$m \cdot S(n) = (m \cdot n) + m.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall m, n \in P (m \cdot S(n) = (m \cdot n) + m).$$

*Remark* (Interpretation). Moving one successor step forward in the right input adds one further copy of  $m$  to the product. This is the recursive step of multiplication written in operation notation.

*Remark* (Dependencies).

- [Multiplication on a Peano System](#)
- [Addition on a Peano System](#)

## Left Identity

The recursive definition controls multiplication when the right input evolves. To use multiplication symmetrically, the first structural task is to understand what happens when the distinguished first element appears on the left. The next theorem identifies that left action explicitly.

### Theorem (One Times $n$ )

**Theorem 6.4.5** (One Times  $n$ ). *Let  $(P, S, 1)$  be a Peano system. For every  $n \in P$ ,*

$$1 \cdot n = n.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall n \in P (1 \cdot n = n).$$

*Remark* (Interpretation). The distinguished first element acts as a multiplicative identity on the left. This is the multiplicative analogue of the theorem that  $1 + n$  produces the successor of  $n$  in the additive theory.

*Remark* (Dependencies).

- [Multiplication With 1](#)
- [Multiplication Successor on the Right](#)
- [Induction Principle for a Peano System](#)

## Distributivity and Associativity

Multiplication is built from repeated addition, so its first major structural law is distributivity over addition. Once distributivity is available, the operation can be shown to satisfy associativity, which makes iterated products well-defined without ambiguity of grouping.

### Theorem (Multiplication Distributes Over Addition)

**Theorem 6.4.6** (Multiplication Distributes Over Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P \ (a \cdot (b + c) = (a \cdot b) + (a \cdot c)).$$

*Remark* (Interpretation). Distributivity expresses formally that multiplying by  $a$  means repeatedly adding copies of  $a$ , so a sum in the second factor splits into two separate blocks of repeated addition. This is the bridge between the additive and multiplicative structures.

*Remark* (Dependencies).

- [Multiplication Successor on the Right](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Induction Principle for a Peano System](#)

#### Theorem (Left Distributivity of Multiplication Over Addition)

**Theorem 6.4.7** (Left Distributivity of Multiplication Over Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a + b) \cdot c = (a \cdot c) + (b \cdot c).$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P \ ((a + b) \cdot c = (a \cdot c) + (b \cdot c)).$$

*Remark* (Interpretation). The existing distributive law expands a sum in the right factor. This theorem expands a sum in the left factor. It follows from right distributivity together with commutativity of multiplication.

*Remark* (Source alignment). This is the house-style form of Feferman's left distributive law for multiplication [[Feferman, 1964](#)].

*Remark* (Dependencies).

- [Multiplication Distributes Over Addition](#)
- [Multiplication Is Commutative](#)
- [Addition Is Commutative](#)

### Theorem (Multiplication Distributes over Addition on Both Sides)

**Theorem 6.4.8** (Multiplication Distributes over Addition on Both Sides). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c) \quad \text{and} \quad (a + b) \cdot c = (a \cdot c) + (b \cdot c).$$

[Go to proof.](#)

*Remark* (Dependencies).

- [Multiplication Distributes Over Addition](#)
- [Left Distributivity of Multiplication Over Addition](#)

### Theorem (Multiplication Is Associative)

**Theorem 6.4.9** (Multiplication Is Associative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a \cdot b) \cdot c = a \cdot (b \cdot c).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P ((a \cdot b) \cdot c = a \cdot (b \cdot c)).$$

*Remark* (Interpretation). Associativity shows that repeated multiplication can be regrouped without changing the value. Once this law is established, longer products can be used without parenthetical ambiguity.

*Remark* (Dependencies).

- [Multiplication Distributes Over Addition](#)
- [Multiplication With 1](#)
- [Multiplication Successor on the Right](#)
- [Induction Principle for a Peano System](#)

## Commutativity

The recursive definition of multiplication treats the right input as the evolving variable, but the resulting operation is not meant to privilege one factor over the other. The next theorem shows that the asymmetry of the construction disappears in the final operation.

### Theorem (Multiplication Is Commutative)

**Theorem 6.4.10** (Multiplication Is Commutative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a \cdot b = b \cdot a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b \in P (a \cdot b = b \cdot a).$$

*Remark* (Interpretation). Although multiplication is defined recursively in one argument, the resulting operation is symmetric in the two factors. Commutativity shows that repeated addition of  $a$  to build  $a \cdot b$  agrees exactly with repeated addition of  $b$  to build  $b \cdot a$ .

*Remark* (Dependencies).

- [One Times  \$n\$](#)
- [Multiplication Successor on the Right](#)
- [Multiplication Distributes Over Addition](#)
- [Addition Is Commutative](#)
- [Induction Principle for a Peano System](#)

### Cancellation Laws

Multiplication by a natural number preserves enough information to allow a common nonzero factor to be removed from an equality. Since every element of the one-based natural-number system is nonzero, no extra nonzero hypothesis is needed in  $\mathbb{N}$ .

### Theorem (Left Cancellation for Multiplication)

**Theorem 6.4.11** (Left Cancellation for Multiplication). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \cdot b = a \cdot c \implies b = c.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  $\forall a, b, c \in P (a \cdot b = a \cdot c \implies b = c).$

*Remark* (Interpretation). Left cancellation says that multiplying two values by the same natural number cannot collapse a genuine difference between them.

*Remark* (Dependencies).

- [Multiplication Preserves and Reflects Strict Order](#)
- [Trichotomy on a Peano System](#)

#### Theorem (Right Cancellation for Multiplication)

**Theorem 6.4.12** (Right Cancellation for Multiplication). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$b \cdot a = c \cdot a \implies b = c.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  $\forall a, b, c \in P (b \cdot a = c \cdot a \implies b = c)$ .

*Remark* (Interpretation). Right cancellation is the symmetric form of multiplicative cancellation. It follows from left cancellation together with commutativity of multiplication.

*Remark* (Dependencies).

- [Left Cancellation for Multiplication](#)
- [Multiplication Is Commutative](#)

#### Theorem (Cancellation for Multiplication)

**Theorem 6.4.13** (Cancellation for Multiplication). *Let  $(P, S, 1)$  be a Peano system. Multiplication on  $P$  satisfies cancellation on both sides:*

$$a \cdot b = a \cdot c \implies b = c \quad \text{and} \quad b \cdot a = c \cdot a \implies b = c.$$

*Go to proof.*

*Remark* (Dependencies).

- [Left Cancellation for Multiplication](#)
- [Right Cancellation for Multiplication](#)



## 6.5 Order on $\mathbb{N}$

### Strict and Non-Strict Order

Order on a Peano system is extracted from addition. Since the chapter is using the distinguished first element 1 rather than an additive zero, the strict order is the cleaner primitive notion:  $a < b$  means that  $b$  is reached from  $a$  by adding a nontrivial amount. The non-strict order is then obtained by allowing equality in addition to strict increase.

#### Definition (Strict Less Than on a Peano System)

**Definition 6.5.1** (Strict Less Than on a Peano System). Let  $(P, S, 1)$  be a Peano system, and let  $a, b \in P$ . One writes

$$a < b$$

exactly when there exists  $c \in P$  such that

$$b = a + c.$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $a, b \in P$ .

$$a < b \iff \exists c \in P (b = a + c).$$

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $a, b \in P$ .

$$a \not< b \iff \forall c \in P (b \neq a + c).$$

*Remark* (Failure modes). Strict inequality fails exactly when no additive witness in  $P$  carries  $a$  to  $b$ . In that case, every nontrivial additive step from  $a$  misses  $b$ .

*Remark* (Failure mode decomposition).

Let  $(P, S, 1)$  be a Peano system and let  $a, b \in P$ .

$$a \not< b \iff \underbrace{\forall c \in P (b \neq a + c)}_{\text{every proposed additive witness fails}}.$$

*Remark* (Interpretation). The strict order records whether  $b$  lies genuinely ahead of  $a$  in the counting structure. The witness  $c$  is the amount of additive displacement needed to move from  $a$  to  $b$ .

*Remark* (Dependencies).

- [Addition on a Peano System](#)

### Definition (Less Than or Equal on a Peano System)

**Definition 6.5.2** (Less Than or Equal on a Peano System). Let  $(P, S, 1)$  be a Peano system, and let  $a, b \in P$ . One writes

$$a \leq b$$

exactly when

$$a < b \quad \text{or} \quad a = b.$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $a, b \in P$ .

$$a \leq b \iff (a < b \vee a = b).$$

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $a, b \in P$ .

$$a \not\leq b \iff (a \not< b \wedge a \neq b).$$

*Remark* (Failure modes). The non-strict order fails in exactly two simultaneous ways:  $a$  is not strictly below  $b$ , and  $a$  is not equal to  $b$ . Thus  $b$  neither lies ahead of  $a$  nor coincides with it.

*Remark* (Failure mode decomposition).

Let  $(P, S, 1)$  be a Peano system and let  $a, b \in P$ .

$$a \not\leq b \iff \underbrace{a \not< b}_{\text{no strict additive witness}} \wedge \underbrace{a \neq b}_{\text{equality also fails}}.$$

*Remark* (Interpretation). The relation  $a \leq b$  allows either equality or a genuine forward step from  $a$  to  $b$ . It is the additive order relation derived from the strict witness definition.

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)

### Basic Order Laws

Once the order relations have been defined, the first task is to show that they satisfy the expected structural laws: reflexivity, transitivity, and antisymmetry. These are the properties that make the additive comparison relation behave as an honest order rather than a merely suggestive notation.

### Theorem (Order Is Reflexive on a Peano System)

**Theorem 6.5.3** (Order Is Reflexive on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a \in P$ ,*

$$a \leq a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a \in P (a \leq a).$$

*Remark* (Interpretation). Every element is comparable to itself. Reflexivity supplies the equality case inside the non-strict order.

*Remark* (Dependencies).

- [Less Than or Equal on a Peano System](#)

### Theorem (Order Is Transitive on a Peano System)

**Theorem 6.5.4** (Order Is Transitive on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a \leq b \wedge b \leq c) \implies a \leq c.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P ((a \leq b \wedge b \leq c) \implies a \leq c).$$

*Remark* (Interpretation). If  $b$  lies at or beyond  $a$ , and  $c$  lies at or beyond  $b$ , then  $c$  also lies at or beyond  $a$ . Transitivity expresses the consistency of additive comparison along a chain.

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Addition Is Associative](#)

### Theorem (Order Is Antisymmetric on a Peano System)

**Theorem 6.5.5** (Order Is Antisymmetric on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$(a \leq b \wedge b \leq a) \implies a = b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b \in P \ ((a \leq b \wedge b \leq a) \implies a = b).$$

*Remark* (Contrapositive quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b \in P \ (a \neq b \implies \neg(a \leq b \wedge b \leq a)).$$

*Remark* (Interpretation). Two distinct elements cannot each lie at or beyond the other. Antisymmetry separates genuine equality from bidirectional comparability.

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Left Cancellation for Addition](#)
- [Addition Is Associative](#)
- [No Object Is Its Own Successor](#)

### Compatibility with Addition

The order on a Peano system is built from addition, so it must interact coherently with addition. The next theorems show that adding the same element to both sides preserves the comparison relation, regardless of whether the common summand is placed on the right or on the left.

### Theorem (Addition Preserves Order on the Right)

**Theorem 6.5.6** (Addition Preserves Order on the Right). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \leq b \implies a + c \leq b + c.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P \ (a \leq b \implies a + c \leq b + c).$$

*Remark* (Interpretation). Adding the same quantity to two comparable elements does not disturb their order. The additive witness for  $a \leq b$  survives after both sides are shifted by the same summand.

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Addition Is Associative](#)

### Theorem (Addition Preserves Order on the Left)

**Theorem 6.5.7** (Addition Preserves Order on the Left). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \leq b \implies c + a \leq c + b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P \ (a \leq b \implies c + a \leq c + b).$$

*Remark* (Interpretation). The same order preservation holds when the common summand is added on the left. This is the left-handed form of the additive compatibility law.

*Remark* (Dependencies).

- [Addition Preserves Order on the Right](#)
- [Addition Is Commutative](#)

### Theorem (Addition Preserves Order)

**Theorem 6.5.8** (Addition Preserves Order). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \leq b \implies a + c \leq b + c \quad \text{and} \quad a \leq b \implies c + a \leq c + b.$$

[Go to proof.](#)

*Remark* (Dependencies).

- [Addition Preserves Order on the Right](#)
- [Addition Preserves Order on the Left](#)

### Compatibility with Multiplication

Multiplication by a natural number is repeated addition by a positive amount. It therefore preserves strict comparisons, and because every multiplier lies at or beyond 1, it also reflects strict comparisons.

### Theorem (Multiplication Preserves and Reflects Strict Order)

**Theorem 6.5.9** (Multiplication Preserves and Reflects Strict Order). *Let  $(P, S, 1)$  be a Peano system. For every  $k, m, n \in P$ ,*

$$m < n \iff k \cdot m < k \cdot n.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall k, m, n \in P \ (m < n \iff k \cdot m < k \cdot n).$

*Remark* (Interpretation). Multiplication by a fixed natural number is an order embedding of the natural numbers into themselves. It shifts strict inequalities forward without collapsing distinct values.

*Remark* (Source alignment). This is the house-style multiplication-order comparison theorem corresponding to Feferman's order laws for products [[Feferman, 1964](#)].

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Multiplication on a Peano System](#)

- [Multiplication Distributes Over Addition](#)
- [Multiplication Is Commutative](#)
- [Addition Preserves Order on the Right](#)
- [Left Cancellation for Addition](#)

## Successor Characterization and Trichotomy

Strict inequality can be recast in terms of stepping one successor beyond the smaller element. This successor reformulation sharpens the order relation and leads to trichotomy, which states that any two elements are linearly comparable.

### Theorem (Strict Order Successor Characterization)

**Theorem 6.5.10** (Strict Order Successor Characterization). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a < b \iff S(a) \leq b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b \in P \ (a < b \iff S(a) \leq b).$$

*Remark* (Interpretation). To say that  $b$  is strictly larger than  $a$  is to say that  $b$  lies at or beyond the first successor step past  $a$ . This theorem converts strict order into a non-strict comparison involving successor.

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Every Element Is Either 1 or a Successor](#)

### Theorem (Trichotomy on a Peano System)

**Theorem 6.5.11** (Trichotomy on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ , exactly one of the following holds:*

$$a < b, \quad a = b, \quad b < a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b \in P \left( (a < b \vee a = b \vee b < a) \wedge \neg(a < b \wedge a = b) \wedge \neg(a < b \wedge b < a) \wedge \neg(a = b \wedge b < a) \right).$$

*Remark* (Interpretation). Any two elements of a Peano system are linearly comparable: one lies below the other, or they coincide. Trichotomy is the theorem that upgrades the additive comparison relation from a partial-order structure to a total order.

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Order Is Reflexive on a Peano System](#)
- [Order Is Transitive on a Peano System](#)
- [Order Is Antisymmetric on a Peano System](#)
- [Strict Order Successor Characterization](#)
- [Induction Principle for a Peano System](#)

### Theorem (Natural-Number Order Is Total)

**Theorem 6.5.12** (Natural-Number Order Is Total). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a \leq b \quad \text{or} \quad b \leq a.$$

*Consequently  $\leq$  is a total order on  $P$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\text{Let } (P, S, 1) \text{ be a Peano system. } \forall a, b \in P (a \leq b \vee b \leq a).$$



*Remark* (Interpretation). Trichotomy says exactly one of  $a < b$ ,  $a = b$ , or  $b < a$  holds. Since the non-strict order allows either strict comparison or equality, trichotomy immediately yields total comparability for  $\leq$ .

*Remark* (Dependencies).

- [Less Than or Equal on a Peano System](#)
- [Trichotomy on a Peano System](#)
- [Order Is Reflexive on a Peano System](#)
- [Order Is Transitive on a Peano System](#)
- [Order Is Antisymmetric on a Peano System](#)

## 6.6 Powers and Growth

### Exponentiation by Recursion

Exponentiation is defined on a Peano system by recursion on the exponent:  $a^1 = a$  and  $a^{S(n)} = a^n \cdot a$ . This makes exponentiation a derived operation built from multiplication in exactly the way multiplication is built from addition. The recursive clauses are immediately isolated as named theorems so that later algebraic arguments can cite them directly.

#### Definition (Exponentiation on a Peano System)

**Definition 6.6.1** (Exponentiation on a Peano System). Let  $(P, S, 1)$  be a Peano system. For each  $a \in P$ , define the function

$$h_a : P \rightarrow P$$

to be the unique function satisfying

$$h_a(1) = a \quad \text{and} \quad \forall n \in P (h_a(S(n)) = h_a(n) \cdot a).$$

For  $a, n \in P$ , define

$$a^n := h_a(n).$$

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system. For each  $a \in P$ , let  $h_a : P \rightarrow P$  satisfy  $h_a(1) = a$  and  $\forall n \in P (h_a(S(n)) = h_a(n) \cdot a)$ . Then for all  $a, n \in P$ ,  $a^n := h_a(n)$ .

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system. For each  $a \in P$ , let  $h_a : P \rightarrow P$  satisfy

$$h_a(1) = a \quad \text{and} \quad \forall n \in P (h_a(S(n)) = h_a(n) \cdot a).$$

Then for all  $a, n, x \in P$ ,  $x \neq a^n \iff x \neq h_a(n)$ .

*Remark* (Interpretation). Exponentiation is repeated multiplication in the same sense that multiplication is repeated addition. The base  $a$  is the fixed left factor; the exponent  $n$  counts how many times  $a$  is multiplied together. The recursive definition propagates rightward:  $a^1 = a$ ,  $a^2 = a^1 \cdot a = a \cdot a$ , and so on.

*Remark* (Dependencies).

- [Peano System](#)
- [General Recursion Theorem for a Peano System](#)
- [Multiplication on a Peano System](#)

#### Theorem (Exponentiation Is Well-Defined on a Peano System)

**Theorem 6.6.2** (Exponentiation Is Well-Defined on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a \in P$ , there exists a unique function*

$$h_a : P \rightarrow P$$

*such that*

$$h_a(1) = a \quad \text{and} \quad \forall n \in P (h_a(S(n)) = h_a(n) \cdot a).$$

*Consequently the rule*

$$(a, n) \longmapsto a^n := h_a(n)$$

*defines a unique binary operation on  $P$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a \in P \exists! h_a : P \rightarrow P \left( h_a(1) = a \wedge \forall n \in P (h_a(S(n)) = h_a(n) \cdot a) \right).$$

*Remark* (Interpretation). The recursive clauses for exponentiation determine exactly one exponent map for each fixed base. Exponentiation is therefore constructed from recursion and multiplication rather than assumed as a primitive operation.

*Remark* (Source alignment). This is the house-style existence-and-uniqueness form of the recursive definition of exponentiation in [\[Feferman, 1964\]](#).

*Remark* (Dependencies).

- [General Recursion Theorem for a Peano System](#)
- [Multiplication on a Peano System](#)

## Exponent Laws

The recursive definition immediately yields the standard exponent laws. These are not assumed; they are proved from the recursive clauses by induction. The three primary laws are: the sum law  $a^{m+n} = a^m \cdot a^n$ , the product law  $(a \cdot b)^n = a^n \cdot b^n$ , and the power law  $(a^m)^n = a^{mn}$ .

### Theorem (Exponent Sum Law)

**Theorem 6.6.3** (Exponent Sum Law). *Let  $(P, S, 1)$  be a Peano system. For all  $a, m, n \in P$ ,*

$$a^{m+n} = a^m \cdot a^n.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  $\forall a, m, n \in P (a^{m+n} = a^m \cdot a^n)$ .

*Remark* (Interpretation). Adding exponents corresponds to concatenating the two blocks of repeated multiplication. The proof is induction on  $n$  using the recursive clause  $a^{S(n)} = a^n \cdot a$  and associativity of multiplication.

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Is Associative](#)
- [Induction Principle for a Peano System](#)

### Theorem (Exponent Product Law)

**Theorem 6.6.4** (Exponent Product Law). *Let  $(P, S, 1)$  be a Peano system. For all  $a, b, n \in P$ ,*

$$(a \cdot b)^n = a^n \cdot b^n.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  $\forall a, b, n \in P ((a \cdot b)^n = a^n \cdot b^n)$ .

*Remark* (Interpretation). Raising a product to a power distributes over the factors. The proof is induction on  $n$ , using commutativity and associativity of multiplication to rearrange the interleaved factors at each successor step.

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)

- [Multiplication Is Associative](#)
- [Multiplication Is Commutative](#)
- [Induction Principle for a Peano System](#)

#### Theorem (Exponent Power Law)

**Theorem 6.6.5** (Exponent Power Law). *Let  $(P, S, 1)$  be a Peano system. For all  $a, m, n \in P$ ,*

$$(a^m)^n = a^{m \cdot n}.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  $\forall a, m, n \in P ((a^m)^n = a^{m \cdot n})$ .

*Remark* (Interpretation). Raising a power to a further power multiplies the exponents. The proof is induction on  $n$  using the exponent sum law and the recursive definition of multiplication.

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Exponent Sum Law](#)
- [Multiplication Successor on the Right](#)
- [Induction Principle for a Peano System](#)

### Order Compatibility of Powers

Exponentiation is order-sensitive in both its base and its exponent. For a fixed positive exponent, taking powers preserves and reflects strict order of bases. For a fixed base beyond 1, increasing the exponent strictly increases the value.

#### Theorem (Powers with Common Exponent Preserve Strict Order)

**Theorem 6.6.6** (Powers with Common Exponent Preserve Strict Order). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, n \in P$ ,*

$$a < b \iff a^n < b^n.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall a, b, n \in P \ (a < b \iff a^n < b^n)$ .

*Remark* (Interpretation). A positive exponent does not collapse the strict order between bases. Since the exponent is a natural number, exponentiation repeatedly multiplies by positive factors and preserves the order information.

*Remark* (Source alignment). This is the house-style common-exponent comparison theorem corresponding to Feferman's natural-number exponent laws [Feferman, 1964].

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Preserves and Reflects Strict Order](#)
- [Induction Principle for a Peano System](#)

#### Theorem (Powers with Common Base Preserve Strict Order)

**Theorem 6.6.7** (Powers with Common Base Preserve Strict Order). *Let  $(P, S, 1)$  be a Peano system. For every  $a, m, n \in P$ ,*

$$a^m < a^n \iff 1 < a \wedge m < n.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.  
 $\forall a, m, n \in P \ (a^m < a^n \iff (1 < a \wedge m < n))$ .

*Remark* (Interpretation). Strict growth in the exponent occurs exactly when the base is beyond the multiplicative identity. If the base is 1, all powers are 1; if the base is larger than 1, each successor step in the exponent multiplies by a factor that strictly increases the value.

*Remark* (Source alignment). This is the house-style common-base comparison theorem corresponding to Feferman's exponent-order theorem [Feferman, 1964].

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Preserves and Reflects Strict Order](#)
- [Exponent Sum Law](#)
- [Induction Principle for a Peano System](#)

## Exponential Growth: $n < 2^n$

The most important growth comparison on  $\mathbb{N}$  is that exponential growth outpaces linear growth:  $n < 2^n$  for all  $n \in P$ . This single inequality captures why exponential towers dwarf polynomial bounds and is the prototypical example of an induction argument whose inductive step uses both the hypothesis and a multiplicative amplification.

**Theorem** ( $n < 2^n$  for All  $n \in \mathbb{N}$ )

**Theorem 6.6.8** ( $n < 2^n$  for All  $n \in \mathbb{N}$ ). *Let  $(P, S, 1)$  be a Peano system, and let  $2 := S(1)$ . For every  $n \in P$ ,*

$$n < 2^n.$$

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system and let  $2 := S(1)$ .  
 $\forall n \in P (n < 2^n)$ .

*Remark* (Interpretation). The base case  $1 < 2 = 2^1$  holds by definition of 2 and the strict order. For the inductive step, assume  $n < 2^n$ . Then  $2^{S(n)} = 2^n \cdot 2$ . Since  $2^n > n \geq 1$ , we have  $2^n \cdot 2 > n \cdot 2 = n + n > n + 1 = S(n)$ , where the last step uses that  $n \geq 1$  so  $n + n > n + 1$ . Hence  $S(n) < 2^{S(n)}$ . The inequality  $n < 2^n$  is the threshold that distinguishes the growth regimes and reappears in every comparison between polynomial and exponential bounds.

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Strict Less Than on a Peano System](#)
- [Multiplication Distributes Over Addition](#)
- [Addition Preserves Order on the Right](#)
- [Induction Principle for a Peano System](#)

## 6.7 Well Ordering Principle

### The Well-Ordering Principle

The well-ordering principle asserts that every nonempty subset of  $\mathbb{N}$  contains a least element. This is not merely a consequence of the order axioms; it is a property that distinguishes  $\mathbb{N}$  from dense ordered sets such as  $\mathbb{Q}$ , which have no smallest positive element. The well-ordering principle is equivalent to ordinary induction over  $\mathbb{N}$ , but it supports a distinct proof style: to establish a universal statement, assume it fails for some element and derive a

contradiction by examining the least failure.

### Theorem (Well-Ordering Principle)

**Theorem 6.7.1** (Well-Ordering Principle). *Let  $(P, S, 1)$  be a Peano system. Every nonempty subset of  $P$  has a least element with respect to the order  $\leq$  on  $P$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall A \subseteq P \left( A \neq \emptyset \implies \exists m \in A \forall a \in A (m \leq a) \right).$$

*Remark* (Predicate reading).

$\text{WellOrderingPrinciple} := \forall A \left( (A \subseteq P \wedge \exists a (a \in A)) \Rightarrow \exists m (m \in A \wedge \forall a (a \in A \Rightarrow m \leq a)) \right).$  ■

*Remark* (Negated quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\exists A \subseteq P \left( A \neq \emptyset \wedge \forall m \in A \exists a \in A (a < m) \right).$$

*Remark* (Failure modes). The well-ordering principle would fail if some nonempty subset of  $P$  had no least element. That means for every candidate minimum, a strictly smaller element of the subset exists, producing an infinite strictly descending chain in  $P$ . Induction rules this out.

*Remark* (Failure mode decomposition).

$$\neg \text{WellOrderingPrinciple} \iff \exists A \subseteq P \left( \underbrace{A \neq \emptyset}_{\text{nonempty}} \wedge \underbrace{\forall m \in A \exists a \in A (a < m)}_{\text{no least element}} \right).$$

*Remark* (Interpretation). The well-ordering principle is the order-theoretic complement to induction. Induction says one can build up: start at 1 and keep stepping forward. Well-ordering says one can always descend to a minimum: any nonempty set has a floor. Proof strategies that use well-ordering typically assume a property fails somewhere, form the set of failures, extract its minimum by well-ordering, and derive a contradiction from the minimality of that element.

*Remark* (Dependencies).

- [Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Induction Principle for a Peano System](#)
- [Trichotomy on a Peano System](#)

## Equivalence of Induction, Strong Induction, and Well-Ordering

Ordinary induction, strong induction, and the well-ordering principle are logically equivalent over a Peano system. Each can be used as the primitive proof engine and the others recovered as theorems. This equivalence matters structurally: it means that induction proofs, strong-induction proofs, and least-counterexample proofs are different presentations of the same foundational mechanism.

### Theorem (Induction, Strong Induction, and Well-Ordering Are Equivalent)

**Theorem 6.7.2** (Induction, Strong Induction, and Well-Ordering Are Equivalent). *Let  $(P, S, 1)$  be a Peano system equipped with the order  $\leq$ . The following are equivalent:*

- 1. The induction principle: for every predicate  $\varphi$  on  $P$ , if  $\varphi(1)$  holds and the successor step holds, then  $\varphi(n)$  holds for all  $n \in P$ .*
- 2. The strong induction principle: for every predicate  $\varphi$  on  $P$ , if  $\varphi(k)$  holds for every  $k < n$  implies  $\varphi(n)$ , then  $\varphi(n)$  holds for every  $n \in P$ .*
- 3. The well-ordering principle: every nonempty subset of  $P$  has a least element with respect to  $\leq$ .*

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\text{WellOrderingPrinciple} \iff \text{StrongInduction} \iff \left( \forall \varphi \left( \varphi(1) \wedge \forall n \in P \left( \varphi(n) \implies \varphi(S(n)) \right) \right) \implies \forall n \in P \varphi(n) \right)$$

*Remark* (Interpretation). Induction proves well-ordering by contrapositive: if a nonempty set  $A$  had no least element, the set of non-members of  $A$  would be inductive, forcing  $A = \emptyset$ , a contradiction. Well-ordering proves induction by forming the set of elements where  $\varphi$  fails and, if nonempty, extracting a least failure to derive a contradiction with the successor step.

*Remark* (Dependencies).

- [Induction Principle for a Peano System](#)
- [Strong Induction on a Peano System](#)
- [Well-Ordering Principle](#)



## 6.8 Induction

### Induction from an Arbitrary Base

Ordinary and strong induction each begin at the distinguished first element 1. Many arguments require starting at a later element. Induction from an arbitrary base  $m$  restricts the claim to elements at or above  $m$  and starts the inductive process there.

#### Theorem (Induction from an Arbitrary Base)

**Theorem 6.8.1** (Induction from an Arbitrary Base). *Let  $(P, S, 1)$  be a Peano system, let  $m \in P$ , and let  $\varphi$  be a predicate on  $P$ . If*

$$\varphi(m) \quad \text{and} \quad \forall n \in P \left( n \geq m \wedge \varphi(n) \implies \varphi(S(n)) \right),$$

*then*

$$\forall n \in P \left( n \geq m \implies \varphi(n) \right).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system,  $m \in P$ , and  $\varphi$  a predicate on  $P$ .

$$\left( \varphi(m) \wedge \forall n \in P \left( n \geq m \wedge \varphi(n) \implies \varphi(S(n)) \right) \right) \implies \forall n \in P \left( n \geq m \implies \varphi(n) \right).$$

*Remark* (Interpretation). Apply ordinary induction to the shifted predicate  $\psi(n) := (n \geq m \implies \varphi(n))$ , which holds vacuously below  $m$  and reduces to  $\varphi$  above  $m$ . This theorem is the formal license for all arguments of the form “for all  $n \geq m$ , proceed by induction.”

*Remark* (Dependencies).

- [Induction Principle for a Peano System](#)
- [Less Than or Equal on a Peano System](#)

## 6.9 Algebraic Structure of $\mathbb{N}$

### $\mathbb{N}$ as a Commutative Monoid Under Addition

The arithmetic established in the preceding sections can now be packaged into a named algebraic structure. Under addition,  $\mathbb{N}$  is a commutative monoid with identity 0 — or, in the Peano-system setting that uses 1 as the distinguished element with no additive identity available internally, the correct statement is that  $(\mathbb{N}, +)$  satisfies associativity and commutativity. This topic states the structural conclusion that follows from the individual arithmetic theorems proved above.

### Theorem ( $\mathbb{N}$ Is Closed, Associative, and Commutative Under Addition)

**Theorem 6.9.1** ( $\mathbb{N}$  Is Closed, Associative, and Commutative Under Addition). *Let  $(P, S, 1)$  be a Peano system. The binary operation  $+$  on  $P$  satisfies:*

1. **Closure.** *For all  $a, b \in P$ ,  $a + b \in P$ .*
2. **Associativity.** *For all  $a, b, c \in P$ ,  $(a + b) + c = a + (b + c)$ .*
3. **Commutativity.** *For all  $a, b \in P$ ,  $a + b = b + a$ .*
4. **Left cancellation.** *For all  $a, b, c \in P$ , if  $a + b = a + c$  then  $b = c$ .*
5. **Right cancellation.** *For all  $a, b, c \in P$ , if  $b + a = c + a$  then  $b = c$ .*

*Go to proof.*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P : (a + b) + c = a + (b + c) \wedge a + b = b + a \wedge (a + b = a + c \implies b = c).$$

*Remark* (Interpretation). This theorem is a packaging result. It names the algebraic structure that has been proved piece by piece: the three individual theorems on associativity, commutativity, and cancellation together certify that  $(P, +)$  is a cancellative commutative semigroup. There is no additive identity in  $P$  because  $P$  has no element  $0$  with  $0 + n = n$  for all  $n$ ; the smallest element under addition is  $1$ , not an identity.

*Remark* (Dependencies).

- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Left Cancellation for Addition](#)
- [Right Cancellation for Addition](#)

### $\mathbb{N}$ as a Commutative Monoid Under Multiplication

Multiplication on  $P$  has a two-sided multiplicative identity, namely  $1$ , the distinguished first element. Under multiplication,  $P$  is therefore a commutative monoid. This topic states the structural packaging for multiplication that parallels the additive packaging above.

### Theorem ( $\mathbb{N}$ Is a Commutative Monoid Under Multiplication)

**Theorem 6.9.2** ( $\mathbb{N}$  Is a Commutative Monoid Under Multiplication). *Let  $(P, S, 1)$  be a Peano system. The binary operation  $\cdot$  on  $P$  satisfies:*

1. **Closure.** *For all  $a, b \in P$ ,  $a \cdot b \in P$ .*
2. **Associativity.** *For all  $a, b, c \in P$ ,  $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ .*
3. **Commutativity.** *For all  $a, b \in P$ ,  $a \cdot b = b \cdot a$ .*
4. **Identity.** *For all  $a \in P$ ,  $1 \cdot a = a = a \cdot 1$ .*

*Consequently  $(P, \cdot, 1)$  is a commutative monoid. [Go to proof.](#)*

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$$\forall a, b, c \in P : (a \cdot b) \cdot c = a \cdot (b \cdot c) \wedge a \cdot b = b \cdot a \wedge 1 \cdot a = a.$$

*Remark* (Interpretation). The distinguished first element 1 is a multiplicative identity on both sides, so multiplication promotes  $P$  from a semigroup to a monoid. The absence of multiplicative inverses (other than  $1^{-1} = 1$ , which does not lie in  $P$  in the usual sense) is what prevents  $(P, \cdot)$  from being a group. The structural limitation is intrinsic: no element of  $P$  other than 1 has a reciprocal in  $P$ .

*Remark* (Dependencies).

- [Multiplication Is Associative](#)
- [Multiplication Is Commutative](#)
- [Multiplication With 1](#)
- [One Times  \$n\$](#)

### $\mathbb{N}$ as a Semiring and the Absence of Subtraction

Together, addition and multiplication on  $P$  form a semiring: addition is a commutative semigroup, multiplication is a commutative monoid, and multiplication distributes over addition on both sides. The critical structural limitation is that subtraction is not a total operation on  $P$ . Since  $P$  has no additive identity and no additive inverses, the equation  $a + x = b$  has a solution in  $P$  only when  $b > a$ ; it has no solution when  $b \leq a$ . This failure of internal subtraction is the reason  $\mathbb{Z}$  is introduced immediately after  $\mathbb{N}$ .

### Theorem ( $\mathbb{N}$ Is a Semiring)

**Theorem 6.9.3** ( $\mathbb{N}$  Is a Semiring). *Let  $(P, S, 1)$  be a Peano system. Then  $(P, +, \cdot)$  satisfies:*

1.  $(P, +)$  is a commutative semigroup.
2.  $(P, \cdot, 1)$  is a commutative monoid.
3. Multiplication distributes over addition: for all  $a, b, c \in P$ ,

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

Let  $(P, S, 1)$  be a Peano system.

$\forall a, b, c \in P : a \cdot (b + c) = (a \cdot b) + (a \cdot c) \wedge (a + b) + c = a + (b + c) \wedge a + b = b + a \wedge 1 \cdot a = a.$

*Remark* (Interpretation). A semiring packages the interaction of two operations without requiring additive inverses. The natural numbers are the canonical example of a semiring that is not a ring: they satisfy every ring axiom except the existence of additive inverses, which fails because there is no element  $-1$  in  $P$ . The integers are the smallest ring extending  $P$  that repairs this failure.

*Remark* (Dependencies).

- [ℕ Is Closed, Associative, and Commutative Under Addition](#)
- [ℕ Is a Commutative Monoid Under Multiplication](#)
- [Multiplication Distributes Over Addition](#)

## Proofs

*Remark* (Return). [Return to Theorem](#)

*Remark* (Proof status).

*Remark* (Proof vault). [Handwritten proof record](#)

**Theorem** (Addition Is Well-Defined on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ , there exists a unique function*

$$f_m : P \rightarrow P$$

*such that*

$$f_m(1) = S(m) \quad \text{and} \quad \forall n \in P (f_m(S(n)) = S(f_m(n))).$$

*Consequently the rule*

$$(m, n) \longmapsto m + n := f_m(n)$$

*defines a unique binary operation on  $P$ .*

*Professional Standard Proof.*

TODO: professional standard proof for addition-well-defined-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-well-defined-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Closure Under Successor](#)
- [Iterator Data on a Peano System](#)
- [Addition on a Peano System](#)
- [Peano Iterator Theorem](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition With 1). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ ,*

$$m + 1 = S(m).$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-with-one. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-with-one. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Successor on the Right). *Let  $(P, S, 1)$  be a Peano system. For every  $m, n \in P$ ,*

$$m + S(n) = S(m + n).$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-successor-on-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-successor-on-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (One Plus  $n$ ). *Let  $(P, S, 1)$  be a Peano system. For every  $n \in P$ ,*

$$1 + n = S(n).$$

*Professional Standard Proof.*

TODO: professional standard proof for one-plus-n. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-plus-n. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [Induction Principle for a Peano System](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Is Associative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a + b) + c = a + (b + c).$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-associative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-associative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Is Commutative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a + b = b + a.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-commutative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-commutative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [One Plus  \$n\$](#)
- [Addition Is Associative](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Cancellation for Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a + b = a + c \implies b = c.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-left-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-left-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Injectivity of Successor](#)
- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [One Plus  \$n\$](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Right Cancellation for Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$b + a = c + a \implies b = c.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-right-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-right-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition Is Commutative](#)
- [Left Cancellation for Addition](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Cancellation for Addition). *Let  $(P, S, 1)$  be a Peano system. Addition on  $P$  satisfies cancellation on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for addition-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Cancellation for Addition](#)
- [Right Cancellation for Addition](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (No Natural Number Equals Itself Plus a Natural Number). *Let  $(P, S, 1)$  be a Peano system. For every  $m, n \in P$ ,*

$$n \neq m + n.$$

*Professional Standard Proof.*

TODO: professional standard proof for no-natural-number-equals-itself-plus-a-natural-number. □

*Detailed Learning Proof.*

TODO: detailed learning proof for no-natural-number-equals-itself-plus-a-natural-number. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [No Object Is Its Own Successor](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Is Well-Defined on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ , there exists a unique function*

$$g_m : P \rightarrow P$$

*such that*

$$g_m(1) = m \quad \text{and} \quad \forall n \in P (g_m(S(n)) = g_m(n) + m).$$

*Consequently the rule*

$$(m, n) \longmapsto m \cdot n := g_m(n)$$

*defines a unique binary operation on  $P$ .*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-well-defined-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-well-defined-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on a Peano System](#)
- [General Recursion Theorem for a Peano System](#)
- [Addition on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication With 1). *Let  $(P, S, 1)$  be a Peano system. For every  $m \in P$ ,*

$$m \cdot 1 = m.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-with-one. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-with-one. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on a Peano System](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Successor on the Right). *Let  $(P, S, 1)$  be a Peano system. For every  $m, n \in P$ ,*

$$m \cdot S(n) = (m \cdot n) + m.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-successor-on-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-successor-on-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on a Peano System](#)
- [Addition on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (One Times  $n$ ). *Let  $(P, S, 1)$  be a Peano system. For every  $n \in P$ ,*

$$1 \cdot n = n.$$

*Professional Standard Proof.*

TODO: professional standard proof for one-times- $n$ . □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-times- $n$ . □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication With 1](#)
- [Multiplication Successor on the Right](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Distributes Over Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-distributes-over-addition. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-distributes-over-addition. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication Successor on the Right](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Distributivity of Multiplication Over Addition). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a + b) \cdot c = (a \cdot c) + (b \cdot c).$$

*Professional Standard Proof.*

TODO: professional standard proof for left-distributivity-of-multiplication-over-addition. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-distributivity-of-multiplication-over-addition. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication Distributes Over Addition](#)
- [Multiplication Is Commutative](#)
- [Addition Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Distributes over Addition on Both Sides). *Let  $(P, S, 1)$  be a Peano system. Multiplication distributes over addition on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-distributes-over-addition-both-sides. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-distributes-over-addition-both-sides. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication Distributes Over Addition](#)
- [Left Distributivity of Multiplication Over Addition](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Is Associative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a \cdot b) \cdot c = a \cdot (b \cdot c).$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-associative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-associative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication Distributes Over Addition](#)
- [Multiplication With 1](#)
- [Multiplication Successor on the Right](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Is Commutative). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a \cdot b = b \cdot a.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-commutative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-commutative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [One Times  \$n\$](#)
- [Multiplication Successor on the Right](#)
- [Multiplication Distributes Over Addition](#)
- [Addition Is Commutative](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Cancellation for Multiplication). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \cdot b = a \cdot c \implies b = c.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-left-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-left-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication Preserves and Reflects Strict Order](#)
- [Trichotomy on a Peano System](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Right Cancellation for Multiplication). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$b \cdot a = c \cdot a \implies b = c.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-right-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-right-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Cancellation for Multiplication](#)
- [Multiplication Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Cancellation for Multiplication). *Let  $(P, S, 1)$  be a Peano system. Multiplication on  $P$  satisfies cancellation on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Left Cancellation for Multiplication](#)
- [Right Cancellation for Multiplication](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order Is Reflexive on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a \in P$ ,*

$$a \leq a.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-reflexive-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-reflexive-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Less Than or Equal on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order Is Transitive on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$(a \leq b \wedge b \leq c) \implies a \leq c.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-transitive-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-transitive-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Addition Is Associative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order Is Antisymmetric on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$(a \leq b \wedge b \leq a) \implies a = b.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-antisymmetric-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-antisymmetric-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Left Cancellation for Addition](#)
- [Addition Is Associative](#)
- [No Object Is Its Own Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Preserves Order on the Right). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \leq b \implies a + c \leq b + c.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-preserves-order-on-right. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-preserves-order-on-right. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Addition Is Associative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Preserves Order on the Left). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, c \in P$ ,*

$$a \leq b \implies c + a \leq c + b.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-preserves-order-on-left. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-preserves-order-on-left. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition Preserves Order on the Right](#)
- [Addition Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Preserves Order). *Let  $(P, S, 1)$  be a Peano system. Addition preserves order on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for addition-preserves-order. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-preserves-order. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition Preserves Order on the Right](#)
- [Addition Preserves Order on the Left](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Preserves and Reflects Strict Order). *Let  $(P, S, 1)$  be a Peano system. For every  $k, m, n \in P$ ,*

$$m < n \iff k \cdot m < k \cdot n.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-preserves-and-reflects-strict-order.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-preserves-and-reflects-strict-order.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Multiplication on a Peano System](#)
- [Multiplication Distributes Over Addition](#)
- [Multiplication Is Commutative](#)
- [Addition Preserves Order on the Right](#)
- [Left Cancellation for Addition](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Strict Order Successor Characterization). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a < b \iff S(a) \leq b.$$

*Professional Standard Proof.*

TODO: professional standard proof for strict-order-successor-characterization. □

*Detailed Learning Proof.*

TODO: detailed learning proof for strict-order-successor-characterization. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Addition With 1](#)
- [Addition Successor on the Right](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Every Element Is Either 1 or a Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Trichotomy on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ , exactly one of the following holds:*

$$a < b, \quad a = b, \quad b < a.$$

*Professional Standard Proof.*

TODO: professional standard proof for trichotomy-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for trichotomy-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Strict Less Than on a Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Order Is Reflexive on a Peano System](#)
- [Order Is Transitive on a Peano System](#)
- [Order Is Antisymmetric on a Peano System](#)
- [Strict Order Successor Characterization](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Natural-Number Order Is Total). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b \in P$ ,*

$$a \leq b \quad \text{or} \quad b \leq a.$$

*Consequently  $\leq$  is a total order on  $P$ .*

*Professional Standard Proof.*

TODO: professional standard proof for natural-number-order-is-total. □

*Detailed Learning Proof.*

TODO: detailed learning proof for natural-number-order-is-total. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Less Than or Equal on a Peano System](#)
- [Trichotomy on a Peano System](#)
- [Order Is Reflexive on a Peano System](#)
- [Order Is Transitive on a Peano System](#)
- [Order Is Antisymmetric on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Exponentiation Is Well-Defined on a Peano System). *Let  $(P, S, 1)$  be a Peano system. For every  $a \in P$ , there exists a unique function*

$$h_a : P \rightarrow P$$

*such that*

$$h_a(1) = a \quad \text{and} \quad \forall n \in P \ (h_a(S(n)) = h_a(n) \cdot a).$$

*Consequently the rule*

$$(a, n) \longmapsto a^n := h_a(n)$$

*defines a unique binary operation on  $P$ .*

*Professional Standard Proof.*

TODO: professional standard proof for exponentiation-well-defined-on-peano-system. □

*Detailed Learning Proof.*

TODO: detailed learning proof for exponentiation-well-defined-on-peano-system. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [General Recursion Theorem for a Peano System](#)
- [Multiplication on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Exponent Sum Law). *Let  $(P, S, 1)$  be a Peano system. For all  $a, m, n \in P$ ,*

$$a^{m+n} = a^m \cdot a^n.$$

*Professional Standard Proof.*

TODO: professional standard proof for exponent-sum-law. □

*Detailed Learning Proof.*

TODO: detailed learning proof for exponent-sum-law. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Is Associative](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Exponent Product Law). *Let  $(P, S, 1)$  be a Peano system. For all  $a, b, n \in P$ ,*

$$(a \cdot b)^n = a^n \cdot b^n.$$

*Professional Standard Proof.*

TODO: professional standard proof for exponent-product-law. □

*Detailed Learning Proof.*

TODO: detailed learning proof for exponent-product-law. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Is Associative](#)
- [Multiplication Is Commutative](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Exponent Power Law). *Let  $(P, S, 1)$  be a Peano system. For all  $a, m, n \in P$ ,*

$$(a^m)^n = a^{m \cdot n}.$$

*Professional Standard Proof.*

TODO: professional standard proof for exponent-power-law. □

*Detailed Learning Proof.*

TODO: detailed learning proof for exponent-power-law. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Exponent Sum Law](#)
- [Multiplication Successor on the Right](#)
- [Induction Principle for a Peano System](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Powers with Common Exponent Preserve Strict Order). *Let  $(P, S, 1)$  be a Peano system. For every  $a, b, n \in P$ ,*

$$a < b \iff a^n < b^n.$$

*Professional Standard Proof.*

TODO: professional standard proof for powers-with-common-exponent-preserve-strict-order. □

*Detailed Learning Proof.*

TODO: detailed learning proof for powers-with-common-exponent-preserve-strict-order. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Preserves and Reflects Strict Order](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Powers with Common Base Preserve Strict Order). *Let  $(P, S, 1)$  be a Peano system. For every  $a, m, n \in P$ ,*

$$a^m < a^n \iff 1 < a \wedge m < n.$$

*Professional Standard Proof.*

TODO: professional standard proof for powers-with-common-base-preserve-strict-order. □

*Detailed Learning Proof.*

TODO: detailed learning proof for powers-with-common-base-preserve-strict-order. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Multiplication Preserves and Reflects Strict Order](#)
- [Exponent Sum Law](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $n < 2^n$  for All  $n \in \mathbb{N}$ ). *Let  $(P, S, 1)$  be a Peano system, and let  $2 := S(1)$ . For every  $n \in P$ ,*

$$n < 2^n.$$

*Professional Standard Proof.*

TODO: professional standard proof for n-less-than-2-to-n. □

*Detailed Learning Proof.*

TODO: detailed learning proof for n-less-than-2-to-n. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Exponentiation on a Peano System](#)
- [Strict Less Than on a Peano System](#)
- [Multiplication Distributes Over Addition](#)
- [Addition Preserves Order on the Right](#)
- [Induction Principle for a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Well-Ordering Principle). *Let  $(P, S, 1)$  be a Peano system. Every nonempty subset of  $P$  has a least element with respect to the order  $\leq$  on  $P$ .*

*Professional Standard Proof.*

TODO: professional standard proof for well-ordering-principle. □

*Detailed Learning Proof.*

TODO: detailed learning proof for well-ordering-principle. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Peano System](#)
- [Less Than or Equal on a Peano System](#)
- [Induction Principle for a Peano System](#)
- [Trichotomy on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induction, Strong Induction, and Well-Ordering Are Equivalent). *Let  $(P, S, 1)$  be a Peano system equipped with the order  $\leq$ . The following are equivalent:*

1. *The induction principle: for every predicate  $\varphi$  on  $P$ , if  $\varphi(1)$  holds and the successor step holds, then  $\varphi(n)$  holds for all  $n \in P$ .*
2. *The strong induction principle: for every predicate  $\varphi$  on  $P$ , if  $\varphi(k)$  holds for every  $k < n$  implies  $\varphi(n)$ , then  $\varphi(n)$  holds for every  $n \in P$ .*
3. *The well-ordering principle: every nonempty subset of  $P$  has a least element with respect to  $\leq$ .*

*Professional Standard Proof.*

TODO: professional standard proof for induction-well-ordering-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induction-well-ordering-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Induction Principle for a Peano System](#)
- [Strong Induction on a Peano System](#)
- [Well-Ordering Principle](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Induction from an Arbitrary Base). *Let  $(P, S, 1)$  be a Peano system, let  $m \in P$ , and let  $\varphi$  be a predicate on  $P$ . If*

$$\varphi(m) \quad \text{and} \quad \forall n \in P \left( n \geq m \wedge \varphi(n) \implies \varphi(S(n)) \right),$$

*then*

$$\forall n \in P \left( n \geq m \implies \varphi(n) \right).$$

*Professional Standard Proof.*

TODO: professional standard proof for induction-from-arbitrary-base. □

*Detailed Learning Proof.*

TODO: detailed learning proof for induction-from-arbitrary-base. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Induction Principle for a Peano System](#)
- [Less Than or Equal on a Peano System](#)

Remark (Return). [Return to Theorem](#)

**Theorem** (*N Is Closed, Associative, and Commutative Under Addition*). Let  $(P, S, 1)$  be a Peano system. The binary operation  $+$  on  $P$  satisfies:

1. **Closure.** For all  $a, b \in P$ ,  $a + b \in P$ .
2. **Associativity.** For all  $a, b, c \in P$ ,  $(a + b) + c = a + (b + c)$ .
3. **Commutativity.** For all  $a, b \in P$ ,  $a + b = b + a$ .
4. **Left cancellation.** For all  $a, b, c \in P$ , if  $a + b = a + c$  then  $b = c$ .
5. **Right cancellation.** For all  $a, b, c \in P$ , if  $b + a = c + a$  then  $b = c$ .

*Professional Standard Proof.*

TODO: professional standard proof for n-additive-structure. □

*Detailed Learning Proof.*

TODO: detailed learning proof for n-additive-structure. □

Remark (Proof structure).

Remark (Dependencies).

- [Addition Is Associative](#)
- [Addition Is Commutative](#)
- [Left Cancellation for Addition](#)
- [Right Cancellation for Addition](#)

Remark (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{N}$  Is a Commutative Monoid Under Multiplication). *Let  $(P, S, 1)$  be a Peano system. The binary operation  $\cdot$  on  $P$  satisfies:*

1. **Closure.** *For all  $a, b \in P$ ,  $a \cdot b \in P$ .*
2. **Associativity.** *For all  $a, b, c \in P$ ,  $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ .*
3. **Commutativity.** *For all  $a, b \in P$ ,  $a \cdot b = b \cdot a$ .*
4. **Identity.** *For all  $a \in P$ ,  $1 \cdot a = a = a \cdot 1$ .*

Consequently  $(P, \cdot, 1)$  is a commutative monoid.

*Professional Standard Proof.*

TODO: professional standard proof for n-multiplicative-monoid. □

*Detailed Learning Proof.*

TODO: detailed learning proof for n-multiplicative-monoid. □

Remark (Proof structure).

Remark (Dependencies).

- [Multiplication Is Associative](#)
- [Multiplication Is Commutative](#)
- [Multiplication With 1](#)
- [One Times  \$n\$](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{N}$  Is a Semiring). *Let  $(P, S, 1)$  be a Peano system. Then  $(P, +, \cdot)$  satisfies:*

1.  $(P, +)$  is a commutative semigroup.
2.  $(P, \cdot, 1)$  is a commutative monoid.
3. Multiplication distributes over addition: for all  $a, b, c \in P$ ,

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c).$$

*Professional Standard Proof.*

TODO: professional standard proof for  $\mathbb{N}$ -is-semiring. □

*Detailed Learning Proof.*

TODO: detailed learning proof for  $\mathbb{N}$ -is-semiring. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [ℕ Is Closed, Associative, and Commutative Under Addition](#)
- [ℕ Is a Commutative Monoid Under Multiplication](#)
- [Multiplication Distributes Over Addition](#)

# Capstone

# Chapter 7

## Constructing the Whole Numbers



### 7.1 Why Whole Numbers Are Introduced

#### Adjoining an Additive Identity

The natural-number system developed in the preceding chapter is one-based: its distinguished first element is 1. This is sufficient for counting and recursive arithmetic, but addition on  $\mathbb{N}$  has no additive identity inside  $\mathbb{N}$ . The whole numbers are introduced by adjoining a new element before constructing the integers.

#### The Missing Identity

*Remark* (Exposition). In the one-based system, every element of  $\mathbb{N}$  is positive. The operation  $+$  is defined and behaves associatively and commutatively, but there is no element  $e \in \mathbb{N}$  satisfying  $e + n = n$  for every  $n \in \mathbb{N}$ . The element needed for that identity is not another successor-stage element; it is a new element placed before 1.

*Remark* (Interpretation). The passage from  $\mathbb{N}$  to  $\mathbb{W}$  is therefore not a change in the positive arithmetic already constructed. It is an extension that adds the missing additive identity and leaves the positive part intact.

*Remark* (Dependencies).

- [Addition on a Peano System](#)
- [Addition Is Associative](#)
- [Addition Is Commutative](#)

*Remark* (Exposition). Once 0 is available, addition can be packaged as a genuine commutative monoid. That structure is the natural input for the integer construction, where ordered pairs are interpreted as formal differences and subtraction is made available by a quotient construction.

## 7.2 Definition of the Whole Numbers

### The Positive Part with a New Zero

The whole numbers are obtained by adjoining a single new element to the one-based natural numbers. The notation records two facts at once: the positive natural numbers remain present by inclusion, and the new element 0 is not one of those positive natural numbers.

### The Underlying Set

#### Definition (Whole Numbers)

**Definition 7.2.1** (Whole Numbers). Let  $\mathbb{N}$  be the one-based natural-number system developed in the preceding chapter. Let 0 be a new element with  $0 \notin \mathbb{N}$ . Define

$$\mathbb{W} := \mathbb{N} \cup \{0\}.$$

*Remark* (Standard quantified statement).

$$\mathbb{W} = \mathbb{N} \cup \{0\} \quad \text{and} \quad 0 \notin \mathbb{N}.$$

*Remark* (Interpretation). Elements of  $\mathbb{W}$  are either the new element 0 or positive natural numbers. The natural numbers embed into  $\mathbb{W}$  by the inclusion map  $n \mapsto n$ . The element 0 is not a successor-stage element of the original one-based Peano system.

*Remark* (Dependencies).

- [Peano System](#)

#### Theorem ( $0 \notin \mathbb{N}$ )

**Theorem 7.2.2** ( $0 \notin \mathbb{N}$ ). *The adjoined element 0 is not an element of the one-based natural-number system:*

$$0 \notin \mathbb{N}.$$

*Go to proof.*

*Remark* (Interpretation). This theorem records the disjointness condition built into the construction of  $\mathbb{W}$ . The element 0 is new; it is not a renamed natural number.

*Remark* (Dependencies).

- [Whole Numbers](#)

## 7.3 Extending Successor

### Successor After Adjoining Zero

The new successor map on  $\mathbb{W}$  agrees with the old successor on the positive part and sends the new initial element 0 to 1. This makes  $\mathbb{W}$  a zero-based successor chain while preserving the one-based successor behavior already proved on  $\mathbb{N}$ .

### The Extended Successor

#### Definition (Successor on $\mathbb{W}$ )

**Definition 7.3.1** (Successor on  $\mathbb{W}$ ). Define  $S_{\mathbb{W}} : \mathbb{W} \rightarrow \mathbb{W}$  by

$$S_{\mathbb{W}}(0) = 1 \quad \text{and} \quad S_{\mathbb{W}}(n) = S_{\mathbb{N}}(n) \quad (n \in \mathbb{N}).$$

*Remark* (Standard quantified statement).

$$\begin{aligned} S_{\mathbb{W}} &: \mathbb{W} \rightarrow \mathbb{W}, \\ S_{\mathbb{W}}(0) &= 1, \\ \forall n \in \mathbb{N} \quad (S_{\mathbb{W}}(n) &= S_{\mathbb{N}}(n)). \end{aligned}$$

*Remark* (Interpretation). The definition inserts 0 immediately before the positive chain. After the first step, successor is exactly the successor already defined on  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Peano System](#)

#### Theorem (Successor on $\mathbb{W}$ Is Well-Defined)

**Theorem 7.3.2** (Successor on  $\mathbb{W}$  Is Well-Defined). *The rule defining  $S_{\mathbb{W}}$  determines a function  $S_{\mathbb{W}} : \mathbb{W} \rightarrow \mathbb{W}$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$S_{\mathbb{W}}$  is a well-defined function from  $\mathbb{W}$  to  $\mathbb{W}$ .

*Remark* (Interpretation). The cases are exclusive because  $0 \notin \mathbb{N}$ , and the positive case lands in  $\mathbb{N} \subseteq \mathbb{W}$ .

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Successor on  \$\mathbb{W}\$](#)

### Theorem (0 Is Not a Successor in $\mathbb{W}$ )

**Theorem 7.3.3** (0 Is Not a Successor in  $\mathbb{W}$ ). *There is no  $a \in \mathbb{W}$  such that  $S_{\mathbb{W}}(a) = 0$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\neg \exists a \in \mathbb{W} (S_{\mathbb{W}}(a) = 0).$$

*Remark* (Interpretation). The new element 0 is the first element of the extended chain. It is placed before 1, not obtained by applying successor to some earlier element.

*Remark* (Dependencies).

- [Successor on  \$\mathbb{W}\$](#)
- [1 Is Not a Successor](#)

### Theorem (Every Nonzero Element of $\mathbb{W}$ Is a Successor)

**Theorem 7.3.4** (Every Nonzero Element of  $\mathbb{W}$  Is a Successor). *If  $a \in \mathbb{W}$  and  $a \neq 0$ , then there exists  $b \in \mathbb{W}$  such that  $a = S_{\mathbb{W}}(b)$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\forall a \in \mathbb{W} (a \neq 0 \implies \exists b \in \mathbb{W} (a = S_{\mathbb{W}}(b))).$$

*Remark* (Interpretation). The element 1 is the successor of 0, and every positive natural number beyond 1 remains a successor in the original Peano chain.

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Successor on  \$\mathbb{W}\$](#)
- [Induction Axiom](#)

### Theorem (Successor on $\mathbb{W}$ Is Injective)

**Theorem 7.3.5** (Successor on  $\mathbb{W}$  Is Injective). *For all  $a, b \in \mathbb{W}$ ,*

$$S_{\mathbb{W}}(a) = S_{\mathbb{W}}(b) \implies a = b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} (S_{\mathbb{W}}(a) = S_{\mathbb{W}}(b) \implies a = b).$$

*Remark* (Interpretation). The extended successor still never merges two inputs. The only new case is the comparison between 0 and a positive natural number, and that case is ruled out by the original non-return condition for the first element.

*Remark* (Dependencies).

- [Successor on  \$\mathbb{W}\$](#)
- [Injectivity of Successor](#)
- [1 Is Not a Successor](#)

#### Theorem $((\mathbb{W}, 0, S_{\mathbb{W}})$ Satisfies the Successor Properties)

**Theorem 7.3.6**  $((\mathbb{W}, 0, S_{\mathbb{W}})$  Satisfies the Successor Properties). *The triple  $(\mathbb{W}, 0, S_{\mathbb{W}})$  satisfies the successor-side properties:*

$$0 \in \mathbb{W}, \quad S_{\mathbb{W}} : \mathbb{W} \rightarrow \mathbb{W},$$

$$\neg \exists a \in \mathbb{W} (S_{\mathbb{W}}(a) = 0), \quad \forall a, b \in \mathbb{W} (S_{\mathbb{W}}(a) = S_{\mathbb{W}}(b) \implies a = b).$$

[Go to proof.](#)

*Remark* (Interpretation). Adjoining 0 shifts the base of the successor chain from 1 to 0. The resulting successor structure has a base element, closes under successor, has no predecessor for the base, and keeps successor injective.

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Successor on  \$\mathbb{W}\$](#)
- [Successor on  \$\mathbb{W}\$  Is Well-Defined](#)
- [0 Is Not a Successor in  \$\mathbb{W}\$](#)
- [Successor on  \$\mathbb{W}\$  Is Injective](#)

## 7.4 Extending Addition to $\mathbb{W}$

### 7.4.1 Extending Addition to $\mathbb{W}$

#### Addition with Zero

Addition on  $\mathbb{W}$  keeps the old natural-number addition whenever both inputs are positive and adds identity clauses for the new element 0. The case split is exclusive because  $0 \notin \mathbb{N}$ .

## The Extended Addition Operation

### Definition (Addition on $\mathbb{W}$ )

**Definition 7.4.1** (Addition on  $\mathbb{W}$ ). Define  $+_{\mathbb{W}} : \mathbb{W} \times \mathbb{W} \rightarrow \mathbb{W}$  by

$$0 +_{\mathbb{W}} a = a, \quad a +_{\mathbb{W}} 0 = a,$$

and, for  $a, b \in \mathbb{N}$ ,

$$a +_{\mathbb{W}} b = a +_{\mathbb{N}} b.$$

*Remark* (Standard quantified statement).

$$\begin{aligned} &+_{\mathbb{W}} : \mathbb{W} \times \mathbb{W} \rightarrow \mathbb{W}, \\ &\forall a \in \mathbb{W} (0 +_{\mathbb{W}} a = a \wedge a +_{\mathbb{W}} 0 = a), \\ &\forall a, b \in \mathbb{N} (a +_{\mathbb{W}} b = a +_{\mathbb{N}} b). \end{aligned}$$

*Remark* (Interpretation). If either input is 0, the new identity clause applies. If both inputs lie in  $\mathbb{N}$ , the old operation is used. Since  $0 \notin \mathbb{N}$ , the positive-positive clause does not overlap with the zero clauses.

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Addition on a Peano System](#)

### Theorem (Addition on $\mathbb{W}$ Is Well-Defined)

**Theorem 7.4.2** (Addition on  $\mathbb{W}$  Is Well-Defined). *The case definition of  $+_{\mathbb{W}}$  determines a binary operation on  $\mathbb{W}$ . [Go to proof](#).*

*Remark* (Standard quantified statement).

$$+_{\mathbb{W}} \text{ is a well-defined binary operation on } \mathbb{W}.$$

*Remark* (Interpretation). Well-definedness is a case check. The zero cases are new, the positive-positive case is inherited from  $\mathbb{N}$ , and compatibility is automatic because the positive-positive case cannot contain 0.

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Addition on  \$\mathbb{W}\$](#)
- [Addition Is Well-Defined on a Peano System](#)



#### Theorem (Zero Is an Additive Identity on $\mathbb{W}$ )

**Theorem 7.4.3** (Zero Is an Additive Identity on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 +_{\mathbb{W}} a = a \quad \text{and} \quad a +_{\mathbb{W}} 0 = a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a \in \mathbb{W} \left( 0 +_{\mathbb{W}} a = a \wedge a +_{\mathbb{W}} 0 = a \right).$$

*Remark* (Interpretation). This is the structural feature missing from the one-based natural numbers. After adjoining 0, addition has a genuine identity element.

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)

#### Theorem (Left Additive Identity on $\mathbb{W}$ )

**Theorem 7.4.4** (Left Additive Identity on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 +_{\mathbb{W}} a = a.$$

[Go to proof.](#)

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)

#### Theorem (Right Additive Identity on $\mathbb{W}$ )

**Theorem 7.4.5** (Right Additive Identity on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$a +_{\mathbb{W}} 0 = a.$$

[Go to proof.](#)

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)

#### Theorem (Addition on $\mathbb{W}$ Extends Addition on $\mathbb{N}$ )

**Theorem 7.4.6** (Addition on  $\mathbb{W}$  Extends Addition on  $\mathbb{N}$ ). *For all  $a, b \in \mathbb{N}$ ,*

$$a +_{\mathbb{W}} b = a +_{\mathbb{N}} b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{N} \ (a +_{\mathbb{W}} b = a +_{\mathbb{N}} b).$$

*Remark* (Interpretation). The extension does not alter sums of positive natural numbers.

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)
- [Addition on a Peano System](#)

#### Theorem (Addition on $\mathbb{W}$ Is Associative)

**Theorem 7.4.7** (Addition on  $\mathbb{W}$  Is Associative). *For all  $a, b, c \in \mathbb{W}$ ,*

$$(a +_{\mathbb{W}} b) +_{\mathbb{W}} c = a +_{\mathbb{W}} (b +_{\mathbb{W}} c).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b, c \in \mathbb{W} \ ((a +_{\mathbb{W}} b) +_{\mathbb{W}} c = a +_{\mathbb{W}} (b +_{\mathbb{W}} c)).$$

*Remark* (Interpretation). Associativity follows by reducing zero cases to the identity law and reducing the positive-positive-positive case to associativity on  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)
- [Addition Is Associative](#)

#### Theorem (Addition on $\mathbb{W}$ Is Commutative)

**Theorem 7.4.8** (Addition on  $\mathbb{W}$  Is Commutative). *For all  $a, b \in \mathbb{W}$ ,*

$$a +_{\mathbb{W}} b = b +_{\mathbb{W}} a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} \ (a +_{\mathbb{W}} b = b +_{\mathbb{W}} a).$$

*Remark* (Interpretation). The zero cases are symmetric by definition, and the positive-positive case is the commutativity theorem already proved for  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)

- [Addition Is Commutative](#)

#### Theorem (Addition on $\mathbb{W}$ Satisfies Cancellation)

**Theorem 7.4.9** (Addition on  $\mathbb{W}$  Satisfies Cancellation). *For all  $a, b, c \in \mathbb{W}$ ,*

$$a +_{\mathbb{W}} b = a +_{\mathbb{W}} c \implies b = c.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b, c \in \mathbb{W} (a +_{\mathbb{W}} b = a +_{\mathbb{W}} c \implies b = c).$$

*Remark* (Interpretation). When  $a = 0$ , cancellation is the identity law. When all relevant entries are positive, it is the natural-number cancellation theorem.

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)
- [Left Cancellation for Addition](#)

## 7.5 Extending Multiplication to $\mathbb{W}$

### 7.5.1 Extending Multiplication to $\mathbb{W}$

#### Multiplication with Zero

Multiplication on  $\mathbb{W}$  keeps the old natural-number multiplication on positive inputs and adds the absorbing behavior of the new element 0. The cases are compatible because  $0 \notin \mathbb{N}$ .

### The Extended Multiplication Operation

#### Definition (Multiplication on $\mathbb{W}$ )

**Definition 7.5.1** (Multiplication on  $\mathbb{W}$ ). Define  $\cdot_{\mathbb{W}} : \mathbb{W} \times \mathbb{W} \rightarrow \mathbb{W}$  by

$$0 \cdot_{\mathbb{W}} a = 0, \quad a \cdot_{\mathbb{W}} 0 = 0,$$

and, for  $a, b \in \mathbb{N}$ ,

$$a \cdot_{\mathbb{W}} b = a \cdot_{\mathbb{N}} b.$$

*Remark* (Standard quantified statement).

$$\begin{aligned} &\cdot_{\mathbb{W}} : \mathbb{W} \times \mathbb{W} \rightarrow \mathbb{W}, \\ &\forall a \in \mathbb{W} (0 \cdot_{\mathbb{W}} a = 0 \wedge a \cdot_{\mathbb{W}} 0 = 0), \\ &\forall a, b \in \mathbb{N} (a \cdot_{\mathbb{W}} b = a \cdot_{\mathbb{N}} b). \end{aligned}$$

*Remark* (Interpretation). The zero cases are new. The positive-positive case is inherited from  $\mathbb{N}$ . Since  $0 \notin \mathbb{N}$ , these clauses do not compete.

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Multiplication on a Peano System](#)

#### Theorem (Multiplication on $\mathbb{W}$ Is Well-Defined)

**Theorem 7.5.2** (Multiplication on  $\mathbb{W}$  Is Well-Defined). *The case definition of  $\cdot_{\mathbb{W}}$  determines a binary operation on  $\mathbb{W}$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$\cdot_{\mathbb{W}}$  is a well-defined binary operation on  $\mathbb{W}$ .

*Remark* (Interpretation). The inherited positive case is already well-defined on  $\mathbb{N}$ , and the new zero cases land in the distinguished element  $0 \in \mathbb{W}$ .

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Is Well-Defined on a Peano System](#)

#### Theorem (Zero Is Absorbing for Multiplication on $\mathbb{W}$ )

**Theorem 7.5.3** (Zero Is Absorbing for Multiplication on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 \cdot_{\mathbb{W}} a = 0 \quad \text{and} \quad a \cdot_{\mathbb{W}} 0 = 0.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a \in \mathbb{W} (0 \cdot_{\mathbb{W}} a = 0 \wedge a \cdot_{\mathbb{W}} 0 = 0).$$

*Remark* (Interpretation). The adjoined element  $0$  is absorbing for multiplication on both sides.

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)

#### Theorem (Left Zero Absorption on $\mathbb{W}$ )

**Theorem 7.5.4** (Left Zero Absorption on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 \cdot_{\mathbb{W}} a = 0.$$

[Go to proof.](#)

*Remark* (Dependencies).

- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)

#### Theorem (Right Zero Absorption on $\mathbb{W}$ )

**Theorem 7.5.5** (Right Zero Absorption on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$a \cdot_{\mathbb{W}} 0 = 0.$$

[Go to proof.](#)

*Remark* (Dependencies).

- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)

#### Theorem (One Is a Multiplicative Identity on $\mathbb{W}$ )

**Theorem 7.5.6** (One Is a Multiplicative Identity on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$1 \cdot_{\mathbb{W}} a = a \quad \text{and} \quad a \cdot_{\mathbb{W}} 1 = a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a \in \mathbb{W} \left( 1 \cdot_{\mathbb{W}} a = a \wedge a \cdot_{\mathbb{W}} 1 = a \right).$$

*Remark* (Interpretation). The old multiplicative identity remains an identity after adjoining 0.

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication With 1](#)
- [One Times  \$n\$](#)

#### Theorem (Multiplication on $\mathbb{W}$ Extends Multiplication on $\mathbb{N}$ )

**Theorem 7.5.7** (Multiplication on  $\mathbb{W}$  Extends Multiplication on  $\mathbb{N}$ ). *For all  $a, b \in \mathbb{N}$ ,*

$$a \cdot_{\mathbb{W}} b = a \cdot_{\mathbb{N}} b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{N} \left( a \cdot_{\mathbb{W}} b = a \cdot_{\mathbb{N}} b \right).$$

*Remark* (Interpretation). The extension does not alter products of positive natural numbers.

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication on a Peano System](#)

#### Theorem (Multiplication on $\mathbb{W}$ Is Associative)

**Theorem 7.5.8** (Multiplication on  $\mathbb{W}$  Is Associative). *For all  $a, b, c \in \mathbb{W}$ ,*

$$(a \cdot_{\mathbb{W}} b) \cdot_{\mathbb{W}} c = a \cdot_{\mathbb{W}} (b \cdot_{\mathbb{W}} c).$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b, c \in \mathbb{W} \left( (a \cdot_{\mathbb{W}} b) \cdot_{\mathbb{W}} c = a \cdot_{\mathbb{W}} (b \cdot_{\mathbb{W}} c) \right).$$

*Remark* (Interpretation). Associativity follows from the absorbing zero cases and the inherited associativity of multiplication on positive natural numbers.

*Remark* (Dependencies).

- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Is Associative](#)

#### Theorem (Multiplication on $\mathbb{W}$ Is Commutative)

**Theorem 7.5.9** (Multiplication on  $\mathbb{W}$  Is Commutative). *For all  $a, b \in \mathbb{W}$ ,*

$$a \cdot_{\mathbb{W}} b = b \cdot_{\mathbb{W}} a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} \left( a \cdot_{\mathbb{W}} b = b \cdot_{\mathbb{W}} a \right).$$

*Remark* (Interpretation). The zero cases are symmetric, and the positive-positive case is the inherited commutativity theorem for  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Is Commutative](#)

### Theorem (Multiplication Distributes over Addition on $\mathbb{W}$ )

**Theorem 7.5.10** (Multiplication Distributes over Addition on  $\mathbb{W}$ ). *For all  $a, b, c \in \mathbb{W}$ ,*

$$a \cdot_{\mathbb{W}} (b +_{\mathbb{W}} c) = (a \cdot_{\mathbb{W}} b) +_{\mathbb{W}} (a \cdot_{\mathbb{W}} c).$$

*Go to proof.*

*Remark* (Standard quantified statement).

$$\forall a, b, c \in \mathbb{W} \ (a \cdot_{\mathbb{W}} (b +_{\mathbb{W}} c) = (a \cdot_{\mathbb{W}} b) +_{\mathbb{W}} (a \cdot_{\mathbb{W}} c)).$$

*Remark* (Interpretation). Distributivity reduces zero cases to the absorbing law and positive cases to the distributive law already proved on  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)
- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Distributes Over Addition](#)

## 7.6 Order on $\mathbb{W}$

### 7.6.1 Order on $\mathbb{W}$

#### Zero as the Least Element

The order on  $\mathbb{W}$  extends the order on  $\mathbb{N}$  and places the new element 0 below every whole number. Thus the positive part keeps its old order, while the extended system gains a least element.

### The Extended Order

#### Definition (Order on $\mathbb{W}$ )

**Definition 7.6.1** (Order on  $\mathbb{W}$ ). Define  $\leq_{\mathbb{W}}$  on  $\mathbb{W}$  by declaring

$$0 \leq_{\mathbb{W}} a \quad (a \in \mathbb{W}),$$

and, for  $a, b \in \mathbb{N}$ ,

$$a \leq_{\mathbb{W}} b \iff a \leq_{\mathbb{N}} b.$$

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} \ (a \leq_{\mathbb{W}} b \iff (a = 0 \vee (a, b \in \mathbb{N} \wedge a \leq_{\mathbb{N}} b))).$$

*Remark* (Interpretation). The non-strict order makes 0 least and otherwise preserves the order already defined on  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Less Than or Equal on a Peano System](#)

#### Definition (Strict Order on $\mathbb{W}$ )

**Definition 7.6.2** (Strict Order on  $\mathbb{W}$ ). Define  $<_{\mathbb{W}}$  on  $\mathbb{W}$  by declaring

$$0 <_{\mathbb{W}} n \quad (n \in \mathbb{N}),$$

and, for  $a, b \in \mathbb{N}$ ,

$$a <_{\mathbb{W}} b \iff a <_{\mathbb{N}} b.$$

No element of  $\mathbb{W}$  is strictly less than 0.

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} \left( a <_{\mathbb{W}} b \iff (a = 0 \wedge b \in \mathbb{N}) \vee (a, b \in \mathbb{N} \wedge a <_{\mathbb{N}} b) \right).$$

*Remark* (Interpretation). The non-strict order makes 0 least. The strict order says exactly that the positive natural numbers lie after 0 and keep their old strict ordering.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Strict Less Than on a Peano System](#)

#### Theorem (Order on $\mathbb{W}$ Extends Order on $\mathbb{N}$ )

**Theorem 7.6.3** (Order on  $\mathbb{W}$  Extends Order on  $\mathbb{N}$ ). For all  $a, b \in \mathbb{N}$ ,

$$a \leq_{\mathbb{W}} b \iff a \leq_{\mathbb{N}} b, \quad a <_{\mathbb{W}} b \iff a <_{\mathbb{N}} b.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{N} \left( (a \leq_{\mathbb{W}} b \iff a \leq_{\mathbb{N}} b) \wedge (a <_{\mathbb{W}} b \iff a <_{\mathbb{N}} b) \right).$$

*Remark* (Interpretation). The extended order changes only comparisons involving the new element 0.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)



- [Strict Order on  \$\mathbb{W}\$](#)

#### Theorem (Zero Is the Least Element of $\mathbb{W}$ )

**Theorem 7.6.4** (Zero Is the Least Element of  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 \leq_{\mathbb{W}} a.$$

*Go to proof.*

*Remark* (Standard quantified statement).

$$\forall a \in \mathbb{W} (0 \leq_{\mathbb{W}} a).$$

*Remark* (Interpretation). The adjoined element 0 is the first element of the whole-number order.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)

#### Theorem (Order on $\mathbb{W}$ Is Reflexive)

**Theorem 7.6.5** (Order on  $\mathbb{W}$  Is Reflexive). *For every  $a \in \mathbb{W}$ ,  $a \leq_{\mathbb{W}} a$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\forall a \in \mathbb{W} (a \leq_{\mathbb{W}} a).$$

*Remark* (Interpretation). Reflexivity is immediate for 0 and inherited from  $\mathbb{N}$  for positive elements.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Order Is Reflexive on a Peano System](#)

#### Theorem (Order on $\mathbb{W}$ Is Antisymmetric)

**Theorem 7.6.6** (Order on  $\mathbb{W}$  Is Antisymmetric). *For all  $a, b \in \mathbb{W}$ ,*

$$a \leq_{\mathbb{W}} b \text{ and } b \leq_{\mathbb{W}} a \implies a = b.$$

*Go to proof.*

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} (a \leq_{\mathbb{W}} b \wedge b \leq_{\mathbb{W}} a \implies a = b).$$

*Remark* (Interpretation). Antisymmetry follows from the least-element behavior of 0 and the inherited antisymmetry on positive natural numbers.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Order Is Antisymmetric on a Peano System](#)

#### Theorem (Order on $\mathbb{W}$ Is Transitive)

**Theorem 7.6.7** (Order on  $\mathbb{W}$  Is Transitive). *For all  $a, b, c \in \mathbb{W}$ ,*

$$a \leq_{\mathbb{W}} b \text{ and } b \leq_{\mathbb{W}} c \implies a \leq_{\mathbb{W}} c.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b, c \in \mathbb{W} \left( a \leq_{\mathbb{W}} b \wedge b \leq_{\mathbb{W}} c \implies a \leq_{\mathbb{W}} c \right).$$

*Remark* (Interpretation). Transitivity reduces to the least-element clause when 0 occurs and to the transitivity theorem on  $\mathbb{N}$  otherwise.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Order Is Transitive on a Peano System](#)

#### Theorem (Order on $\mathbb{W}$ Is Total)

**Theorem 7.6.8** (Order on  $\mathbb{W}$  Is Total). *For all  $a, b \in \mathbb{W}$ ,*

$$a \leq_{\mathbb{W}} b \text{ or } b \leq_{\mathbb{W}} a.$$

[Go to proof.](#)

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} \left( a \leq_{\mathbb{W}} b \vee b \leq_{\mathbb{W}} a \right).$$

*Remark* (Interpretation). Any comparison involving 0 is decided by leastness, while comparisons between positive elements are decided by totality on  $\mathbb{N}$ .

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Trichotomy on a Peano System](#)

#### Theorem (Well-Ordering Principle for $\mathbb{W}$ )

**Theorem 7.6.9** (Well-Ordering Principle for  $\mathbb{W}$ ). *Every nonempty subset of  $\mathbb{W}$  has a least element with respect to  $\leq_{\mathbb{W}}$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\forall A \subseteq \mathbb{W} (A \neq \emptyset \implies \exists m \in A \forall a \in A (m \leq_{\mathbb{W}} a)).$$

*Remark* (Interpretation). A nonempty subset either contains 0, which is least, or lies in the positive part where the well-ordering principle for  $\mathbb{N}$  applies.

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Zero Is the Least Element of  \$\mathbb{W}\$](#)
- [Well-Ordering Principle](#)

## 7.7 Algebraic Structure of $\mathbb{W}$

### 7.7.1 Algebraic Structure of $\mathbb{W}$

#### Packaging the Extended Operations

After addition, multiplication, and order have been extended, the main point of  $\mathbb{W}$  can be recorded structurally. The whole numbers form the zero-based arithmetic system used as the bridge from positive natural numbers to the integer construction.

### Algebraic Packages

**Theorem** ( $(\mathbb{W}, +, 0)$  Is a Commutative Monoid)

**Theorem 7.7.1** ( $(\mathbb{W}, +, 0)$  Is a Commutative Monoid). *The structure  $(\mathbb{W}, +_{\mathbb{W}}, 0)$  is a commutative monoid. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$(\mathbb{W}, +_{\mathbb{W}}, 0) \text{ is a commutative monoid.}$$

*Remark* (Interpretation). Addition on  $\mathbb{W}$  has an identity element, is associative, and is commutative.

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)
- [Addition on  \$\mathbb{W}\$  Is Associative](#)
- [Addition on  \$\mathbb{W}\$  Is Commutative](#)

**Theorem  $((\mathbb{W}, \cdot, 1)$  Is a Commutative Monoid)**

**Theorem 7.7.2**  $((\mathbb{W}, \cdot, 1)$  Is a Commutative Monoid). *The structure  $(\mathbb{W}, \cdot_{\mathbb{W}}, 1)$  is a commutative monoid. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$(\mathbb{W}, \cdot_{\mathbb{W}}, 1)$  is a commutative monoid.

*Remark* (Interpretation). Multiplication on  $\mathbb{W}$  has identity 1, is associative, and is commutative.

*Remark* (Dependencies).

- [One Is a Multiplicative Identity on  \$\mathbb{W}\$](#)
- [Multiplication on  \$\mathbb{W}\$  Is Associative](#)
- [Multiplication on  \$\mathbb{W}\$  Is Commutative](#)

**Theorem  $((\mathbb{W}, +, \cdot, 0, 1)$  Is a Commutative Semiring)**

**Theorem 7.7.3**  $((\mathbb{W}, +, \cdot, 0, 1)$  Is a Commutative Semiring). *The structure  $(\mathbb{W}, +_{\mathbb{W}}, \cdot_{\mathbb{W}}, 0, 1)$  is a commutative semiring. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$(\mathbb{W}, +_{\mathbb{W}}, \cdot_{\mathbb{W}}, 0, 1)$  is a commutative semiring.

*Remark* (Interpretation). The whole numbers have the additive and multiplicative structure needed for ordinary arithmetic, but no subtraction operation is included.

*Remark* (Dependencies).

- [\$\(\mathbb{W}, +, 0\)\$  Is a Commutative Monoid](#)
- [\$\(\mathbb{W}, \cdot, 1\)\$  Is a Commutative Monoid](#)
- [Multiplication Distributes over Addition on  \$\mathbb{W}\$](#)
- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)

**Theorem  $(\mathbb{W}$  Is a Commutative Ordered Semiring)**

**Theorem 7.7.4**  $(\mathbb{W}$  Is a Commutative Ordered Semiring). *With the operations  $+_{\mathbb{W}}$  and  $\cdot_{\mathbb{W}}$  and the order  $\leq_{\mathbb{W}}$ , the whole numbers form a commutative ordered semiring. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$(\mathbb{W}, +_{\mathbb{W}}, \cdot_{\mathbb{W}}, 0, 1, \leq_{\mathbb{W}})$  is a commutative ordered semiring.

*Remark* (Interpretation). This packages the algebraic semiring laws together with the order facts:  $\mathbb{W}$  is totally ordered, 0 is least, and the inherited arithmetic is compatible with that order.

*Remark* (Dependencies).

- [\( \$\mathbb{W}, +, \cdot, 0, 1\$ \) Is a Commutative Semiring](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)
- [Zero Is the Least Element of  \$\mathbb{W}\$](#)
- [Order on  \$\mathbb{W}\$  Extends Order on  \$\mathbb{N}\$](#)

#### Theorem ( $\mathbb{N}$ Embeds into $\mathbb{W}$ as the Positive Part)

**Theorem 7.7.5** ( $\mathbb{N}$  Embeds into  $\mathbb{W}$  as the Positive Part). *The inclusion map identifies  $\mathbb{N}$  with the positive part  $\mathbb{W} \setminus \{0\}$  and preserves successor, 1, addition, multiplication, and order. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\iota : \mathbb{N} \hookrightarrow \mathbb{W}, \quad \iota(n) = n,$$

identifies  $\mathbb{N}$  with  $\mathbb{W} \setminus \{0\}$  and preserves successor, 1, addition, multiplication, and order.

*Remark* (Interpretation). The positive part of  $\mathbb{W}$  is exactly the previously constructed natural-number system, with the same arithmetic and order.

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Addition on  \$\mathbb{W}\$  Extends Addition on  \$\mathbb{N}\$](#)
- [Multiplication on  \$\mathbb{W}\$  Extends Multiplication on  \$\mathbb{N}\$](#)
- [Order on  \$\mathbb{W}\$  Extends Order on  \$\mathbb{N}\$](#)

#### Theorem ( $\mathbb{W}$ Has No Additive Inverses Except 0)

**Theorem 7.7.6** ( $\mathbb{W}$  Has No Additive Inverses Except 0). *If  $a, b \in \mathbb{W}$  and*

$$a +_{\mathbb{W}} b = 0,$$

*then  $a = 0$  and  $b = 0$ . [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$\forall a, b \in \mathbb{W} \ (a +_{\mathbb{W}} b = 0 \implies (a = 0 \wedge b = 0)).$$

*Remark* (Interpretation). The only way to sum to zero inside  $\mathbb{W}$  is for both summands already to be zero.

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)
- [Zero Is the Least Element of  \$\mathbb{W}\$](#)
- [No Natural Number Equals Itself Plus a Natural Number](#)

#### Theorem ( $\mathbb{W}$ Is Not a Ring)

**Theorem 7.7.7** ( $\mathbb{W}$  Is Not a Ring). *With the operations  $+_{\mathbb{W}}$  and  $\cdot_{\mathbb{W}}$ , the whole numbers do not form a ring. [Go to proof.](#)*

*Remark* (Standard quantified statement).

$$(\mathbb{W}, +_{\mathbb{W}}, \cdot_{\mathbb{W}}, 0, 1) \text{ is not a ring.}$$

*Remark* (Interpretation). The obstruction is not associativity, commutativity, or distributivity. The obstruction is the absence of additive inverses for nonzero whole numbers.

*Remark* (Dependencies).

- [\( \$\mathbb{W}, +, \cdot, 0, 1\$ \) Is a Commutative Semiring](#)
- [\$\mathbb{W}\$  Has No Additive Inverses Except 0](#)

## 7.8 Transition to $\mathbb{Z}$

### 7.8.1 Transition to $\mathbb{Z}$

#### From Zero to Additive Inverses

The whole numbers fix the missing additive identity, but they do not make subtraction available in general. The integer construction repairs this by adjoining additive inverses formally.

### The Next Construction

*Remark* (Exposition). The passage from  $\mathbb{N}$  to  $\mathbb{W}$  adds the element needed for  $0 + a = a$ . It does not add solutions to equations such as  $a + x = b$  when  $b < a$ . Thus  $\mathbb{W}$  is still not closed under subtraction.

*Remark* (Exposition). The construction of  $\mathbb{Z}$  repairs this by adjoining additive inverses formally. One standard route uses ordered pairs from  $\mathbb{W} \times \mathbb{W}$ , with a pair  $(a, b)$  interpreted as a formal difference  $a - b$ . The equivalence relation then identifies different pairs that represent the same intended difference.

*Remark* (Dependencies).

- $(\mathbb{W}, +, 0)$  Is a Commutative Monoid
- $\mathbb{W}$  Has No Additive Inverses Except 0
- $\mathbb{W}$  Is Not a Ring

## Proofs

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $0 \notin \mathbb{N}$ ). *The adjoined element 0 is not an element of the one-based natural-number system:*

$$0 \notin \mathbb{N}.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-not-in-n. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-not-in-n. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Whole Numbers](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Successor on  $W$  Is Well-Defined). *The rule defining  $S_{\mathbb{W}}$  determines a function  $S_{\mathbb{W}} : \mathbb{W} \rightarrow \mathbb{W}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for successor-on-w-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for successor-on-w-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Successor on  \$\mathbb{W}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (0 Is Not a Successor in  $W$ ). *There is no  $a \in \mathbb{W}$  such that  $S_{\mathbb{W}}(a) = 0$ .*

*Professional Standard Proof.*

TODO: professional standard proof for zero-is-not-successor-in-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-is-not-successor-in-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Successor on  \$\mathbb{W}\$](#)
- [1 Is Not a Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Every Nonzero Element of  $\mathbb{W}$  Is a Successor). *If  $a \in \mathbb{W}$  and  $a \neq 0$ , then there exists  $b \in \mathbb{W}$  such that  $a = S_{\mathbb{W}}(b)$ .*

*Professional Standard Proof.*

TODO: professional standard proof for every-nonzero-w-is-successor. □

*Detailed Learning Proof.*

TODO: detailed learning proof for every-nonzero-w-is-successor. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Successor on  \$\mathbb{W}\$](#)
- [Induction Axiom](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Successor on  $W$  Is Injective). *For all  $a, b \in \mathbb{W}$ ,*

$$S_{\mathbb{W}}(a) = S_{\mathbb{W}}(b) \implies a = b.$$

*Professional Standard Proof.*

TODO: professional standard proof for successor-on-w-injective. □

*Detailed Learning Proof.*

TODO: detailed learning proof for successor-on-w-injective. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Successor on  \$\mathbb{W}\$](#)
- [Injectivity of Successor](#)
- [1 Is Not a Successor](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem**  $((\mathbb{W}, 0, S_{\mathbb{W}})$  Satisfies the Successor Properties). *The triple  $(\mathbb{W}, 0, S_{\mathbb{W}})$  satisfies the successor-side properties:*

$$\begin{aligned} 0 \in \mathbb{W}, \quad S_{\mathbb{W}} : \mathbb{W} \rightarrow \mathbb{W}, \\ \neg \exists a \in \mathbb{W} (S_{\mathbb{W}}(a) = 0), \quad \forall a, b \in \mathbb{W} (S_{\mathbb{W}}(a) = S_{\mathbb{W}}(b) \implies a = b). \end{aligned}$$

*Professional Standard Proof.*

TODO: professional standard proof for w-successor-system-properties. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-successor-system-properties. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Successor on  \$\mathbb{W}\$](#)
- [Successor on  \$\mathbb{W}\$  Is Well-Defined](#)
- [0 Is Not a Successor in  \$\mathbb{W}\$](#)
- [Successor on  \$\mathbb{W}\$  Is Injective](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $W$  Is Well-Defined). *The case definition of  $+_{\mathbb{W}}$  determines a binary operation on  $\mathbb{W}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-w-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-w-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Addition on  \$\mathbb{W}\$](#)
- [Addition Is Well-Defined on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is an Additive Identity on  $W$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 +_{\mathbb{W}} a = a \quad \text{and} \quad a +_{\mathbb{W}} 0 = a.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-additive-identity-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-additive-identity-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Additive Identity on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 +_{\mathbb{W}} a = a.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-additive-identity-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-additive-identity-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Right Additive Identity on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$a +_{\mathbb{W}} 0 = a.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-additive-identity-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-additive-identity-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $W$  Extends Addition on  $N$ ). *For all  $a, b \in \mathbb{N}$ ,*

$$a +_{\mathbb{W}} b = a +_{\mathbb{N}} b.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-w-extends-addition-on-n. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-w-extends-addition-on-n. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$W\$](#)
- [Addition on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $\mathbb{W}$  Is Associative). *For all  $a, b, c \in \mathbb{W}$ ,*

$$(a +_{\mathbb{W}} b) +_{\mathbb{W}} c = a +_{\mathbb{W}} (b +_{\mathbb{W}} c).$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-w-associative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-w-associative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)
- [Addition Is Associative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $\mathbb{W}$  Is Commutative). *For all  $a, b \in \mathbb{W}$ ,*

$$a +_{\mathbb{W}} b = b +_{\mathbb{W}} a.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-w-commutative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-w-commutative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)
- [Addition Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $\mathbb{W}$  Satisfies Cancellation). *For all  $a, b, c \in \mathbb{W}$ ,*

$$a +_{\mathbb{W}} b = a +_{\mathbb{W}} c \implies b = c.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-w-cancellation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-w-cancellation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)
- [Left Cancellation for Addition](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{W}$  Is Well-Defined). *The case definition of  $\cdot_{\mathbb{W}}$  determines a binary operation on  $\mathbb{W}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-w-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-w-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Is Well-Defined on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is Absorbing for Multiplication on  $W$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 \cdot_{\mathbb{W}} a = 0 \quad \text{and} \quad a \cdot_{\mathbb{W}} 0 = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-absorbing-for-multiplication-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-absorbing-for-multiplication-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Zero Absorption on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 \cdot_{\mathbb{W}} a = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-zero-absorption-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-zero-absorption-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Right Zero Absorption on  $\mathbb{W}$ ). *For every  $a \in \mathbb{W}$ ,*

$$a \cdot_{\mathbb{W}} 0 = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-zero-absorption-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-zero-absorption-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (One Is a Multiplicative Identity on  $W$ ). *For every  $a \in \mathbb{W}$ ,*

$$1 \cdot_{\mathbb{W}} a = a \quad \text{and} \quad a \cdot_{\mathbb{W}} 1 = a.$$

*Professional Standard Proof.*

TODO: professional standard proof for one-multiplicative-identity-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-multiplicative-identity-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication With 1](#)
- [One Times  \$n\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{W}$  Extends Multiplication on  $\mathbb{N}$ ). *For all  $a, b \in \mathbb{N}$ ,*

$$a \cdot_{\mathbb{W}} b = a \cdot_{\mathbb{N}} b.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-w-extends-multiplication-on-n. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-w-extends-multiplication-on-n. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{W}$  Is Associative). *For all  $a, b, c \in \mathbb{W}$ ,*

$$(a \cdot_{\mathbb{W}} b) \cdot_{\mathbb{W}} c = a \cdot_{\mathbb{W}} (b \cdot_{\mathbb{W}} c).$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-w-associative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-w-associative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Is Associative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{W}$  Is Commutative). *For all  $a, b \in \mathbb{W}$ ,*

$$a \cdot_{\mathbb{W}} b = b \cdot_{\mathbb{W}} a.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-w-commutative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-w-commutative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Distributes over Addition on  $W$ ). *For all  $a, b, c \in \mathbb{W}$ ,*

$$a \cdot_{\mathbb{W}} (b +_{\mathbb{W}} c) = (a \cdot_{\mathbb{W}} b) +_{\mathbb{W}} (a \cdot_{\mathbb{W}} c).$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-distributes-over-addition-on-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-distributes-over-addition-on-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)
- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication Distributes Over Addition](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $\mathbb{W}$  Extends Order on  $\mathbb{N}$ ). *For all  $a, b \in \mathbb{N}$ ,*

$$a \leq_{\mathbb{W}} b \iff a \leq_{\mathbb{N}} b, \quad a <_{\mathbb{W}} b \iff a <_{\mathbb{N}} b.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-w-extends-order-on-n. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-w-extends-order-on-n. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Strict Order on  \$\mathbb{W}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is the Least Element of  $W$ ). *For every  $a \in \mathbb{W}$ ,*

$$0 \leq_{\mathbb{W}} a.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-least-element-of-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-least-element-of-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $W$  Is Reflexive). *For every  $a \in \mathbb{W}$ ,  $a \leq_{\mathbb{W}} a$ .*

*Professional Standard Proof.*

TODO: professional standard proof for order-on-w-reflexive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-w-reflexive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Order Is Reflexive on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $W$  Is Antisymmetric). *For all  $a, b \in \mathbb{W}$ ,*

$$a \leq_{\mathbb{W}} b \text{ and } b \leq_{\mathbb{W}} a \implies a = b.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-w-antisymmetric. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-w-antisymmetric. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Order Is Antisymmetric on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $W$  Is Transitive). *For all  $a, b, c \in \mathbb{W}$ ,*

$$a \leq_{\mathbb{W}} b \text{ and } b \leq_{\mathbb{W}} c \implies a \leq_{\mathbb{W}} c.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-w-transitive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-w-transitive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Order Is Transitive on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $W$  Is Total). *For all  $a, b \in \mathbb{W}$ ,*

$$a \leq_{\mathbb{W}} b \quad \text{or} \quad b \leq_{\mathbb{W}} a.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-w-total. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-w-total. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Trichotomy on a Peano System](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Well-Ordering Principle for  $\mathbb{W}$ ). *Every nonempty subset of  $\mathbb{W}$  has a least element with respect to  $\leq_{\mathbb{W}}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for well-ordering-principle-for-w. □

*Detailed Learning Proof.*

TODO: detailed learning proof for well-ordering-principle-for-w. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$](#)
- [Zero Is the Least Element of  \$\mathbb{W}\$](#)
- [Well-Ordering Principle](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem**  $((W, +, 0)$  Is a Commutative Monoid). *The structure  $(\mathbb{W}, +_{\mathbb{W}}, 0)$  is a commutative monoid.*

*Professional Standard Proof.*

TODO: professional standard proof for w-additive-commutative-monoid. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-additive-commutative-monoid. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)
- [Addition on  \$\mathbb{W}\$  Is Associative](#)
- [Addition on  \$\mathbb{W}\$  Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem**  $((W, +, 1)$  Is a Commutative Monoid). *The structure  $(\mathbb{W}, \cdot_{\mathbb{W}}, 1)$  is a commutative monoid.*

*Professional Standard Proof.*

TODO: professional standard proof for w-multiplicative-commutative-monoid. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-multiplicative-commutative-monoid. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [One Is a Multiplicative Identity on  \$\mathbb{W}\$](#)
- [Multiplication on  \$\mathbb{W}\$  Is Associative](#)
- [Multiplication on  \$\mathbb{W}\$  Is Commutative](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem**  $((W, +, \cdot, 0, 1)$  Is a Commutative Semiring). *The structure  $(\mathbb{W}, +_{\mathbb{W}}, \cdot_{\mathbb{W}}, 0, 1)$  is a commutative semiring.*

*Professional Standard Proof.*

TODO: professional standard proof for w-commutative-semiring. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-commutative-semiring. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [\( \$\mathbb{W}, +, 0\$ \) Is a Commutative Monoid](#)
- [\( \$\mathbb{W}, \cdot, 1\$ \) Is a Commutative Monoid](#)
- [Multiplication Distributes over Addition on  \$\mathbb{W}\$](#)
- [Zero Is Absorbing for Multiplication on  \$\mathbb{W}\$](#)



*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{W}$  Is a Commutative Ordered Semiring). *With the operations  $+_{\mathbb{W}}$  and  $\cdot_{\mathbb{W}}$  and the order  $\leq_{\mathbb{W}}$ , the whole numbers form a commutative ordered semiring.*

*Professional Standard Proof.*

TODO: professional standard proof for w-commutative-ordered-semiring. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-commutative-ordered-semiring. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [\( \$\mathbb{W}, +, \cdot, 0, 1\$ \) Is a Commutative Semiring](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)
- [Zero Is the Least Element of  \$\mathbb{W}\$](#)
- [Order on  \$\mathbb{W}\$  Extends Order on  \$\mathbb{N}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (*N Embeds into W as the Positive Part*). *The inclusion map identifies  $\mathbb{N}$  with the positive part  $\mathbb{W} \setminus \{0\}$  and preserves successor, 1, addition, multiplication, and order.*

*Professional Standard Proof.*

TODO: professional standard proof for n-embeds-into-w-as-positive-part. □

*Detailed Learning Proof.*

TODO: detailed learning proof for n-embeds-into-w-as-positive-part. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Addition on  \$\mathbb{W}\$  Extends Addition on  \$\mathbb{N}\$](#)
- [Multiplication on  \$\mathbb{W}\$  Extends Multiplication on  \$\mathbb{N}\$](#)
- [Order on  \$\mathbb{W}\$  Extends Order on  \$\mathbb{N}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{W}$  Has No Additive Inverses Except 0). If  $a, b \in \mathbb{W}$  and

$$a +_{\mathbb{W}} b = 0,$$

then  $a = 0$  and  $b = 0$ .

*Professional Standard Proof.*

TODO: professional standard proof for w-has-no-additive-inverses-except-zero. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-has-no-additive-inverses-except-zero. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$](#)
- [Zero Is the Least Element of  \$\mathbb{W}\$](#)
- [No Natural Number Equals Itself Plus a Natural Number](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (*W Is Not a Ring*). *With the operations  $+\mathbb{W}$  and  $\cdot_{\mathbb{W}}$ , the whole numbers do not form a ring.*

*Professional Standard Proof.*

TODO: professional standard proof for w-is-not-a-ring. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-is-not-a-ring. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [\( \$\mathbb{W}, +, \cdot, 0, 1\$ \) Is a Commutative Semiring](#)
- [W Has No Additive Inverses Except 0](#)

# Capstone

# Chapter 8

## Integers ( $\mathbb{Z}$ )



### 8.1 Construction of $\mathbb{Z}$

#### Integer Prepairs

##### Definition

**Definition 8.1.1** (Integer Prepair). An *integer prepair* is an ordered pair  $(a, b) \in \mathbb{W} \times \mathbb{W}$ , read as the formal difference  $a - b$ .

*Remark* (Dependencies).

- [Ordered Pair](#)
- [Whole Numbers](#)

##### Definition

**Definition 8.1.2** (Integer Prepair Set). The integer prepair set is

$$\text{Pre } \mathbb{Z} := \mathbb{W} \times \mathbb{W}.$$

*Remark* (Dependencies).

- [Cartesian Product](#)
- [Whole Numbers](#)

## Formal-Difference Equivalence

### Definition

**Definition 8.1.3** (Formal-Difference Equivalence). For  $(a, b), (c, d) \in \text{Pre } \mathbb{Z}$ , define

$$(a, b) \sim_{\mathbb{Z}} (c, d) \quad :\Longleftrightarrow \quad a + d = c + b.$$

*Remark* (Dependencies).

- [Relation](#)
- [Integer Prepair Set](#)
- [Addition on  \$\mathbb{W}\$](#)

### Lemma

**Lemma 8.1.4** (Formal-Difference Equivalence Is Reflexive). *For every  $(a, b) \in \text{Pre } \mathbb{Z}$ ,*

$$(a, b) \sim_{\mathbb{Z}} (a, b).$$

*Remark* (Dependencies).

- [Formal-Difference Equivalence](#)
- [Addition on  \$\mathbb{W}\$](#)

### Lemma

**Lemma 8.1.5** (Formal-Difference Equivalence Is Symmetric). *For all prepairs  $(a, b), (c, d)$ ,*

$$(a, b) \sim_{\mathbb{Z}} (c, d) \implies (c, d) \sim_{\mathbb{Z}} (a, b).$$

*Remark* (Dependencies).

- [Formal-Difference Equivalence](#)
- [Addition on  \$\mathbb{W}\$](#)

### Lemma

**Lemma 8.1.6** (Formal-Difference Equivalence Is Transitive). *For all prepairs  $(a, b), (c, d), (e, f)$ ,*

$$(a, b) \sim_{\mathbb{Z}} (c, d) \wedge (c, d) \sim_{\mathbb{Z}} (e, f) \implies (a, b) \sim_{\mathbb{Z}} (e, f).$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)

### Theorem

**Theorem 8.1.7** (Formal-Difference Equivalence Is an Equivalence Relation). *The relation  $\sim_{\mathbb{Z}}$  is an equivalence relation on  $\text{Pre } \mathbb{Z}$ .*

*Remark* (Dependencies).

- [Formal-Difference Equivalence](#)
- [Formal-Difference Equivalence Is Reflexive](#)
- [Formal-Difference Equivalence Is Symmetric](#)
- [Formal-Difference Equivalence Is Transitive](#)

## Definition of $\mathbb{Z}$

### Definition

**Definition 8.1.8** (Integers). The integers are the quotient

$$\mathbb{Z} := \text{Pre } \mathbb{Z} / \sim_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [Quotient Set](#)
- [Integer Prepair Set](#)
- [Formal-Difference Equivalence](#)

### Definition

**Definition 8.1.9** (Integer Class Notation). The notation  $[a, b]$  denotes the  $\sim_{\mathbb{Z}}$ -equivalence class of the prepair  $(a, b)$ .

*Remark* (Dependencies).

- [Integers](#)
- [Formal-Difference Equivalence](#)

## Zero and One in $\mathbb{Z}$

### Definition

**Definition 8.1.10** (Zero in  $\mathbb{Z}$ ). Define

$$0_{\mathbb{Z}} := [0, 0].$$

*Remark* (Dependencies).



- [Integer Class Notation](#)
- [Whole Numbers](#)

#### Definition

**Definition 8.1.11** (One in  $\mathbb{Z}$ ). Define

$$1_{\mathbb{Z}} := [1, 0].$$

*Remark* (Dependencies).

- [Integer Class Notation](#)
- [Whole Numbers](#)

#### Theorem

**Theorem 8.1.12** (Zero Is an Integer). *The class  $0_{\mathbb{Z}}$  is an element of  $\mathbb{Z}$ .*

*Remark* (Dependencies).

- [Integers](#)
- [Zero in  \$\mathbb{Z}\$](#)
- [Integer Class Notation](#)

#### Theorem

**Theorem 8.1.13** (One Is an Integer). *The class  $1_{\mathbb{Z}}$  is an element of  $\mathbb{Z}$ .*

*Remark* (Dependencies).

- [Integers](#)
- [One in  \$\mathbb{Z}\$](#)
- [Integer Class Notation](#)

#### Theorem

**Theorem 8.1.14** (Zero Is Not One in  $\mathbb{Z}$ ). *In  $\mathbb{Z}$ ,*

$$0_{\mathbb{Z}} \neq 1_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [0  \$\notin \mathbb{N}\$](#)

## 8.2 Operations on $\mathbb{Z}$ -Classes

### Addition on $\mathbb{Z}$ -Classes

#### Definition

**Definition 8.2.1** (Addition on  $\mathbb{Z}$ -Classes). Define addition on representatives by

$$[a, b] + [c, d] := [a + c, b + d].$$

*Remark* (Dependencies).

- [Integer Class Notation](#)
- [Addition on  \$\mathbb{W}\$](#)

#### Lemma

**Lemma 8.2.2** (Addition Respects Formal-Difference Equivalence on the Left). *If  $[a, b] = [a', b']$ , then*

$$[a, b] + [c, d] = [a', b'] + [c, d].$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Formal-Difference Equivalence](#)
- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)

#### Lemma

**Lemma 8.2.3** (Addition Respects Formal-Difference Equivalence on the Right). *If  $[c, d] = [c', d']$ , then*

$$[a, b] + [c, d] = [a, b] + [c', d'].$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Formal-Difference Equivalence](#)
- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)

#### Lemma

**Lemma 8.2.4** (Addition Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$  and  $[c, d] = [c', d']$ , then*

$$[a, b] + [c, d] = [a', b'] + [c', d'].$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Addition Respects Formal-Difference Equivalence on the Left](#)
- [Addition Respects Formal-Difference Equivalence on the Right](#)

#### Theorem

**Theorem 8.2.5** (Addition on  $\mathbb{Z}$  Is Well-Defined). *Addition determines a well-defined binary operation  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Addition Respects Formal-Difference Equivalence](#)

#### Theorem

**Theorem 8.2.6** (Addition on  $\mathbb{Z}$  Is Associative). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$(x + y) + z = x + (y + z).$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Addition on  \$\mathbb{W}\$  Is Associative](#)

#### Theorem

**Theorem 8.2.7** (Addition on  $\mathbb{Z}$  Is Commutative). *For all  $x, y \in \mathbb{Z}$ ,*

$$x + y = y + x.$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Addition on  \$\mathbb{W}\$  Is Commutative](#)

#### Theorem

**Theorem 8.2.8** (Zero Is a Left Additive Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$0_{\mathbb{Z}} + x = x.$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Zero in  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Zero Is an Additive Identity on  \$\mathbb{W}\$](#)

#### Corollary

**Corollary 8.2.9** (Zero Is a Right Additive Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$x + 0_{\mathbb{Z}} = x.$$

*Remark* (Dependencies).

- [Zero Is a Left Additive Identity on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Commutative](#)

#### Theorem

**Theorem 8.2.10** (Zero Is an Additive Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$0_{\mathbb{Z}} + x = x \quad \text{and} \quad x + 0_{\mathbb{Z}} = x.$$

*Remark* (Dependencies).

- [Zero Is a Left Additive Identity on  \$\mathbb{Z}\$](#)
- [Zero Is a Right Additive Identity on  \$\mathbb{Z}\$](#)

## Negation on $\mathbb{Z}$ -Classes

#### Definition

**Definition 8.2.11** (Negation on  $\mathbb{Z}$ -Classes). Define negation on representatives by

$$-[a, b] := [b, a].$$

*Remark* (Dependencies).

- [Integer Class Notation](#)

#### Lemma

**Lemma 8.2.12** (Negation Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$ , then*

$$-[a, b] = -[a', b'].$$

*Remark* (Dependencies).

- [Negation on  \$\mathbb{Z}\$](#)
- [Formal-Difference Equivalence](#)

#### Theorem

**Theorem 8.2.13** (Negation on  $\mathbb{Z}$  Is Well-Defined). *Negation determines a well-defined unary operation  $\mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Remark* (Dependencies).

- [Negation on  \$\mathbb{Z}\$](#)
- [Negation Respects Formal-Difference Equivalence](#)

#### Theorem

**Theorem 8.2.14** (Negation Gives a Left Additive Inverse on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$(-x) + x = 0_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [Negation on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$](#)
- [Zero in  \$\mathbb{Z}\$](#)
- [Negation on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)

#### Corollary

**Corollary 8.2.15** (Negation Gives a Right Additive Inverse on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$x + (-x) = 0_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [Negation Gives a Left Additive Inverse on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Commutative](#)

#### Theorem

**Theorem 8.2.16** (Every Integer Has an Additive Inverse). *For every  $x \in \mathbb{Z}$  there exists  $y \in \mathbb{Z}$  such that*

$$x + y = 0_{\mathbb{Z}} \quad \text{and} \quad y + x = 0_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [Negation Gives a Left Additive Inverse on  \$\mathbb{Z}\$](#)
- [Negation Gives a Right Additive Inverse on  \$\mathbb{Z}\$](#)

## Subtraction on $\mathbb{Z}$ -Classes

### Definition

**Definition 8.2.17** (Subtraction on  $\mathbb{Z}$ ). For  $x, y \in \mathbb{Z}$ , define

$$x - y := x + (-y).$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Negation on  \$\mathbb{Z}\$](#)

### Theorem

**Theorem 8.2.18** (Subtraction on  $\mathbb{Z}$  Is Well-Defined). *Subtraction determines a well-defined binary operation  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Remark* (Dependencies).

- [Subtraction on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Negation on  \$\mathbb{Z}\$  Is Well-Defined](#)

## Multiplication on $\mathbb{Z}$ -Classes

### Definition

**Definition 8.2.19** (Multiplication on  $\mathbb{Z}$ -Classes). Define multiplication on representatives by

$$[a, b] \cdot [c, d] := [ac + bd, ad + bc].$$

*Remark* (Dependencies).

- [Integer Class Notation](#)
- [Addition on  \$\mathbb{W}\$](#)
- [Multiplication on  \$\mathbb{W}\$](#)

### Lemma

**Lemma 8.2.20** (Multiplication Respects Formal-Difference Equivalence on the Left). *If  $[a, b] = [a', b']$ , then*

$$[a, b] \cdot [c, d] = [a', b'] \cdot [c, d].$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

#### Lemma

**Lemma 8.2.21** (Multiplication Respects Formal-Difference Equivalence on the Right). *If  $[c, d] = [c', d']$ , then*

$$[a, b] \cdot [c, d] = [a, b] \cdot [c', d'].$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

#### Lemma

**Lemma 8.2.22** (Multiplication Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$  and  $[c, d] = [c', d']$ , then*

$$[a, b] \cdot [c, d] = [a', b'] \cdot [c', d'].$$

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{Z}\$](#)
- [Multiplication Respects Formal-Difference Equivalence on the Left](#)
- [Multiplication Respects Formal-Difference Equivalence on the Right](#)

#### Theorem

**Theorem 8.2.23** (Multiplication on  $\mathbb{Z}$  Is Well-Defined). *Multiplication determines a well-defined binary operation  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{Z}\$](#)
- [Multiplication Respects Formal-Difference Equivalence](#)

#### Theorem

**Theorem 8.2.24** (Multiplication on  $\mathbb{Z}$  Is Associative). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$(x \cdot y) \cdot z = x \cdot (y \cdot z).$$

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{Z}\$](#)

- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Multiplication on  \$\mathbb{W}\$  Is Associative](#)

#### Theorem

**Theorem 8.2.25** (Multiplication on  $\mathbb{Z}$  Is Commutative). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \cdot y = y \cdot x.$$

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Multiplication on  \$\mathbb{W}\$  Is Commutative](#)

#### Theorem

**Theorem 8.2.26** (One Is a Left Multiplicative Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$1_{\mathbb{Z}} \cdot x = x.$$

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [One Is a Multiplicative Identity on  \$\mathbb{W}\$](#)

#### Corollary

**Corollary 8.2.27** (One Is a Right Multiplicative Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$x \cdot 1_{\mathbb{Z}} = x.$$

*Remark* (Dependencies).

- [One Is a Left Multiplicative Identity on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Commutative](#)

#### Theorem

**Theorem 8.2.28** (One Is a Multiplicative Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$1_{\mathbb{Z}} \cdot x = x \quad \text{and} \quad x \cdot 1_{\mathbb{Z}} = x.$$

*Remark* (Dependencies).

- [One Is a Left Multiplicative Identity on  \$\mathbb{Z}\$](#)
- [One Is a Right Multiplicative Identity on  \$\mathbb{Z}\$](#)



## Distributivity on $\mathbb{Z}$ -Classes

### Theorem

**Theorem 8.2.29** (Left Distributivity on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \cdot (y + z) = x \cdot y + x \cdot z.$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$](#)
- [Multiplication Distributes over Addition on  \$\mathbb{W}\$](#)

### Corollary

**Corollary 8.2.30** (Right Distributivity on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$(y + z) \cdot x = y \cdot x + z \cdot x.$$

*Remark* (Dependencies).

- [Left Distributivity on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Commutative](#)

### Theorem

**Theorem 8.2.31** (Multiplication Distributes over Addition on  $\mathbb{Z}$ ). *Multiplication distributes over addition on both sides in  $\mathbb{Z}$ .*

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$](#)
- [Left Distributivity on  \$\mathbb{Z}\$](#)
- [Right Distributivity on  \$\mathbb{Z}\$](#)

## Class-Level Ring Summary

### Theorem

**Theorem 8.2.32** ( $\mathbb{Z}$  Is an Additive Abelian Group). *The structure  $(\mathbb{Z}, +, 0_{\mathbb{Z}})$  is an abelian group.*

*Remark* (Dependencies).

- [Addition on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Associative](#)
- [Addition on  \$\mathbb{Z}\$  Is Commutative](#)
- [Zero Is an Additive Identity on  \$\mathbb{Z}\$](#)
- [Every Integer Has an Additive Inverse](#)

#### Theorem

**Theorem 8.2.33** ( $\mathbb{Z}$  Is a Multiplicative Commutative Monoid). *The structure  $(\mathbb{Z}, \cdot, 1_{\mathbb{Z}})$  is a commutative monoid.*

*Remark* (Dependencies).

- [Multiplication on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Associative](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Commutative](#)
- [One Is a Multiplicative Identity on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.2.34** ( $\mathbb{Z}$  Is a Commutative Ring). *The structure  $(\mathbb{Z}, +, \cdot, 0_{\mathbb{Z}}, 1_{\mathbb{Z}})$  is a commutative ring.*

*Remark* (Dependencies).

- [\$\mathbb{Z}\$  Is an Additive Abelian Group](#)
- [\$\mathbb{Z}\$  Is a Multiplicative Commutative Monoid](#)
- [Multiplication Distributes over Addition on  \$\mathbb{Z}\$](#)

## 8.3 Algebraic Structure of $\mathbb{Z}$

### Integral Domain Structure

#### Theorem

**Theorem 8.3.1** ( $\mathbb{Z}$  Has No Zero Divisors). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \cdot y = 0 \implies x = 0 \text{ or } y = 0.$$

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$  Is Total](#)

- [Cancellation for Multiplication](#)

#### Theorem

**Theorem 8.3.2** ( $\mathbb{Z}$  Is an Integral Domain). *The integers form an integral domain:  $\mathbb{Z}$  is a commutative ring,  $0 \neq 1$ , and  $\mathbb{Z}$  has no zero divisors.*

*Remark* (Dependencies).

- [\$\mathbb{Z}\$  Is a Commutative Ring](#)
- [Zero Is Not One in  \$\mathbb{Z}\$](#)
- [\$\mathbb{Z}\$  Has No Zero Divisors](#)

#### Corollary

**Corollary 8.3.3** (Multiplicative Cancellation on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$z \neq 0 \wedge z \cdot x = z \cdot y \implies x = y.$$

*Remark* (Dependencies).

- [\$\mathbb{Z}\$  Has No Zero Divisors](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Commutative](#)

## Derived Laws on $\mathbb{Z}$

#### Corollary

**Corollary 8.3.4** (Zero Absorption on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$0 \cdot x = 0 \quad \text{and} \quad x \cdot 0 = 0.$$

*Remark* (Dependencies).

- [\$\mathbb{Z}\$  Is a Commutative Ring](#)
- [Zero in  \$\mathbb{Z}\$](#)

#### Corollary

**Corollary 8.3.5** (Sign Rule on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$(-x) \cdot y = -(x \cdot y) \quad \text{and} \quad x \cdot (-y) = -(x \cdot y).$$

*Remark* (Dependencies).

- [\$\mathbb{Z}\$  Is a Commutative Ring](#)

- [Negation on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Multiplication Distributes over Addition on  \$\mathbb{Z}\$](#)

#### Corollary

**Corollary 8.3.6** (Negative One Rule on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$(-1) \cdot x = -x.$$

*Remark* (Dependencies).

- [Sign Rule on  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)

#### Corollary

**Corollary 8.3.7** (Product of Negatives on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$(-x) \cdot (-y) = x \cdot y.$$

*Remark* (Dependencies).

- [Sign Rule on  \$\mathbb{Z}\$](#)
- [Negative One Rule on  \$\mathbb{Z}\$](#)

## 8.4 Order on $\mathbb{Z}$

### Positivity on $\mathbb{Z}$

#### Definition

**Definition 8.4.1** (Positive Integer). An integer  $x \in \mathbb{Z}$  is *positive* if  $x = [a, b]$  for some  $a, b \in \mathbb{W}$  with  $b < a$ .

*Remark* (Dependencies).

- [Integer Class Notation](#)
- [Whole Numbers](#)
- [Order on  \$\mathbb{W}\$](#)

#### Lemma

**Lemma 8.4.2** (Positivity Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$ , then*

$$b < a \iff b' < a'.$$

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

#### Theorem

**Theorem 8.4.3** (Positivity on  $\mathbb{Z}$  Is Well-Defined). *The predicate “ $x$  is positive” is well-defined on equivalence classes in  $\mathbb{Z}$ .*

*Remark* (Dependencies).

- [Positive Integer](#)
- [Formal-Difference Equivalence](#)
- [Positivity Respects Formal-Difference Equivalence](#)

#### Definition

**Definition 8.4.4** (Strict Order on  $\mathbb{Z}$ ). For  $x, y \in \mathbb{Z}$ , define

$$x < y \quad :\Longleftrightarrow \quad y - x \text{ is positive.}$$

*Remark* (Dependencies).

- [Subtraction on  \$\mathbb{Z}\$](#)
- [Positive Integer](#)

#### Definition

**Definition 8.4.5** (Order on  $\mathbb{Z}$ ). For  $x, y \in \mathbb{Z}$ , define

$$x \leq y \quad :\Longleftrightarrow \quad x < y \text{ or } x = y.$$

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)

#### Proposition

**Proposition 8.4.6** (Product of Positives Is Positive). *For all  $x, y \in \mathbb{Z}$ ,*

$$0 < x \wedge 0 < y \implies 0 < x \cdot y.$$

*Remark* (Dependencies).

- [Positive Integer](#)
- [Multiplication on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Order on  \$\mathbb{W}\$](#)

## Canonical Form and Sign

### Theorem

**Theorem 8.4.7** (Canonical Form of Integers). *For every  $x \in \mathbb{Z}$ , exactly one of the following holds:*

$$x = [n, 0] \text{ for some nonzero } n \in \mathbb{W}, \quad x = [0, 0], \quad x = [0, n] \text{ for some nonzero } n \in \mathbb{W}.$$

*Remark* (Dependencies).

- [Integers](#)
- [Integer Class Notation](#)
- [Formal-Difference Equivalence](#)
- [Whole Numbers](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

### Corollary

**Corollary 8.4.8** (Integers Split by Sign). *Every integer is nonnegative or the negative of a nonnegative integer, and the two descriptions overlap only at  $0_{\mathbb{Z}}$ .*

*Remark* (Dependencies).

- [Canonical Form of Integers](#)
- [Positive Integer](#)
- [Zero in  \$\mathbb{Z}\$](#)

### Corollary

**Corollary 8.4.9** (Image of the Embedding Is the Nonnegative Integers). *The image of the canonical embedding  $\mathbb{W} \rightarrow \mathbb{Z}$  is exactly the set of nonnegative integers.*

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Integers Split by Sign](#)
- [Embedding  \$\mathbb{W}\$  into  \$\mathbb{Z}\$  Is Injective](#)

### Theorem

**Theorem 8.4.10** (Sign Trichotomy on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ , exactly one of*

$$x < 0, \quad x = 0, \quad 0 < x$$

*holds.*

*Remark* (Dependencies).

- [Canonical Form of Integers](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

### Theorem

**Theorem 8.4.11** (Trichotomy on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ , exactly one of*

$$x < y, \quad x = y, \quad y < x$$

*holds.*

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)
- [Subtraction on  \$\mathbb{Z}\$](#)
- [Sign Trichotomy on  \$\mathbb{Z}\$](#)

## Total Order on $\mathbb{Z}$

### Theorem

**Theorem 8.4.12** (Strict Order on  $\mathbb{Z}$  Is Irreflexive). *For every  $x \in \mathbb{Z}$ , not  $x < x$ .*

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)
- [Subtraction on  \$\mathbb{Z}\$](#)
- [Zero Is an Integer](#)

### Theorem

**Theorem 8.4.13** (Strict Order on  $\mathbb{Z}$  Is Asymmetric). *For all  $x, y \in \mathbb{Z}$ ,*

$$x < y \implies \neg(y < x).$$

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)
- [Strict Order on  \$\mathbb{Z}\$  Is Irreflexive](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.14** (Strict Order on  $\mathbb{Z}$  Is Transitive). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x < y \wedge y < z \implies x < z.$$

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)
- [Subtraction on  \$\mathbb{Z}\$](#)
- [Positive Integer](#)
- [Addition Preserves Strict Order on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.15** (Order on  $\mathbb{Z}$  Is Reflexive). *For every  $x \in \mathbb{Z}$ ,  $x \leq x$ .*

*Remark* (Dependencies).

- [Order on  \$\mathbb{Z}\$](#)
- [Strict Order on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.16** (Order on  $\mathbb{Z}$  Is Antisymmetric). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \leq y \wedge y \leq x \implies x = y.$$

*Remark* (Dependencies).

- [Order on  \$\mathbb{Z}\$](#)
- [Strict Order on  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.17** (Order on  $\mathbb{Z}$  Is Transitive). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \leq y \wedge y \leq z \implies x \leq z.$$

*Remark* (Dependencies).



- [Order on  \$\mathbb{Z}\$](#)
- [Strict Order on  \$\mathbb{Z}\$](#)
- [Strict Order on  \$\mathbb{Z}\$  Is Transitive](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.18** (Order on  $\mathbb{Z}$  Is Total). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \leq y \text{ or } y \leq x.$$

*Remark* (Dependencies).

- [Order on  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.19** ( $\mathbb{Z}$  Is a Totally Ordered Set). *The ordered pair  $(\mathbb{Z}, \leq)$  is a totally ordered set.*

*Remark* (Dependencies).

- [Ordered Set](#)
- [Total Order](#)
- [Order on  \$\mathbb{Z}\$](#)
- [Order on  \$\mathbb{Z}\$  Is Reflexive](#)
- [Order on  \$\mathbb{Z}\$  Is Antisymmetric](#)
- [Order on  \$\mathbb{Z}\$  Is Transitive](#)
- [Order on  \$\mathbb{Z}\$  Is Total](#)

## Addition Order Compatibility on $\mathbb{Z}$

#### Theorem

**Theorem 8.4.20** (Left Addition Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x < y \implies z + x < z + y.$$

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)

- [Addition on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)

#### Corollary

**Corollary 8.4.21** (Right Addition Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x < y \implies x + z < y + z.$$

*Remark* (Dependencies).

- [Left Addition Preserves Strict Order on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Commutative](#)

#### Theorem

**Theorem 8.4.22** (Addition Preserves Strict Order on  $\mathbb{Z}$ ). *Addition preserves strict order on both sides in  $\mathbb{Z}$ .*

*Remark* (Dependencies).

- [Left Addition Preserves Strict Order on  \$\mathbb{Z}\$](#)
- [Right Addition Preserves Strict Order on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.23** (Left Addition Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \leq y \implies z + x \leq z + y.$$

*Remark* (Dependencies).

- [Order on  \$\mathbb{Z}\$](#)
- [Left Addition Preserves Strict Order on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)

#### Corollary

**Corollary 8.4.24** (Right Addition Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \leq y \implies x + z \leq y + z.$$

*Remark* (Dependencies).

- [Left Addition Preserves Order on  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Commutative](#)

### Theorem

**Theorem 8.4.25** (Addition Preserves Order on  $\mathbb{Z}$ ). *Addition preserves non-strict order on both sides in  $\mathbb{Z}$ .*

*Remark* (Dependencies).

- [Left Addition Preserves Order on  \$\mathbb{Z}\$](#)
- [Right Addition Preserves Order on  \$\mathbb{Z}\$](#)

### Corollary

**Corollary 8.4.26** (Addition Reflects Order on  $\mathbb{Z}$ ). *Addition by a fixed integer reflects both strict and non-strict order.*

*Remark* (Dependencies).

- [Addition Preserves Strict Order on  \$\mathbb{Z}\$](#)
- [Addition Preserves Order on  \$\mathbb{Z}\$](#)
- [Negation Is a Left Additive Inverse on  \$\mathbb{Z}\$](#)

## Multiplication Order Compatibility on $\mathbb{Z}$

### Theorem

**Theorem 8.4.27** (Left Multiplication by Positives Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x < y \implies z \cdot x < z \cdot y.$$

*Remark* (Dependencies).

- [Strict Order on  \$\mathbb{Z}\$](#)
- [Product of Positives Is Positive](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Multiplication Distributes over Addition on  \$\mathbb{Z}\$](#)

### Corollary

**Corollary 8.4.28** (Right Multiplication by Positives Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x < y \implies x \cdot z < y \cdot z.$$

*Remark* (Dependencies).

- [Left Multiplication by Positives Preserves Strict Order on  \$\mathbb{Z}\$](#)

- [Multiplication on  \$\mathbb{Z}\$  Is Commutative](#)

#### Theorem

**Theorem 8.4.29** (Multiplication by Positives Preserves Strict Order on  $\mathbb{Z}$ ). *Multiplication by a positive integer preserves strict order on both sides.*

*Remark* (Dependencies).

- [Left Multiplication by Positives Preserves Strict Order on  \$\mathbb{Z}\$](#)
- [Right Multiplication by Positives Preserves Strict Order on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.30** (Left Multiplication by Positives Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x \leq y \implies z \cdot x \leq z \cdot y.$$

*Remark* (Dependencies).

- [Order on  \$\mathbb{Z}\$](#)
- [Left Multiplication by Positives Preserves Strict Order on  \$\mathbb{Z}\$](#)

#### Corollary

**Corollary 8.4.31** (Right Multiplication by Positives Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x \leq y \implies x \cdot z \leq y \cdot z.$$

*Remark* (Dependencies).

- [Left Multiplication by Positives Preserves Order on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Commutative](#)

#### Theorem

**Theorem 8.4.32** (Multiplication by Positives Preserves Order on  $\mathbb{Z}$ ). *Multiplication by a positive integer preserves non-strict order on both sides.*

*Remark* (Dependencies).

- [Left Multiplication by Positives Preserves Order on  \$\mathbb{Z}\$](#)
- [Right Multiplication by Positives Preserves Order on  \$\mathbb{Z}\$](#)

### Theorem

**Theorem 8.4.33** (Multiplication by Negatives Reverses Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$z < 0 \wedge x < y \implies z \cdot y < z \cdot x.$$

*Remark* (Dependencies).

- [Multiplication by Positives Preserves Strict Order on  \$\mathbb{Z}\$](#)
- [Sign Rule on  \$\mathbb{Z}\$](#)
- [Strict Order on  \$\mathbb{Z}\$  Is Asymmetric](#)

### Corollary

**Corollary 8.4.34** (Multiplication by Negatives Reverses Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$z < 0 \wedge x \leq y \implies z \cdot y \leq z \cdot x.$$

*Remark* (Dependencies).

- [Multiplication by Negatives Reverses Strict Order on  \$\mathbb{Z}\$](#)
- [Order on  \$\mathbb{Z}\$](#)

### Theorem

**Theorem 8.4.35** (Multiplication by Zero Collapses Order on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$0 \cdot x = 0 \cdot y.$$

*Remark* (Dependencies).

- [Zero Absorption on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)

## Absolute Value on $\mathbb{Z}$

### Definition

**Definition 8.4.36** (Absolute Value on  $\mathbb{Z}$ ). Define

$$|x| := \begin{cases} x, & 0 \leq x, \\ -x, & x < 0. \end{cases}$$

*Remark* (Dependencies).

- [Order on  \$\mathbb{Z}\$](#)

- [Trichotomy on  \$\mathbb{Z}\$](#)
- [Negation on  \$\mathbb{Z}\$  Is Well-Defined](#)

#### Theorem

**Theorem 8.4.37** (Absolute Value Is Nonnegative). *For every  $x \in \mathbb{Z}$ ,*

$$0 \leq |x|.$$

*Remark* (Dependencies).

- [Absolute Value on  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.38** (Absolute Value Vanishes Only at Zero). *For every  $x \in \mathbb{Z}$ ,*

$$|x| = 0 \iff x = 0.$$

*Remark* (Dependencies).

- [Absolute Value on  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.39** (Absolute Value Is Symmetric under Negation). *For every  $x \in \mathbb{Z}$ ,*

$$|-x| = |x|.$$

*Remark* (Dependencies).

- [Absolute Value on  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)
- [Sign Rule on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.40** (Absolute Value Is Multiplicative). *For all  $x, y \in \mathbb{Z}$ ,*

$$|x \cdot y| = |x| \cdot |y|.$$

*Remark* (Dependencies).

- [Absolute Value on  \$\mathbb{Z}\$](#)

- [Product of Positives Is Positive](#)
- [Product of Negatives on  \$\mathbb{Z}\$](#)
- [Multiplication by Positives Preserves Order on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.4.41** (Bounding by Absolute Value). *For every  $x \in \mathbb{Z}$ ,*

$$-|x| \leq x \leq |x|.$$

*Remark* (Dependencies).

- [Absolute Value on  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)
- [Absolute Value Is Symmetric under Negation](#)

#### Theorem

**Theorem 8.4.42** (Triangle Inequality on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$|x + y| \leq |x| + |y|.$$

*Remark* (Dependencies).

- [Absolute Value on  \$\mathbb{Z}\$](#)
- [Bounding by Absolute Value](#)
- [Addition Preserves Order on  \$\mathbb{Z}\$](#)

## Well-Ordering Fails on $\mathbb{Z}$

#### Theorem

**Theorem 8.4.43** ( $\mathbb{Z}$  Is Not Well-Ordered). *The ordered set  $(\mathbb{Z}, \leq)$  is not a well-order.*

*Remark* (Dependencies).

- [Well-Ordered Set](#)
- [Order on  \$\mathbb{Z}\$](#)
- [Zero in  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)

## 8.5 Embedding $\mathbb{W}$ into $\mathbb{Z}$

### Embedding $\mathbb{W}$ into $\mathbb{Z}$

#### Definition

**Definition 8.5.1** (Embedding of  $\mathbb{W}$  into  $\mathbb{Z}$ ). Define

$$\iota : \mathbb{W} \rightarrow \mathbb{Z}, \quad \iota(n) := [n, 0].$$

*Remark* (Dependencies).

- [Whole Numbers](#)
- [Integers](#)
- [Integer Class Notation](#)
- [Function](#)

#### Theorem

**Theorem 8.5.2** (Embedding Preserves Zero). *The canonical embedding satisfies*

$$\iota(0_{\mathbb{W}}) = 0_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Zero in  \$\mathbb{Z}\$](#)
- [Whole Numbers](#)

#### Theorem

**Theorem 8.5.3** (Embedding Preserves One). *The canonical embedding satisfies*

$$\iota(1_{\mathbb{W}}) = 1_{\mathbb{Z}}.$$

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)
- [Whole Numbers](#)



### Theorem

**Theorem 8.5.4** (Embedding  $\mathbb{W}$  into  $\mathbb{Z}$  Is Injective). *For all  $a, b \in \mathbb{W}$ ,*

$$\iota(a) = \iota(b) \implies a = b.$$

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Injective Function](#)
- [Formal-Difference Equivalence](#)
- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)

### Theorem

**Theorem 8.5.5** (Embedding Preserves Addition). *For all  $a, b \in \mathbb{W}$ ,*

$$\iota(a + b) = \iota(a) + \iota(b).$$

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{W}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)

### Theorem

**Theorem 8.5.6** (Embedding Preserves Multiplication). *For all  $a, b \in \mathbb{W}$ ,*

$$\iota(a \cdot b) = \iota(a) \cdot \iota(b).$$

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{W}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)

### Theorem

**Theorem 8.5.7** (Embedding Preserves Order). *For all  $a, b \in \mathbb{W}$ ,*

$$a \leq b \iff \iota(a) \leq \iota(b).$$

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Order on  \$\mathbb{W}\$](#)
- [Order on  \$\mathbb{Z}\$](#)

#### Theorem

**Theorem 8.5.8** ( $\mathbb{W}$  Embeds into  $\mathbb{Z}$ ). *The map  $\iota : \mathbb{W} \rightarrow \mathbb{Z}$  is an injective order-preserving semiring homomorphism.*

*Remark* (Dependencies).

- [Embedding of  \$\mathbb{W}\$  into  \$\mathbb{Z}\$](#)
- [Embedding  \$\mathbb{W}\$  into  \$\mathbb{Z}\$  Is Injective](#)
- [Embedding Preserves Addition](#)
- [Embedding Preserves Multiplication](#)
- [Embedding Preserves Order](#)

## 8.6 Divisibility in $\mathbb{Z}$

### Divisibility in $\mathbb{Z}$

#### Definition

**Definition 8.6.1** (Divisibility in  $\mathbb{Z}$ ). For  $a, b \in \mathbb{Z}$ , define

$$a \mid b \quad :\Longleftrightarrow \quad \exists c \in \mathbb{Z} (b = a \cdot c).$$

*Remark* (Dependencies).

- [Integers](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Existential Formula](#)

### Basic Divisibility Laws

#### Proposition

**Proposition 8.6.2** (Divisibility Is Reflexive on  $\mathbb{Z}$ ). *For every  $a \in \mathbb{Z}$ ,*

$$a \mid a.$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)
- [\$\mathbb{Z}\$  Multiplicative Commutative Monoid](#)

#### Proposition

**Proposition 8.6.3** (Divisibility Is Transitive on  $\mathbb{Z}$ ). *For all  $a, b, c \in \mathbb{Z}$ ,*

$$a \mid b \wedge b \mid c \implies a \mid c.$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Associative](#)

#### Proposition

**Proposition 8.6.4** (One and Negative One Divide Every Integer). *For every  $a \in \mathbb{Z}$ ,*

$$1 \mid a \quad \text{and} \quad (-1) \mid a.$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [One in  \$\mathbb{Z}\$](#)
- [Negative One Rule on  \$\mathbb{Z}\$](#)

#### Proposition

**Proposition 8.6.5** (Every Integer Divides Zero). *For every  $a \in \mathbb{Z}$ ,*

$$a \mid 0.$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Zero Absorption on  \$\mathbb{Z}\$](#)

#### Proposition

**Proposition 8.6.6** (Divisibility Is Compatible with Negation on the Divisor). *For all  $a, b \in \mathbb{Z}$ ,*

$$a \mid b \iff (-a) \mid b.$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Sign Rule on  \$\mathbb{Z}\$](#)

#### Proposition

**Proposition 8.6.7** (Divisibility Is Compatible with Negation on the Dividend). *For all  $a, b \in \mathbb{Z}$ ,*

$$a \mid b \iff a \mid (-b).$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Sign Rule on  \$\mathbb{Z}\$](#)

#### Proposition

**Proposition 8.6.8** (Divisibility Is Compatible with Negation on  $\mathbb{Z}$ ). *For all  $a, b \in \mathbb{Z}$ ,*

$$a \mid b \iff (-a) \mid b \iff a \mid (-b).$$

*Remark* (Dependencies).

- [Divisibility Is Compatible with Negation on the Divisor](#)
- [Divisibility Is Compatible with Negation on the Dividend](#)

#### Proposition

**Proposition 8.6.9** (Divisibility Is Compatible with Addition on  $\mathbb{Z}$ ). *For all  $a, b, c \in \mathbb{Z}$ ,*

$$a \mid b \wedge a \mid c \implies a \mid (b + c).$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Addition on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Multiplication Distributes over Addition on  \$\mathbb{Z}\$](#)

#### Proposition

**Proposition 8.6.10** (Divisibility Respects Linear Combinations on  $\mathbb{Z}$ ). *For all  $a, b, c, x, y \in \mathbb{Z}$ ,*

$$a \mid b \wedge a \mid c \implies a \mid (b \cdot x + c \cdot y).$$

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Divisibility Is Compatible with Addition on  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Associative](#)
- [Multiplication Distributes over Addition on  \$\mathbb{Z}\$](#)

## Units in $\mathbb{Z}$

### Definition

**Definition 8.6.11** (Unit in  $\mathbb{Z}$ ). An integer  $u \in \mathbb{Z}$  is a *unit* if

$$\exists v \in \mathbb{Z} (u \cdot v = 1).$$

*Remark* (Dependencies).

- [Integers](#)
- [One in  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)
- [Existential Formula](#)

### Theorem

**Theorem 8.6.12** (The Units of  $\mathbb{Z}$  Are Exactly  $\pm 1$ ). *An integer  $u$  is a unit if and only if*

$$u = 1 \quad \text{or} \quad u = -1.$$

*Remark* (Dependencies).

- [Unit in  \$\mathbb{Z}\$](#)
- [\$\mathbb{Z}\$  Is an Integral Domain](#)
- [Trichotomy on  \$\mathbb{Z}\$](#)
- [Negative One Rule on  \$\mathbb{Z}\$](#)

### Definition

**Definition 8.6.13** (Associates in  $\mathbb{Z}$ ). Integers  $a, b \in \mathbb{Z}$  are *associates* if  $a = u \cdot b$  for some unit  $u \in \mathbb{Z}$ .

*Remark* (Dependencies).

- [Unit in  \$\mathbb{Z}\$](#)
- [Multiplication on  \$\mathbb{Z}\$  Is Well-Defined](#)

## Division Algorithm

### Theorem

**Theorem 8.6.14** (Division Algorithm on  $\mathbb{Z}$ ). *If  $a, b \in \mathbb{Z}$  and  $b \neq 0$ , then there exist unique  $q, r \in \mathbb{Z}$  such that*

$$a = bq + r \quad \text{and} \quad 0 \leq r < |b|.$$

*Remark* (Dependencies).

- [Well-Ordering Principle](#)
- [Multiplicative Cancellation on  \$\mathbb{Z}\$](#)
- [\$\mathbb{Z}\$  Is Not Well-Ordered](#)

## Greatest Common Divisor

### Definition

**Definition 8.6.15** (Greatest Common Divisor in  $\mathbb{Z}$ ). A greatest common divisor  $\gcd(a, b)$  is the positive common divisor of  $a$  and  $b$  divisible by every other common divisor.

*Remark* (Dependencies).

- [Divisibility in  \$\mathbb{Z}\$](#)
- [Positive Integer](#)
- [Order on  \$\mathbb{Z}\$](#)

## Bezout Identity

### Theorem

**Theorem 8.6.16** (Bezout Identity on  $\mathbb{Z}$ ). *For  $a, b \in \mathbb{Z}$ , there exist  $x, y \in \mathbb{Z}$  such that*

$$\gcd(a, b) = a \cdot x + b \cdot y.$$

*Remark* (Dependencies).

- [Well-Ordering Principle](#)

## 8.7 Transition to $\mathbb{Q}$

*Remark* (Why Rational Numbers Are Introduced). The integers have additive inverses, but they do not have multiplicative inverses for every nonzero element. The rational numbers are introduced by adjoining those missing inverses through equivalence classes of integer pairs with nonzero second coordinate.

# Proofs

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Formal-Difference Equivalence Is Reflexive). *For every  $(a, b) \in \text{Pre } \mathbb{Z}$ ,*

$$(a, b) \sim_{\mathbb{Z}} (a, b).$$

*Professional Standard Proof.*

TODO: professional standard proof for formal-difference-equivalence-reflexive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for formal-difference-equivalence-reflexive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Lemma** (Formal-Difference Equivalence Is Symmetric). *For all prepairs  $(a, b), (c, d)$ ,*

$$(a, b) \sim_{\mathbb{Z}} (c, d) \implies (c, d) \sim_{\mathbb{Z}} (a, b).$$

*Professional Standard Proof.*

TODO: professional standard proof for formal-difference-equivalence-symmetric. □

*Detailed Learning Proof.*

TODO: detailed learning proof for formal-difference-equivalence-symmetric. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Formal-Difference Equivalence Is Transitive). *For all prepairs  $(a, b), (c, d), (e, f)$ ,*

$$(a, b) \sim_{\mathbb{Z}} (c, d) \wedge (c, d) \sim_{\mathbb{Z}} (e, f) \implies (a, b) \sim_{\mathbb{Z}} (e, f).$$

*Professional Standard Proof.*

TODO: professional standard proof for formal-difference-equivalence-transitive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for formal-difference-equivalence-transitive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Formal-Difference Equivalence Is an Equivalence Relation). *The relation  $\sim_{\mathbb{Z}}$  is an equivalence relation on  $\text{Pre}\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for formal-difference-equivalence-relation. □

*Detailed Learning Proof.*

TODO: detailed learning proof for formal-difference-equivalence-relation. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is an Integer). *The class  $0_{\mathbb{Z}}$  is an element of  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for zero-is-an-integer. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-is-an-integer. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (One Is an Integer). *The class  $1_{\mathbb{Z}}$  is an element of  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for one-is-an-integer. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-is-an-integer. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is Not One in  $\mathbb{Z}$ ). *In  $\mathbb{Z}$ ,*

$$0_{\mathbb{Z}} \neq 1_{\mathbb{Z}}.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-not-one-in-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-not-one-in-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [0  \$\notin \mathbb{N}\$](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Addition Respects Formal-Difference Equivalence on the Left). *If  $[a, b] = [a', b']$ , then*

$$[a, b] + [c, d] = [a', b'] + [c, d].$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-z-classes-respects-left-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-z-classes-respects-left-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Addition Respects Formal-Difference Equivalence on the Right). *If  $[c, d] = [c', d']$ , then*

$$[a, b] + [c, d] = [a, b] + [c', d'].$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-z-classes-respects-right-equivalence.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-z-classes-respects-right-equivalence.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Lemma** (Addition Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$  and  $[c, d] = [c', d']$ , then*

$$[a, b] + [c, d] = [a', b'] + [c', d'].$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-z-classes-respects-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-z-classes-respects-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $\mathbb{Z}$  Is Well-Defined). *Addition determines a well-defined binary operation  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-z-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-z-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $\mathbb{Z}$  Is Associative). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$(x + y) + z = x + (y + z).$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-z-associative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-z-associative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition on  $\mathbb{Z}$  Is Commutative). *For all  $x, y \in \mathbb{Z}$ ,*

$$x + y = y + x.$$

*Professional Standard Proof.*

TODO: professional standard proof for addition-on-z-commutative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-on-z-commutative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is a Left Additive Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$0_{\mathbb{Z}} + x = x.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-left-additive-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-left-additive-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Zero Is a Right Additive Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$x + 0_{\mathbb{Z}} = x.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-right-additive-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-right-additive-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Zero Is an Additive Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$0_{\mathbb{Z}} + x = x \quad \text{and} \quad x + 0_{\mathbb{Z}} = x.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-additive-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-additive-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Negation Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$ , then*

$$-[a, b] = -[a', b'].$$

*Professional Standard Proof.*

TODO: professional standard proof for negation-on-z-classes-respects-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negation-on-z-classes-respects-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Negation on  $\mathbb{Z}$  Is Well-Defined). *Negation determines a well-defined unary operation  $\mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for negation-on-z-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negation-on-z-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Negation Gives a Left Additive Inverse on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$(-x) + x = 0_{\mathbb{Z}}.$$

*Professional Standard Proof.*

TODO: professional standard proof for negation-left-additive-inverse-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negation-left-additive-inverse-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Negation Gives a Right Additive Inverse on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$x + (-x) = 0_{\mathbb{Z}}.$$

*Professional Standard Proof.*

TODO: professional standard proof for negation-right-additive-inverse-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negation-right-additive-inverse-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Every Integer Has an Additive Inverse). *For every  $x \in \mathbb{Z}$  there exists  $y \in \mathbb{Z}$  such that*

$$x + y = 0_{\mathbb{Z}} \quad \text{and} \quad y + x = 0_{\mathbb{Z}}.$$

*Professional Standard Proof.*

TODO: professional standard proof for every-integer-has-additive-inverse. □

*Detailed Learning Proof.*

TODO: detailed learning proof for every-integer-has-additive-inverse. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Subtraction on  $\mathbb{Z}$  Is Well-Defined). *Subtraction determines a well-defined binary operation  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for subtraction-on-z-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for subtraction-on-z-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Multiplication Respects Formal-Difference Equivalence on the Left). *If  $[a, b] = [a', b']$ , then*

$$[a, b] \cdot [c, d] = [a', b'] \cdot [c, d].$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-z-classes-respects-left-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-z-classes-respects-left-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Multiplication Respects Formal-Difference Equivalence on the Right). *If  $[c, d] = [c', d']$ , then*

$$[a, b] \cdot [c, d] = [a, b] \cdot [c', d'].$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-z-classes-respects-right-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-z-classes-respects-right-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Multiplication Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$  and  $[c, d] = [c', d']$ , then*

$$[a, b] \cdot [c, d] = [a', b'] \cdot [c', d'].$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-z-classes-respects-equivalence.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-z-classes-respects-equivalence.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{Z}$  Is Well-Defined). *Multiplication determines a well-defined binary operation  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-z-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-z-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{Z}$  Is Associative). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$(x \cdot y) \cdot z = x \cdot (y \cdot z).$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-z-associative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-z-associative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication on  $\mathbb{Z}$  Is Commutative). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \cdot y = y \cdot x.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-on-z-commutative. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-on-z-commutative. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (One Is a Left Multiplicative Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$1_{\mathbb{Z}} \cdot x = x.$$

*Professional Standard Proof.*

TODO: professional standard proof for one-left-multiplicative-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-left-multiplicative-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (One Is a Right Multiplicative Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$x \cdot 1_{\mathbb{Z}} = x.$$

*Professional Standard Proof.*

TODO: professional standard proof for one-right-multiplicative-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-right-multiplicative-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (One Is a Multiplicative Identity on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$1_{\mathbb{Z}} \cdot x = x \quad \text{and} \quad x \cdot 1_{\mathbb{Z}} = x.$$

*Professional Standard Proof.*

TODO: professional standard proof for one-multiplicative-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for one-multiplicative-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Distributivity on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \cdot (y + z) = x \cdot y + x \cdot z.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-distributivity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-distributivity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Right Distributivity on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$(y + z) \cdot x = y \cdot x + z \cdot x.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-distributivity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-distributivity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication Distributes over Addition on  $\mathbb{Z}$ ). *Multiplication distributes over addition on both sides in  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for multiplication-distributes-over-addition-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplication-distributes-over-addition-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}$  Is an Additive Abelian Group). *The structure  $(\mathbb{Z}, +, 0_{\mathbb{Z}})$  is an abelian group.*

*Professional Standard Proof.*

TODO: professional standard proof for  $\mathbb{Z}$ -additive-abelian-group. □

*Detailed Learning Proof.*

TODO: detailed learning proof for  $\mathbb{Z}$ -additive-abelian-group. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}$  Is a Multiplicative Commutative Monoid). *The structure  $(\mathbb{Z}, \cdot, 1_{\mathbb{Z}})$  is a commutative monoid.*

*Professional Standard Proof.*

TODO: professional standard proof for z-multiplicative-commutative-monoid. □

*Detailed Learning Proof.*

TODO: detailed learning proof for z-multiplicative-commutative-monoid. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}$  Is a Commutative Ring). *The structure  $(\mathbb{Z}, +, \cdot, 0_{\mathbb{Z}}, 1_{\mathbb{Z}})$  is a commutative ring.*

*Professional Standard Proof.*

TODO: professional standard proof for  $\mathbb{Z}$ -commutative-ring. □

*Detailed Learning Proof.*

TODO: detailed learning proof for  $\mathbb{Z}$ -commutative-ring. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}$  Has No Zero Divisors). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \cdot y = 0 \implies x = 0 \text{ or } y = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for  $\mathbb{Z}$ -has-no-zero-divisors. □

*Detailed Learning Proof.*

TODO: detailed learning proof for  $\mathbb{Z}$ -has-no-zero-divisors. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Order on  \$\mathbb{W}\$  Is Total](#)
- Multiplicative cancellation inherited from the positive natural-number multiplication theory.

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}$  Is an Integral Domain). *The integers form an integral domain:  $\mathbb{Z}$  is a commutative ring,  $0 \neq 1$ , and  $\mathbb{Z}$  has no zero divisors.*

*Professional Standard Proof.*

TODO: professional standard proof for  $\mathbb{Z}$ -integral-domain. □

*Detailed Learning Proof.*

TODO: detailed learning proof for  $\mathbb{Z}$ -integral-domain. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [ℤ Is a Commutative Ring](#)
- [Zero Is Not One in ℤ](#)
- [ℤ Has No Zero Divisors](#)

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Multiplicative Cancellation on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$z \neq 0 \wedge z \cdot x = z \cdot y \implies x = y.$$

*Professional Standard Proof.*

TODO: professional standard proof for multiplicative-cancellation-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for multiplicative-cancellation-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Zero Absorption on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$0 \cdot x = 0 \quad \text{and} \quad x \cdot 0 = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-absorption-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-absorption-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Corollary** (Sign Rule on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$(-x) \cdot y = -(x \cdot y) \quad \text{and} \quad x \cdot (-y) = -(x \cdot y).$$

*Professional Standard Proof.*

TODO: professional standard proof for sign-rule-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for sign-rule-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Negative One Rule on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ ,*

$$(-1) \cdot x = -x.$$

*Professional Standard Proof.*

TODO: professional standard proof for negative-one-rule-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negative-one-rule-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Product of Negatives on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$(-x) \cdot (-y) = x \cdot y.$$

*Professional Standard Proof.*

TODO: professional standard proof for product-of-negatives-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for product-of-negatives-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Lemma** (Positivity Respects Formal-Difference Equivalence). *If  $[a, b] = [a', b']$ , then*

$$b < a \iff b' < a'.$$

*Professional Standard Proof.*

TODO: professional standard proof for positivity-respects-equivalence. □

*Detailed Learning Proof.*

TODO: detailed learning proof for positivity-respects-equivalence. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Addition on  \$\mathbb{W}\$  Satisfies Cancellation](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Positivity on  $\mathbb{Z}$  Is Well-Defined). *The predicate “ $x$  is positive” is well-defined on equivalence classes in  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for positivity-on-z-well-defined. □

*Detailed Learning Proof.*

TODO: detailed learning proof for positivity-on-z-well-defined. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Product of Positives Is Positive). *For all  $x, y \in \mathbb{Z}$ ,*

$$0 < x \wedge 0 < y \implies 0 < x \cdot y.$$

*Professional Standard Proof.*

TODO: professional standard proof for product-of-positives-is-positive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for product-of-positives-is-positive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Canonical Form of Integers). *For every  $x \in \mathbb{Z}$ , exactly one of the following holds:*

$x = [n, 0]$  for some nonzero  $n \in \mathbb{W}$ ,       $x = [0, 0]$ ,       $x = [0, n]$  for some nonzero  $n \in \mathbb{W}$ .

*Professional Standard Proof.*

TODO: professional standard proof for canonical-form-of-integers. □

*Detailed Learning Proof.*

TODO: detailed learning proof for canonical-form-of-integers. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Integers Split by Sign). *Every integer is nonnegative or the negative of a non-negative integer, and the two descriptions overlap only at  $0_{\mathbb{Z}}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for integers-split-by-sign. □

*Detailed Learning Proof.*

TODO: detailed learning proof for integers-split-by-sign. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Corollary** (Image of the Embedding Is the Nonnegative Integers). *The image of the canonical embedding  $\mathbb{W} \rightarrow \mathbb{Z}$  is exactly the set of nonnegative integers.*

*Professional Standard Proof.*

TODO: professional standard proof for image-of-embedding-is-nonnegatives. □

*Detailed Learning Proof.*

TODO: detailed learning proof for image-of-embedding-is-nonnegatives. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Sign Trichotomy on  $\mathbb{Z}$ ). *For every  $x \in \mathbb{Z}$ , exactly one of*

$$x < 0, \quad x = 0, \quad 0 < x$$

*holds.*

*Professional Standard Proof.*

TODO: professional standard proof for sign-trichotomy-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for sign-trichotomy-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Canonical Form of Integers](#)
- [Order on  \$\mathbb{W}\$  Is Total](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Trichotomy on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ , exactly one of*

$$x < y, \quad x = y, \quad y < x$$

*holds.*

*Professional Standard Proof.*

TODO: professional standard proof for trichotomy-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for trichotomy-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Strict Order on  $\mathbb{Z}$  Is Irreflexive). *For every  $x \in \mathbb{Z}$ , not  $x < x$ .*

*Professional Standard Proof.*

TODO: professional standard proof for strict-order-on-z-irreflexive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for strict-order-on-z-irreflexive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Strict Order on  $\mathbb{Z}$  Is Asymmetric). *For all  $x, y \in \mathbb{Z}$ ,*

$$x < y \implies \neg(y < x).$$

*Professional Standard Proof.*

TODO: professional standard proof for strict-order-on-z-asymmetric. □

*Detailed Learning Proof.*

TODO: detailed learning proof for strict-order-on-z-asymmetric. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Strict Order on  $\mathbb{Z}$  Is Transitive). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x < y \wedge y < z \implies x < z.$$

*Professional Standard Proof.*

TODO: professional standard proof for strict-order-on-z-transitive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for strict-order-on-z-transitive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $Z$  Is Reflexive). *For every  $x \in \mathbb{Z}$ ,  $x \leq x$ .*

*Professional Standard Proof.*

TODO: professional standard proof for order-on-z-reflexive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-z-reflexive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $\mathbb{Z}$  Is Antisymmetric). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \leq y \wedge y \leq x \implies x = y.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-z-antisymmetric. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-z-antisymmetric. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $\mathbb{Z}$  Is Transitive). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \leq y \wedge y \leq z \implies x \leq z.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-z-transitive. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-z-transitive. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Order on  $Z$  Is Total). *For all  $x, y \in \mathbb{Z}$ ,*

$$x \leq y \text{ or } y \leq x.$$

*Professional Standard Proof.*

TODO: professional standard proof for order-on-z-total. □

*Detailed Learning Proof.*

TODO: detailed learning proof for order-on-z-total. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** ( $\mathbb{Z}$  Is a Totally Ordered Set). *The ordered pair  $(\mathbb{Z}, \leq)$  is a totally ordered set.*

*Professional Standard Proof.*

TODO: professional standard proof for  $\mathbb{Z}$ -totally-ordered-set. □

*Detailed Learning Proof.*

TODO: detailed learning proof for  $\mathbb{Z}$ -totally-ordered-set. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Addition Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x < y \implies z + x < z + y.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-addition-preserves-strict-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-addition-preserves-strict-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Right Addition Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x < y \implies x + z < y + z.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-addition-preserves-strict-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-addition-preserves-strict-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Preserves Strict Order on  $\mathbb{Z}$ ). *Addition preserves strict order on both sides in  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for addition-preserves-strict-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-preserves-strict-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Addition Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \leq y \implies z + x \leq z + y.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-addition-preserves-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-addition-preserves-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Right Addition Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$x \leq y \implies x + z \leq y + z.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-addition-preserves-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-addition-preserves-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Addition Preserves Order on  $\mathbb{Z}$ ). *Addition preserves non-strict order on both sides in  $\mathbb{Z}$ .*

*Professional Standard Proof.*

TODO: professional standard proof for addition-preserves-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-preserves-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Addition Reflects Order on  $\mathbb{Z}$ ). *Addition by a fixed integer reflects both strict and non-strict order.*

*Professional Standard Proof.*

TODO: professional standard proof for addition-reflects-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for addition-reflects-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Multiplication by Positives Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x < y \implies z \cdot x < z \cdot y.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-positive-multiplication-preserves-strict-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for left-positive-multiplication-preserves-strict-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Right Multiplication by Positives Preserves Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x < y \implies x \cdot z < y \cdot z.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-positive-multiplication-preserves-strict-order-on- $\mathbb{Z}$ . □

*Detailed Learning Proof.*

TODO: detailed learning proof for right-positive-multiplication-preserves-strict-order-on- $\mathbb{Z}$ . □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication by Positives Preserves Strict Order on  $\mathbb{Z}$ ). *Multiplication by a positive integer preserves strict order on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for positive-multiplication-preserves-strict-order-on-z.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for positive-multiplication-preserves-strict-order-on-z.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Left Multiplication by Positives Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x \leq y \implies z \cdot x \leq z \cdot y.$$

*Professional Standard Proof.*

TODO: professional standard proof for left-positive-multiplication-preserves-order-on-z.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for left-positive-multiplication-preserves-order-on-z.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Right Multiplication by Positives Preserves Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$0 < z \wedge x \leq y \implies x \cdot z \leq y \cdot z.$$

*Professional Standard Proof.*

TODO: professional standard proof for right-positive-multiplication-preserves-order-on-z.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for right-positive-multiplication-preserves-order-on-z.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication by Positives Preserves Order on  $\mathbb{Z}$ ). *Multiplication by a positive integer preserves non-strict order on both sides.*

*Professional Standard Proof.*

TODO: professional standard proof for positive-multiplication-preserves-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for positive-multiplication-preserves-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication by Negatives Reverses Strict Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$z < 0 \wedge x < y \implies z \cdot y < z \cdot x.$$

*Professional Standard Proof.*

TODO: professional standard proof for negative-multiplication-reverses-strict-order-on-z.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for negative-multiplication-reverses-strict-order-on-z.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Corollary** (Multiplication by Negatives Reverses Order on  $\mathbb{Z}$ ). *For all  $x, y, z \in \mathbb{Z}$ ,*

$$z < 0 \wedge x \leq y \implies z \cdot y \leq z \cdot x.$$

*Professional Standard Proof.*

TODO: professional standard proof for negative-multiplication-reverses-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for negative-multiplication-reverses-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Multiplication by Zero Collapses Order on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$0 \cdot x = 0 \cdot y.$$

*Professional Standard Proof.*

TODO: professional standard proof for zero-multiplication-collapses-order-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for zero-multiplication-collapses-order-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Absolute Value Is Nonnegative). *For every  $x \in \mathbb{Z}$ ,*

$$0 \leq |x|.$$

*Professional Standard Proof.*

TODO: professional standard proof for absolute-value-nonnegative-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for absolute-value-nonnegative-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Absolute Value Vanishes Only at Zero). *For every  $x \in \mathbb{Z}$ ,*

$$|x| = 0 \iff x = 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for absolute-value-zero-iff-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for absolute-value-zero-iff-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Absolute Value Is Symmetric under Negation). *For every  $x \in \mathbb{Z}$ ,*

$$|-x| = |x|.$$

*Professional Standard Proof.*

TODO: professional standard proof for absolute-value-negation-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for absolute-value-negation-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Absolute Value Is Multiplicative). *For all  $x, y \in \mathbb{Z}$ ,*

$$|x \cdot y| = |x| \cdot |y|.$$

*Professional Standard Proof.*

TODO: professional standard proof for absolute-value-multiplicative-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for absolute-value-multiplicative-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Bounding by Absolute Value). *For every  $x \in \mathbb{Z}$ ,*

$$-|x| \leq x \leq |x|.$$

*Professional Standard Proof.*

TODO: professional standard proof for absolute-value-bounds-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for absolute-value-bounds-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (Triangle Inequality on  $\mathbb{Z}$ ). *For all  $x, y \in \mathbb{Z}$ ,*

$$|x + y| \leq |x| + |y|.$$

*Professional Standard Proof.*

TODO: professional standard proof for triangle-inequality-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for triangle-inequality-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (*Z Is Not Well-Ordered*). *The ordered set  $(\mathbb{Z}, \leq)$  is not a well-order.*

*Professional Standard Proof.*

TODO: professional standard proof for z-is-not-well-ordered. □

*Detailed Learning Proof.*

TODO: detailed learning proof for z-is-not-well-ordered. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Embedding Preserves Zero). *The canonical embedding satisfies*

$$\iota(0_{\mathbb{W}}) = 0_{\mathbb{Z}}.$$

*Professional Standard Proof.*

TODO: professional standard proof for embedding-w-into-z-preserves-zero. □

*Detailed Learning Proof.*

TODO: detailed learning proof for embedding-w-into-z-preserves-zero. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Embedding Preserves One). *The canonical embedding satisfies*

$$\iota(1_{\mathbb{W}}) = 1_{\mathbb{Z}}.$$

*Professional Standard Proof.*

TODO: professional standard proof for embedding-w-into-z-preserves-one. □

*Detailed Learning Proof.*

TODO: detailed learning proof for embedding-w-into-z-preserves-one. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Embedding  $W$  into  $Z$  Is Injective). *For all  $a, b \in \mathbb{W}$ ,*

$$\iota(a) = \iota(b) \implies a = b.$$

*Professional Standard Proof.*

TODO: professional standard proof for embedding-w-into-z-injective. □

*Detailed Learning Proof.*

TODO: detailed learning proof for embedding-w-into-z-injective. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Embedding Preserves Addition). *For all  $a, b \in \mathbb{W}$ ,*

$$\iota(a + b) = \iota(a) + \iota(b).$$

*Professional Standard Proof.*

TODO: professional standard proof for embedding-w-into-z-preserves-addition. □

*Detailed Learning Proof.*

TODO: detailed learning proof for embedding-w-into-z-preserves-addition. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Embedding Preserves Multiplication). *For all  $a, b \in \mathbb{W}$ ,*

$$\iota(a \cdot b) = \iota(a) \cdot \iota(b).$$

*Professional Standard Proof.*

TODO: professional standard proof for embedding-w-into-z-preserves-multiplication. □

*Detailed Learning Proof.*

TODO: detailed learning proof for embedding-w-into-z-preserves-multiplication. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Embedding Preserves Order). *For all  $a, b \in \mathbb{W}$ ,*

$$a \leq b \iff \iota(a) \leq \iota(b).$$

*Professional Standard Proof.*

TODO: professional standard proof for embedding-w-into-z-preserves-order. □

*Detailed Learning Proof.*

TODO: detailed learning proof for embedding-w-into-z-preserves-order. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Theorem** (*W Embeds into Z*). *The map  $\iota : \mathbb{W} \rightarrow \mathbb{Z}$  is an injective order-preserving semiring homomorphism.*

*Professional Standard Proof.*

TODO: professional standard proof for w-embeds-into-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for w-embeds-into-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Is Reflexive on  $\mathbb{Z}$ ). *For every  $a \in \mathbb{Z}$ ,*

$$a \mid a.$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-reflexive-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-reflexive-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Is Transitive on  $\mathbb{Z}$ ). *For all  $a, b, c \in \mathbb{Z}$ ,*

$$a \mid b \wedge b \mid c \implies a \mid c.$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-transitive-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-transitive-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (One and Negative One Divide Every Integer). *For every  $a \in \mathbb{Z}$ ,*

$$1 \mid a \quad \text{and} \quad (-1) \mid a.$$

*Professional Standard Proof.*

TODO: professional standard proof for units-divide-everything-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for units-divide-everything-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Every Integer Divides Zero). *For every  $a \in \mathbb{Z}$ ,*

$$a \mid 0.$$

*Professional Standard Proof.*

TODO: professional standard proof for everything-divides-zero-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for everything-divides-zero-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Is Compatible with Negation on the Divisor). *For all  $a, b \in \mathbb{Z}$ ,*

$$a \mid b \iff (-a) \mid b.$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-compatible-with-negation-divisor-on-z.  $\square$

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-compatible-with-negation-divisor-on-z.  $\square$

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Is Compatible with Negation on the Dividend). *For all  $a, b \in \mathbb{Z}$ ,*

$$a \mid b \iff a \mid (-b).$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-compatible-with-negation-dividend-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-compatible-with-negation-dividend-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Is Compatible with Negation on  $\mathbb{Z}$ ). *For all  $a, b \in \mathbb{Z}$ ,*

$$a \mid b \iff (-a) \mid b \iff a \mid (-b).$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-compatible-with-negation-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-compatible-with-negation-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.



*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Is Compatible with Addition on  $\mathbb{Z}$ ). *For all  $a, b, c \in \mathbb{Z}$ ,*

$$a \mid b \wedge a \mid c \implies a \mid (b + c).$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-compatible-with-addition-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-compatible-with-addition-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Proposition** (Divisibility Respects Linear Combinations on  $\mathbb{Z}$ ). *For all  $a, b, c, x, y \in \mathbb{Z}$ ,*

$$a \mid b \wedge a \mid c \implies a \mid (b \cdot x + c \cdot y).$$

*Professional Standard Proof.*

TODO: professional standard proof for divisibility-respects-linear-combinations-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for divisibility-respects-linear-combinations-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (The Units of  $Z$  Are Exactly 1). *An integer  $u$  is a unit if and only if*

$$u = 1 \quad \text{or} \quad u = -1.$$

*Professional Standard Proof.*

TODO: professional standard proof for units-of-z-are-pm-one. □

*Detailed Learning Proof.*

TODO: detailed learning proof for units-of-z-are-pm-one. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- TODO: list mathematical dependencies.

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Division Algorithm on  $\mathbb{Z}$ ). *If  $a, b \in \mathbb{Z}$  and  $b \neq 0$ , then there exist unique  $q, r \in \mathbb{Z}$  such that*

$$a = bq + r \quad \text{and} \quad 0 \leq r < |b|.$$

*Professional Standard Proof.*

TODO: professional standard proof for division-algorithm-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for division-algorithm-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Well-Ordering Principle](#)
- [Multiplicative Cancellation on  \$\mathbb{Z}\$](#)
- [\$\mathbb{Z}\$  Is Not Well-Ordered](#)

*Remark* (Return). [Return to Theorem](#)

**Theorem** (Bezout Identity on  $\mathbb{Z}$ ). *For  $a, b \in \mathbb{Z}$ , there exist  $x, y \in \mathbb{Z}$  such that*

$$\gcd(a, b) = a \cdot x + b \cdot y.$$

*Professional Standard Proof.*

TODO: professional standard proof for bezout-identity-on-z. □

*Detailed Learning Proof.*

TODO: detailed learning proof for bezout-identity-on-z. □

*Remark* (Proof structure).

*Remark* (Dependencies).

- [Well-Ordering Principle](#)

# Capstone

# Bibliography

- Richard Dedekind. *Essays on the Theory of Numbers*. Open Court, Chicago, 1901. URL <https://www.gutenberg.org/ebooks/21016>. Project Gutenberg eBook #21016; contains *Continuity and Irrational Numbers* and *The Nature and Meaning of Numbers*.
- Solomon Feferman. *The Number Systems: Foundations of Algebra and Analysis*. Addison-Wesley Publishing Company, Reading, MA, 1964. URL <https://archive.org/details/numbersystemsfo0000fefe>. Internet Archive scan.
- Ronald C. Freiwald. Peano systems and the whole number system. Washington University in St. Louis course notes, 2009. URL <https://www.math.wustl.edu/~freiwald/310peanosys.pdf>.
- Karel Hrbacek and Thomas Jech. *Introduction to Set Theory*. Marcel Dekker, New York, 3 edition, 1999. ISBN 0824779150.
- F. William Lawvere. Adjointness in foundations. *Dialectica*, 23(3–4):281–296, 1969. doi: 10.1111/j.1746-8361.1969.tb01194.x.
- Elliott Mendelson. *Number Systems and the Foundations of Analysis*. Robert E. Krieger Publishing Company, Malabar, FL, 1982. Reprint of the 1973 Academic Press edition.
- Giuseppe Peano. *Arithmetices principia, nova methodo exposita*. Fratres Bocca, Turin, 1889. Latin title commonly translated as *The Principles of Arithmetic, Presented by a New Method*.
- Stephen G. Simpson. *Subsystems of Second Order Arithmetic*. Perspectives in Logic. Cambridge University Press, Cambridge, 2 edition, 2009.
- H. A. Thurston. *The Number System*. Blackie and Son, London and Glasgow, 1956.